Universidade Federal de Ouro Preto - UFOP
Instituto de Ciências Exatas e Biológicas - ICEB
Departamento de Computação - DECOM
Ciência da Computação

# Trabalho

PCC104 - Projeto e Análise de Algoritmos

Henrique Ulhôa de Faria
Professor: Pedro Silva

Ouro Preto
13 de novembro de 2026

# Sumário


**1 Introdução** **1**
- 1.1 Especificações do problema . . . 1
- 1.2 Considerações iniciais . . . 1
- 1.3 Ferramentas utilizadas . . . 1
- 1.4 Especificações da máquina . . . 1

**2 Metodologia** **2**
- 2.1 Análise dos enunciados . . . 2
- 2.2 Projeto e implementação dos algoritmos . . . 2
- 2.3 Análise de complexidade . . . 2
- 2.4 Testes e validação . . . 2
- 2.5 Organização e documentação . . . 2

**3 Comandos para Execução dos Algoritmos** **3**
- 3.1 Tarefa 1 – Multiplicação de Matrizes . . . 3
- 3.2 Tarefa 2 – Dijkstra utilizando Vetores . . . 3
- 3.3 Tarefa 3 – Dijkstra utilizando Listas Encadeadas . . . 5
- 3.4 Tarefa 4 – Dijkstra utilizando Heap Binário . . . 6
- 3.5 Tarefa 5 – Busca Completa . . . 8
- 3.6 Tarefa 6 – Backtracking com Podas . . . 9
- 3.7 Tarefa 7 – Algoritmo Guloso para Caminhos Mínimos . . . 11
- 3.8 Tarefa 8 – Simulated Annealing . . . 12
- 3.9 Tarefa 9 – Algoritmo de Floyd–Warshall . . . 14

**4 Tarefa 1 – Multiplicação de Matrizes** **16**
- 4.1 Descrição do problema . . . 16
- 4.2 Estratégia de solução . . . 16
  - 4.2.1 Tratamento de matrizes de ordem arbitrária . . . 17
- 4.3 Implementação . . . 17
  - 4.3.1 Estrutura de dados . . . 17
  - 4.3.2 Operações básicas . . . 18
  - 4.3.3 Funções auxiliares . . . 18
  - 4.3.4 Multiplicação recursiva . . . 18
  - 4.3.5 Função principal de multiplicação . . . 18
  - 4.3.6 Programa principal . . . 19
- 4.4 Análise de complexidade . . . 19
  - 4.4.1 Complexidade de tempo . . . 19
  - 4.4.2 Complexidade de espaço . . . 19
  - 4.4.3 Análise do preenchimento com zeros . . . 20
- 4.5 Experimentos e resultados . . . 20
  - 4.5.1 Metodologia dos testes . . . 20
  - 4.5.2 Casos de teste . . . 21
  - 4.5.3 Teste 01 – Matrizes de ordem 2 . . . 21
  - 4.5.4 Teste 02 – Caso base . . . 21
  - 4.5.5 Teste 03 – Ordem não potência de dois . . . 21
  - 4.5.6 Teste 04 – Matriz identidade . . . 22
  - 4.5.7 Análise dos resultados . . . 22
- 4.6 Considerações da tarefa . . . 22

**5 Tarefa 2 – Algoritmo de Dijkstra utilizando vetores** **23**
- 5.1 Descrição do problema . . . 23
- 5.2 Estratégia de solução . . . 23
  - 5.2.1 Representação do grafo . . . 24
  - 5.2.2 Seleção dos vértices . . . 24
  - 5.2.3 Reconstrução dos caminhos . . . 24

5.3 Implementação . . . 24
5.3.1 Estruturas de dados . . . 24
5.3.2 Operações básicas do grafo . . . 26
5.3.3 Implementação do algoritmo de Dijkstra . . . 27
5.3.4 Reconstrução dos caminhos . . . 28
5.3.5 Apresentação de todos os caminhos . . . 29
5.3.6 Programa principal . . . 29
5.4 Análise de complexidade . . . 31
5.4.1 Complexidade de tempo . . . 31
5.4.2 Complexidade de espaço . . . 32
5.4.3 Complexidade da reconstrução dos caminhos . . . 32
5.4.4 Complexidade das operações do grafo . . . 33
5.5 Experimentos e resultados . . . 33
5.5.1 Metodologia experimental . . . 33
5.5.2 Resultados dos experimentos . . . 34
5.5.3 Comparação entre os grafos de Roma e Nova York . . . 36
5.5.4 Análise dos resultados . . . 37
5.6 Considerações da tarefa . . . 38

**6 Tarefa 3 – Algoritmo de Dijkstra utilizando Listas Encadeadas 40**
6.1 Descrição do problema . . . 40
6.2 Estratégia de solução . . . 40
6.2.1 Representação do grafo . . . 40
6.2.2 Execução do algoritmo . . . 40
6.3 Implementação . . . 41
6.3.1 Estruturas de dados . . . 41
6.3.2 Inserção de arestas . . . 41
6.3.3 Adaptação do algoritmo de Dijkstra . . . 42
6.3.4 Programa principal . . . 42
6.4 Análise de complexidade . . . 42
6.4.1 Complexidade de tempo . . . 42
6.4.2 Complexidade de espaço . . . 43
6.4.3 Comparação entre representações . . . 43
6.5 Experimentos e resultados . . . 43
6.5.1 Metodologia dos testes . . . 43
6.5.2 Resultados experimentais . . . 44
6.5.3 Análise comparativa dos resultados . . . 44
6.6 Considerações da tarefa . . . 45

**7 Tarefa 4 – Algoritmo de Dijkstra utilizando Fila de Prioridade 46**
7.1 Descrição do problema . . . 46
7.2 Estratégia de solução . . . 46
7.3 Implementação . . . 46
7.4 Análise de complexidade . . . 47
7.4.1 Complexidade das operações do heap . . . 47
7.4.2 Complexidade de tempo . . . 47
7.4.3 Complexidade de espaço . . . 47
7.4.4 Comparação com as implementações anteriores . . . 48
7.5 Experimentos e resultados . . . 48
7.5.1 Testes de funcionamento . . . 48
7.5.2 Metodologia dos experimentos . . . 48
7.5.3 Tempos de execução . . . 49
7.5.4 Operações da fila de prioridade . . . 49
7.5.5 Comparação de desempenho . . . 49
7.5.6 Análise dos resultados . . . 50
7.6 Considerações da tarefa . . . 50

**8 Tarefa 5 – Caminho Mínimo por Busca Completa** **51**
8.1 Descrição do problema . . . 51
8.2 Estratégia de solução . . . 51
8.2.1 Exploração recursiva dos caminhos . . . 51
8.2.2 Controle dos vértices visitados . . . 52
8.2.3 Atualização do melhor caminho . . . 52
8.2.4 Controle do tempo de execução . . . 52
8.3 Implementação . . . 53
8.3.1 Representação do grafo . . . 53
8.3.2 Estrutura de resultados . . . 53
8.3.3 Função recursiva de busca . . . 53
8.3.4 Controle de visitas e backtracking . . . 54
8.3.5 Atualização da melhor solução . . . 54
8.3.6 Controle do tempo de execução . . . 54
8.3.7 Função principal e apresentação dos resultados . . . 54
8.4 Análise de complexidade . . . 55
8.4.1 Quantidade de caminhos simples . . . 55
8.4.2 Complexidade de tempo . . . 55
8.4.3 Complexidade de espaço . . . 56
8.4.4 Comparação teórica com Dijkstra . . . 56
8.5 Experimentos e resultados . . . 57
8.5.1 Metodologia dos experimentos . . . 57
8.5.2 Resultados de desempenho . . . 57
8.5.3 Comparação com Dijkstra . . . 58
8.5.4 Análise dos resultados . . . 58
8.6 Considerações da tarefa . . . 58

**9 Tarefa 6 – Caminho Mínimo utilizando Backtracking** **60**
9.1 Descrição do Problema . . . 60
9.2 Estratégia de Solução . . . 60
9.3 Implementação . . . 60
9.4 Análise de Complexidade . . . 61
9.5 Validação dos Resultados . . . 61
9.6 Experimentos e Resultados . . . 61
9.7 Discussão dos Resultados . . . 61
9.8 Considerações da tarefa . . . 62
9.9 Questões Teóricas . . . 62

**10 Tarefa 7 – Estratégia Gulosa para o Problema de Caminho Mínimo** **64**
10.1 Descrição do Problema . . . 64
10.2 Estratégia de Solução . . . 64
10.3 Implementação . . . 64
10.4 Análise de Complexidade . . . 64
10.5 Experimentos e Resultados . . . 65
10.6 Contraexemplo da Estratégia Gulosa . . . 65
10.7 Diferença entre a Estratégia Gulosa e o Algoritmo de Dijkstra . . . 66
10.8 Considerações da tarefa . . . 66

**11 Tarefa 8 – Caminho Mínimo utilizando Simulated Annealing** **68**
11.1 Descrição do problema . . . 68
11.2 Estratégia de solução . . . 68
11.2.1 Representação das soluções . . . 68
11.2.2 Função de avaliação . . . 68
11.2.3 Construção da solução inicial . . . 69
11.2.4 Geração de soluções vizinhas . . . 69
11.2.5 Critério de aceitação . . . 69
11.2.6 Resfriamento . . . 69

11.3 Implementação . . . 69
11.3.1 Estrutura dos caminhos . . . 70
11.3.2 Parâmetros da metaheurística . . . 70
11.3.3 Execução do algoritmo . . . 70
11.4 Análise de complexidade . . . 71
11.4.1 Complexidade de tempo . . . 71
11.4.2 Complexidade de espaço . . . 71
11.5 Experimentos e resultados . . . 72
11.5.1 Metodologia experimental . . . 72
11.5.2 Qualidade das soluções . . . 72
11.5.3 Comparação dos tempos de execução . . . 73
11.5.4 Discussão dos resultados . . . 73
11.6 Considerações da tarefa . . . 74

**12 Tarefa 9 – Caminhos Mínimos utilizando Programação Dinâmica 75**
12.1 Descrição do problema . . . 75
12.2 Estratégia de solução . . . 75
12.2.1 Reconstrução dos caminhos . . . 75
12.3 Implementação . . . 75
12.3.1 Estrutura de resultados . . . 76
12.3.2 Inicialização das distâncias . . . 76
12.3.3 Atualização por programação dinâmica . . . 76
12.3.4 Consulta e apresentação dos resultados . . . 76
12.4 Análise de complexidade . . . 77
12.4.1 Complexidade de tempo . . . 77
12.4.2 Complexidade de espaço . . . 77
12.4.3 Comparação teórica com Dijkstra . . . 77
12.5 Experimentos e resultados . . . 77
12.5.1 Metodologia dos testes . . . 77
12.5.2 Validação da correção . . . 78
12.5.3 Resultados de desempenho . . . 78
12.5.4 Discussão dos resultados . . . 78
12.6 Considerações da tarefa . . . 79

**13 Análise Comparativa Final 80**
13.1 Objetivos e critérios de comparação . . . 80
13.2 Comparação teórica das estratégias . . . 80
13.3 Impacto da estrutura de dados em Dijkstra . . . 80
13.4 Busca completa e impacto das podas . . . 81
13.5 Qualidade das soluções aproximadas . . . 81
13.6 Floyd–Warshall e consultas entre todos os pares . . . 81
13.7 Consumo de memória e número de operações . . . 82
13.8 Análise de desempenho e possíveis gargalos . . . 82
13.9 Síntese da análise comparativa . . . 82

**14 Considerações Finais 83**

# Lista de Tabelas

1 Casos de teste da multiplicação de matrizes. . . . 21
2 Características dos grafos utilizados nos experimentos. . . . 33
3 Tempos de execução do algoritmo de Dijkstra para o grafo de Roma. . . . 35
4 Tempos de execução do algoritmo de Dijkstra para o grafo de Nova York. . . . 35
5 Resumo dos tempos de execução dos grafos avaliados. . . . 35
6 Comparação das razões de crescimento entre os grafos. . . . 36
7 Instâncias utilizadas nos experimentos. . . . 44

8 Tempos de execução para o grafo de Roma (s). . . . . 44
9 Tempos de execução para o grafo de Nova York (s). . . . . 44
10 Comparação das implementações de Dijkstra. . . . . 48
11 Tempos de execução da Tarefa 4. . . . . 49
12 Operações registradas na Tarefa 4. . . . . 49
13 Comparação dos tempos médios de Dijkstra. . . . . 49
14 Fatores de aceleração da Tarefa 4. . . . . 50
15 Resultados experimentais da busca completa. . . . . 57
16 Comparação dos custos com Dijkstra. . . . . 58
17 Tempo médio de execução dos algoritmos . . . . . 61
18 Redução das chamadas recursivas . . . . . 62
19 Classificação das soluções encontradas pela estratégia gulosa . . . . . 65
20 Arestas do grafo utilizado como contraexemplo . . . . . 65
21 Comparação dos tempos médios de execução em microssegundos . . . . . 66
22 Parâmetros utilizados no Simulated Annealing. . . . . 70
23 Classificação das execuções do Simulated Annealing. . . . . 73
24 Tempos médios de execução do SA e de Dijkstra, em segundos. . . . . 73
25 Tempos médios para o cálculo de caminhos mínimos entre todos os pares (segundos). . . 78
26 Comparação teórica dos algoritmos de caminhos mínimos. . . . . 80
27 Tempos médios das três implementações de Dijkstra (s). . . . . 80
28 Redução das chamadas recursivas com podas. . . . . 81
29 Comparação para todos os pares em grafos de 500 vértices. . . . . 82

# Lista de Códigos Fonte

1 Definição do TAD Matriz. . . . . 17
2 Caso base da multiplicação recursiva. . . . . 18
3 Estrutura utilizada para representar um vértice. . . . . 24
4 Estrutura utilizada para representar uma aresta. . . . . 25
5 Estrutura utilizada para representar o grafo. . . . . 25
6 Estrutura utilizada para armazenar os resultados de Dijkstra. . . . . 25
7 Inserção de uma aresta utilizando índices. . . . . 26
8 Exemplo de entrada no formato DIMACS. . . . . 26
9 Inicialização dos vetores do algoritmo de Dijkstra. . . . . 27
10 Seleção do vértice de menor distância. . . . . 27
11 Percurso e relaxamento das arestas. . . . . 28
12 Reconstrução do caminho por predecessores. . . . . 29
13 Formato de saída dos caminhos mínimos. . . . . 29
14 Exemplo de saída de um caminho mínimo. . . . . 29
15 Medição do tempo de execução do algoritmo. . . . . 30
16 Execução do programa com o grafo de Roma. . . . . 30
17 Estrutura de uma aresta na lista encadeada. . . . . 41
18 Estrutura de um vértice com lista de adjacência. . . . . 41
19 Estrutura do grafo com listas encadeadas. . . . . 41
20 Inserção de uma aresta no início da lista. . . . . 41
21 Percurso e relaxamento das listas encadeadas. . . . . 42
22 Estruturas utilizadas na fila de prioridade. . . . . 46
23 Estrutura de resultados da busca completa. . . . . 53
24 Controle de visitas durante a busca recursiva. . . . . 54
25 Estrutura utilizada para representar um caminho. . . . . 70
26 Estrutura de configuracao do Simulated Annealing. . . . . 70
27 Criterio de aceitacao do Simulated Annealing. . . . . 71
28 Estrutura de resultados do Floyd–Warshall. . . . . 76
29 Atualização das distâncias no Floyd–Warshall. . . . . 76
30 Compilação e consultas da Tarefa 9. . . . . 77

# 1 Introdução

Este relatório apresenta o desenvolvimento das atividades propostas no Trabalho Prático da disciplina PCC104 – Projeto e Análise de Algoritmos. O trabalho contempla a implementação, a análise de complexidade e a validação de algoritmos, com o objetivo de relacionar os conceitos teóricos estudados na disciplina às suas aplicações práticas.

As soluções são desenvolvidas na linguagem de programação C, considerando aspectos como organização do código, correção dos resultados, eficiência computacional e utilização de estruturas de dados adequadas.

## 1.1 Especificações do problema

O Trabalho Prático é composto por diferentes tarefas de implementação de algoritmos. Cada tarefa apresenta um problema específico, com requisitos de entrada, saída e processamento que devem ser respeitados.

Para cada problema, busca-se construir uma solução correta, analisar sua complexidade de tempo e espaço e verificar seu funcionamento por meio de testes. As especificações particulares de cada atividade são apresentadas nos respectivos capítulos deste relatório.

## 1.2 Considerações iniciais

O relatório está organizado em uma parte introdutória, uma seção de metodologia geral e capítulos dedicados individualmente às tarefas. Cada capítulo de tarefa apresenta a descrição do problema, a estratégia de solução, os aspectos relevantes da implementação, a análise de complexidade, os testes e os resultados obtidos.

O código-fonte é desenvolvido em C e organizado em arquivos separados, conforme as necessidades de cada atividade. A documentação é elaborada em LaTeX, utilizando a plataforma Overleaf.

## 1.3 Ferramentas utilizadas

As ferramentas empregadas no desenvolvimento e na validação das implementações incluem:

- **Linguagem C**: implementação dos algoritmos e das estruturas de dados.
- **Visual Studio Code**: edição e organização dos arquivos de código-fonte.
- **GCC**: compilação dos programas em C.
- **GNU Make**: automatização da compilação e da execução dos testes.
- **MSYS2 UCRT64**: ambiente utilizado para compilação e testes no Windows.
- **Git e GitHub**: controle de versões e armazenamento do projeto.
- **Overleaf e LaTeX**: elaboração e formatação do relatório.

## 1.4 Especificações da máquina

O desenvolvimento e os testes foram realizados em um computador com as seguintes características, a serem confirmadas:

- **Processador**: Intel I5 11° Geração.
- **Memória RAM**: 24 Ram.
- **Sistema operacional**: Windows 10.
- **Sistema operacional**: Ubuntu.
- **Compilador**: GCC no ambiente MSYS2 UCRT64 [informar versão utilizada].

# 2 Metodologia

O desenvolvimento das atividades segue uma sequência de etapas que busca assegurar a correção dos algoritmos e permitir a análise de seu desempenho. Os procedimentos gerais são descritos a seguir; as particularidades de cada solução são detalhadas nos capítulos correspondentes.

## 2.1 Análise dos enunciados

Inicialmente, são identificados o objetivo de cada problema, os dados de entrada, os resultados esperados e as restrições estabelecidas. Essa etapa orienta a escolha da estratégia algorítmica e a definição das estruturas de dados necessárias.

## 2.2 Projeto e implementação dos algoritmos

Após a interpretação do problema, é definida uma estratégia de solução, considerando as técnicas estudadas na disciplina. A implementação é realizada em linguagem C, com separação de responsabilidades entre funções e arquivos quando apropriado.

A documentação de cada tarefa descreve as principais funções, as decisões de implementação e os casos especiais tratados. Trechos relevantes de código podem ser incluídos no relatório utilizando o ambiente `lstlisting` disponibilizado pelo modelo.

## 2.3 Análise de complexidade

A eficiência das soluções é estudada por meio da análise assintótica de tempo e espaço. São identificadas as operações predominantes e, para algoritmos recursivos, formuladas relações de recorrência. Quando aplicável, essas relações são resolvidas por técnicas como o Teorema Mestre ou árvores de recorrência.

A análise teórica considera as hipóteses necessárias a cada algoritmo, que são explicitadas nas respectivas tarefas.

## 2.4 Testes e validação

As implementações são verificadas por meio de casos de teste que contemplam entradas simples, casos representativos e situações particulares previstas nos enunciados. Os resultados produzidos são comparados com as saídas esperadas.

Quando disponível, um `Makefile` automatiza a compilação e a execução dos testes. A documentação apresenta os casos utilizados e registra os resultados observados, distinguindo a verificação de correção de eventuais medições experimentais de desempenho.

## 2.5 Organização e documentação

O projeto utiliza Git para o versionamento dos arquivos de implementação e teste. Este relatório reúne a descrição das soluções, as análises de complexidade e os resultados obtidos, mantendo uma estrutura uniforme para facilitar a comparação e a consulta às diferentes tarefas.

# 3 Comandos para Execução dos Algoritmos

Esta seção apresenta os procedimentos necessários para compilar, executar e testar os algoritmos desenvolvidos ao longo das nove tarefas do trabalho. Os comandos foram organizados considerando o ambiente Linux Ubuntu, utilizando o compilador GCC e a ferramenta GNU Make.

Para executar os comandos apresentados, deve-se acessar, pelo terminal, o diretório correspondente à tarefa desejada, a partir da raiz do repositório do projeto.

## 3.1 Tarefa 1 – Multiplicação de Matrizes

Para a compilação do programa, deve-se acessar o diretório `tarefa01` e executar o comando:

Compilando a Tarefa 1

```
make
```

O comando utiliza o Makefile para compilar os arquivos-fonte e gerar o executável `tarefa01`. Para remover os arquivos gerados anteriormente e realizar uma nova compilação, utilizam-se os comandos:

Recompilando o programa

```
make clean
make
```

Para executar o programa, basta digitar:

Executando a Tarefa 1

```
./tarefa01 < tests/entrada01.txt
```

O programa recebe pela entrada padrão um número inteiro positivo $n$, que representa a ordem das matrizes quadradas, seguido dos elementos das matrizes $A$ e $B$. Após realizar a multiplicação, o programa apresenta a matriz resultante $C = A \cdot B$.

Também é possível executar o programa sem utilizar um arquivo de entrada:

Execução manual

```
./tarefa01
```

Nesse caso, os valores devem ser informados diretamente pelo terminal, começando pela ordem $n$, seguida dos $n^2$ elementos da primeira matriz e dos $n^2$ elementos da segunda matriz.

Para executar os testes automatizados definidos no Makefile, utiliza-se:

Executando os testes

```
make test
```

O comando executa os casos de teste e compara os resultados produzidos com as saídas esperadas. Na validação realizada no Linux, os quatro testes da Tarefa 1 foram executados com sucesso.

## 3.2 Tarefa 2 – Dijkstra utilizando Vetores

Para a compilação do programa, deve-se acessar o diretório `tarefa02` e executar o comando:

Compilando a Tarefa 2

```
make
```

O Makefile utiliza o compilador GCC e os arquivos `main.c`, `grafo.c` e `dijkstra.c`, gerando o executável `tarefa02`.

Na compilação são utilizadas as seguintes opções:

- *-Wall*: habilita avisos sobre possíveis problemas no código;
- *-Wextra*: habilita avisos adicionais durante a compilação;
- *-std=c11*: utiliza o padrão C11 da linguagem C;
- *-O2*: habilita otimizações do compilador;
- *-Iinclude*: indica o diretório onde estão localizados os arquivos de cabeçalho.

Para remover os arquivos gerados anteriormente e realizar uma nova compilação, utilizam-se:

Recompilando o programa

```
make clean
make
```

Para executar o algoritmo considerando o vértice 1 como origem e calcular os caminhos mínimos para todos os destinos, utiliza-se:

Executando a Tarefa 2

```
./tarefa02 < tests/rome99c.gr
```

Executando a Tarefa 2

```
./tarefa02 < tests/grafo_pequeno.gr
```

Também é possível informar o vértice de origem como argumento. Por exemplo, para utilizar o vértice 2 como origem:

Definindo o vértice de origem

```
./tarefa02 2 < tests/rome99c.gr
```

Definindo o vértice de origem

```
./tarefa02 2 < tests/grafo_pequeno.gr
```

Para calcular e apresentar o caminho mínimo entre dois vértices específicos, informam-se a origem e o destino:

Definindo origem e destino

```
./tarefa02 1 4 < tests/rome99c.gr
```

Definindo origem e destino

```
./tarefa02 1 4 < tests/grafo_pequeno.gr
```

Os argumentos utilizados possuem os seguintes significados:

- `origem`: vértice inicial do algoritmo. Caso não seja informado, assume o valor 1;
- `destino`: vértice para o qual se deseja apresentar o caminho mínimo. Caso não seja informado, são apresentados os caminhos para todos os destinos.

O grafo é fornecido pela entrada padrão por meio do operador `<`, que redireciona o conteúdo do arquivo de entrada para o programa.

O programa apresenta a distância mínima e o caminho encontrado entre os vértices especificados. Caso não exista caminho, uma mensagem é exibida informando que o destino é inalcançável.

Além disso, o tempo de execução do algoritmo é apresentado em segundos, utilizando a função `clock()` da biblioteca padrão da linguagem C.

### 3.3 Tarefa 3 – Dijkstra utilizando Listas Encadeadas

Para a compilação do programa, deve-se acessar o diretório `tarefa03` e executar o comando:

Compilando a Tarefa 3

```
make
```

O Makefile utiliza o compilador GCC e os arquivos `main.c`, `grafo.c` e `dijkstra.c`, gerando o executável `tarefa03`.

Na compilação são utilizadas as seguintes opções:

- *-Wall*: habilita avisos sobre possíveis problemas no código;
- *-Wextra*: habilita avisos adicionais durante a compilação;
- *-std=c11*: utiliza o padrão C11 da linguagem C;
- *-O2*: habilita otimizações do compilador;
- *-Iinclude*: indica o diretório onde estão localizados os arquivos de cabeçalho.

Para remover os arquivos gerados anteriormente e realizar uma nova compilação, utilizam-se:

Recompilando o programa

```
make clean
make
```

Para executar o algoritmo considerando o vértice 1 como origem e calcular os caminhos mínimos para todos os destinos, utiliza-se:

Executando a Tarefa 3

```
./tarefa03 < tests/rome99c.gr
```

Também é possível executar o algoritmo utilizando um grafo de menor dimensão:

Executando com grafo pequeno

```
./tarefa03 < tests/grafo_pequeno.gr
```

Para informar um vértice de origem diferente, por exemplo, o vértice 2, utiliza-se:

Definindo o vértice de origem

```
./tarefa03 2 < tests/rome99c.gr
```

Definindo o vértice de origem

```
./tarefa03 2 < tests/grafo_pequeno.gr
```

Para calcular e apresentar o caminho mínimo entre dois vértices específicos, informam-se a origem e o destino:

Definindo origem e destino

```
./tarefa03 1 4 < tests/rome99c.gr
```

Definindo origem e destino

```
./tarefa03 1 4 < tests/grafo_pequeno.gr
```

Os argumentos utilizados possuem os seguintes significados:

- `origem`: vértice inicial do algoritmo. Caso não seja informado, assume o valor 1;
- `destino`: vértice para o qual se deseja apresentar o caminho mínimo. Caso não seja informado, são apresentados os caminhos para todos os destinos.

Os vértices informados devem ser representados por números inteiros positivos e devem pertencer ao intervalo de vértices existente no grafo.

O grafo é fornecido pela entrada padrão por meio do operador `<`, que redireciona o conteúdo do arquivo de entrada para o programa.

O programa executa o algoritmo de Dijkstra utilizando a representação de grafo baseada em listas encadeadas. Quando um destino específico é informado, apresenta o caminho calculado até esse vértice. Caso contrário, são apresentados os caminhos para todos os destinos.

Além disso, o tempo de execução do algoritmo é apresentado em segundos, utilizando a função `clock()` da biblioteca padrão da linguagem C.

## 3.4 Tarefa 4 – Dijkstra utilizando Heap Binário

Para a compilação do programa, deve-se acessar o diretório `tarefa04` e executar o comando:

Compilando a Tarefa 4

```
make
```

O Makefile utiliza o compilador GCC e os arquivos `main.c`, `grafo.c`, `dijkstra.c` e `fila_prioridade.c`, gerando o executável `tarefa04`.

Na compilação são utilizadas as seguintes opções:

- *-Wall*: habilita avisos sobre possíveis problemas no código;
- *-Wextra*: habilita avisos adicionais durante a compilação;
- *-std=c11*: utiliza o padrão C11 da linguagem C;
- *-O2*: habilita otimizações do compilador;
- *-Iinclude*: indica o diretório onde estão localizados os arquivos de cabeçalho.

Para remover o executável gerado anteriormente e realizar uma nova compilação, utilizam-se:

Recompilando o programa

```
make clean
make
```

Para executar o algoritmo considerando o vértice 1 como origem e calcular os caminhos mínimos para todos os destinos, utiliza-se:

Executando a Tarefa 4

```
./tarefa04 < tests/rome99c.gr
```

Também é possível executar o algoritmo utilizando um grafo de menor dimensão:

Executando com grafo pequeno

```
./tarefa04 < tests/grafo_pequeno.gr
```

Para informar um vértice de origem diferente, por exemplo, o vértice 2, utiliza-se:

Definindo o vértice de origem

```
./tarefa04 2 < tests/rome99c.gr
```

Definindo o vértice de origem

```
./tarefa04 2 < tests/grafo_pequeno.gr
```

Para calcular e apresentar o caminho mínimo entre dois vértices específicos, informam-se a origem e o destino:

Definindo origem e destino

```
./tarefa04 1 4 < tests/rome99c.gr
```

Definindo origem e destino

```
./tarefa04 1 4 < tests/grafo_pequeno.gr
```

Os argumentos utilizados possuem os seguintes significados:

- `origem`: vértice inicial do algoritmo. Caso não seja informado, assume o valor 1;
- `destino`: vértice para o qual se deseja apresentar o caminho mínimo. Caso não seja informado, são apresentados os caminhos para todos os destinos.

Os vértices informados devem ser inteiros positivos e pertencer ao intervalo de vértices existente no grafo.

O grafo é fornecido pela entrada padrão por meio do operador `<`, que redireciona o conteúdo do arquivo de entrada para o programa.

O programa executa o algoritmo de Dijkstra utilizando uma fila de prioridade implementada com heap binário. Quando um destino específico é informado, apresenta o caminho calculado até esse vértice. Caso contrário, são apresentados os caminhos para todos os destinos.

Além do resultado do algoritmo, o programa apresenta informações sobre sua execução, incluindo:

- Tempo de execução do algoritmo, em segundos, calculado utilizando a função `clock()`;
- Número de inserções realizadas na fila de prioridade;
- Número de remoções realizadas na fila de prioridade;
- Número de atualizações de distância realizadas durante o processamento;
- Número de vértices processados pelo algoritmo.

Essas informações são apresentadas na saída de erro padrão e permitem analisar o comportamento da implementação e comparar sua eficiência com as versões anteriores do algoritmo de Dijkstra.

## 3.5 Tarefa 5 – Busca Completa

Para a compilação do programa, deve-se acessar o diretório `tarefa05` e executar o comando:

Compilando a Tarefa 5

```
make
```

O Makefile utiliza o compilador GCC e os arquivos `main.c`, `grafo.c` e `busca_completa.c`, gerando o executável `tarefa05`.

Na compilação são utilizadas as seguintes opções:

- *-Wall*: habilita avisos sobre possíveis problemas no código;
- *-Wextra*: habilita avisos adicionais durante a compilação;
- *-std=c11*: utiliza o padrão C11 da linguagem C;
- *-O2*: habilita otimizações do compilador;
- *-Iinclude*: indica o diretório onde estão localizados os arquivos de cabeçalho.

Para remover os arquivos gerados anteriormente e realizar uma nova compilação, utilizam-se:

Recompilando o programa

```
make clean
make
```

Para executar o algoritmo de busca completa, é necessário informar os vértices de origem e destino. Por exemplo, para encontrar o menor caminho entre os vértices 1 e 4, utiliza-se:

Executando a Tarefa 5

```
./tarefa05 1 4 < tests/rome99c.gr
```

Também é possível executar o algoritmo utilizando um grafo de menor dimensão:

Executando com grafo pequeno

```
./tarefa05 1 4 < tests/grafo_pequeno.gr
```

Devido ao elevado custo computacional da busca completa, o programa permite definir um limite de tempo para a execução do algoritmo.

Para estabelecer um tempo máximo de 10 segundos, por exemplo, utiliza-se:

Execução com limite de tempo

```
./tarefa05 1 4 10 < tests/rome99c.gr
```

Execução com limite de tempo

```
./tarefa05 1 4 10 < tests/grafo_pequeno.gr
```

Os argumentos utilizados possuem os seguintes significados:

- `origem`: vértice inicial do caminho, sendo um argumento obrigatório;
- `destino`: vértice final do caminho, também obrigatório;
- `tempo_maximo`: parâmetro opcional que define o limite de tempo da busca, em segundos, conforme a implementação do algoritmo. Quando omitido, recebe o valor 0.

Os vértices informados devem ser inteiros positivos e pertencer ao intervalo de vértices existente no grafo. O parâmetro de tempo, quando fornecido, deve ser um número real finito e não negativo.

O grafo é fornecido pela entrada padrão por meio do operador `<`, que redireciona o conteúdo do arquivo de entrada para o programa.

O algoritmo utiliza a estratégia de busca completa para explorar as possibilidades de caminhos entre os vértices especificados, buscando identificar o caminho de menor custo.

Por se tratar de uma abordagem de custo computacional elevado, sua execução pode demandar um tempo considerável em grafos com muitos vértices e arestas. O parâmetro de tempo máximo permite controlar esse processamento.

Ao final da execução, o programa apresenta o tempo gasto pelo algoritmo, em segundos, calculado utilizando a função `clock()`, além das informações produzidas pela função responsável por imprimir o resultado da busca completa.

## 3.6 Tarefa 6 – Backtracking com Podas

Para a compilação do programa, deve-se acessar o diretório `tarefa06` e executar o comando:

Compilando a Tarefa 6

```
make
```

O Makefile utiliza o compilador GCC e os arquivos `main.c`, `grafo.c`, `busca_completa.c` e `backtracking.c`, gerando o executável `tarefa06`.

Na compilação são utilizadas as seguintes opções:

- *-Wall*: habilita avisos sobre possíveis problemas no código;
- *-Wextra*: habilita avisos adicionais durante a compilação;
- *-Wpedantic*: verifica o cumprimento das regras do padrão da linguagem C;
- *-std=c11*: utiliza o padrão C11 da linguagem C;
- *-O2*: habilita otimizações do compilador;
- *-Iinclude*: indica o diretório onde estão localizados os arquivos de cabeçalho.

Para remover o executável gerado anteriormente e realizar uma nova compilação, utilizam-se:

Recompilando o programa

```
make clean
make
```

Para executar o algoritmo, é necessário informar os vértices de origem e destino. Opcionalmente, podem ser fornecidos o tempo máximo de execução e o modo de busca.

A estrutura geral do comando é:

Estrutura do comando

```
./tarefa06 origem destino [tempo_maximo] [modo] < arquivo.gr
```

Os argumentos utilizados possuem os seguintes significados:

- `origem`: vértice inicial do caminho, sendo um argumento obrigatório;
- `destino`: vértice final do caminho, também obrigatório;
- `tempo_maximo`: parâmetro opcional que define o limite de tempo da busca, em segundos. Caso não seja informado, assume o valor de 60 segundos;

- `modo`: parâmetro opcional que seleciona a estratégia de busca. Caso não seja informado, assume o valor 1.

O programa permite selecionar três modos de execução:

- **Modo 0**: executa o algoritmo de busca completa, sem a aplicação de podas;
- **Modo 1**: executa a primeira configuração do algoritmo de backtracking;
- **Modo 2**: executa a segunda configuração do algoritmo de backtracking.

Para executar o algoritmo de busca completa, utilizando os vértices 1 e 5 e um limite de 60 segundos, utiliza-se:

Executando o Modo 0

```
./tarefa06 1 5 60 0 < tests/entrada01.gr
```

Para executar a primeira configuração de backtracking, utiliza-se:

Executando o Modo 1

```
./tarefa06 1 5 60 1 < tests/entrada01.gr
```

Para executar a segunda configuração de backtracking, utiliza-se:

Executando o Modo 2

```
./tarefa06 1 5 60 2 < tests/entrada01.gr
```

Também é possível executar o programa sem informar os parâmetros opcionais, utilizando o tempo máximo e o modo definidos como padrão:

Execução com parâmetros padrão

```
./tarefa06 1 5 < tests/entrada01.gr
```

O grafo é fornecido pela entrada padrão por meio do operador `<`, que redireciona o conteúdo do arquivo de entrada para o programa.

Os vértices informados devem ser inteiros positivos e pertencer ao intervalo de vértices do grafo. O tempo máximo deve ser um número real finito e não negativo, enquanto o modo deve assumir um dos valores 0, 1 ou 2.

Ao final da execução, o programa apresenta o resultado da busca de acordo com o modo selecionado. No modo 0, são apresentados o tempo de execução e a quantidade de podas, que é igual a zero, além do resultado da busca completa.

Nos modos 1 e 2, as informações são apresentadas pela função responsável por imprimir o resultado do backtracking.

Para executar automaticamente os três modos de busca, utiliza-se o comando:

Executando os testes

```
make test
```

O comando compila o programa e executa os modos 0, 1 e 2, utilizando o arquivo `tests/entrada01.gr`, com origem no vértice 1, destino no vértice 5 e limite de 60 segundos.

Essas execuções permitem observar o comportamento das diferentes configurações de busca e comparar os resultados apresentados pelos algoritmos.

## 3.7 Tarefa 7 – Algoritmo Guloso para Caminhos Mínimos

Para a compilação do programa, deve-se acessar o diretório `tarefa07` e executar o comando:

Compilando a Tarefa 7

```
make
```

O Makefile utiliza o compilador GCC e os arquivos `main.c`, `grafo.c` e `guloso.c`, gerando o executável `tarefa07`.

Na compilação são utilizadas as seguintes opções:

- *-Wall*: habilita avisos sobre possíveis problemas no código;
- *-Wextra*: habilita avisos adicionais durante a compilação;
- *-Wpedantic*: verifica a conformidade do código com o padrão da linguagem C;
- *-std=c11*: utiliza o padrão C11 da linguagem C;
- *-O2*: habilita otimizações do compilador;
- *-Iinclude*: indica o diretório onde estão localizados os arquivos de cabeçalho.

Para remover o executável gerado anteriormente e realizar uma nova compilação, utilizam-se:

Recompilando o programa

```
make clean
make
```

Para executar o algoritmo guloso, é necessário informar os vértices de origem e destino. Opcionalmente, pode-se fornecer um limite de tempo para a execução.

A estrutura geral do comando é:

Estrutura do comando

```
./tarefa07 origem destino [tempo_maximo] < arquivo.gr
```

Os argumentos utilizados possuem os seguintes significados:

- `origem`: vértice inicial do caminho, sendo um argumento obrigatório;
- `destino`: vértice final do caminho, também obrigatório;
- `tempo_maximo`: parâmetro opcional que define o limite de tempo da execução, em segundos. Caso não seja informado, assume o valor de 60 segundos.

Para executar o algoritmo considerando o vértice 1 como origem, o vértice 4 como destino e o limite de 60 segundos, utiliza-se:

Executando a Tarefa 7

```
./tarefa07 1 4 60 < tests/contraexemplo.gr
```

Também é possível executar o programa sem informar o tempo máximo, utilizando o valor padrão:

Execução com tempo padrão

```
./tarefa07 1 4 < tests/contraexemplo.gr
```

Para definir um limite de tempo diferente, por exemplo, 10 segundos, utiliza-se:

Execução com limite de tempo

```
./tarefa07 1 4 10 < tests/contraexemplo.gr
```

O grafo é fornecido pela entrada padrão por meio do operador `<`, que redireciona o conteúdo do arquivo de entrada para o programa.

Os vértices informados devem ser inteiros positivos. O parâmetro de tempo, quando fornecido, deve ser um número real finito e não negativo.

O programa executa o algoritmo por meio da função `executarGuloso()`, que recebe o grafo, os vértices de origem e destino e o limite de tempo.

A estratégia gulosa realiza escolhas locais durante a construção de uma solução para o problema de caminhos mínimos. Diferentemente de algoritmos exatos, essa abordagem não garante necessariamente a obtenção do caminho de menor custo global.

Ao final da execução, os resultados são apresentados pela função `imprimirResultadoGuloso()`, permitindo analisar a solução encontrada pelo algoritmo.

Para executar o teste definido no Makefile, utiliza-se:

Executando os testes

```
make test
```

Esse comando compila o programa e executa automaticamente a seguinte instrução:

Comando executado pelo teste

```
./tarefa07 1 4 60 < tests/contraexemplo.gr
```

O arquivo `contraexemplo.gr` é utilizado para avaliar o comportamento da estratégia gulosa em uma instância específica do problema.

A execução desse teste permite observar a solução produzida e analisar as limitações da estratégia de escolha local.

## 3.8 Tarefa 8 – Simulated Annealing

Para a compilação do programa, deve-se acessar o diretório `tarefa08` e executar o comando:

Compilando a Tarefa 8

```
make
```

O Makefile utiliza o compilador GCC e os arquivos `main.c`, `grafo.c`, `caminho.c` e `simulated_annealing.c`, gerando o executável `tarefa08`.

Na compilação são utilizadas as seguintes opções:

- *-Wall*: habilita avisos sobre possíveis problemas no código;
- *-Wextra*: habilita avisos adicionais durante a compilação;
- *-Wpedantic*: verifica a conformidade do código com o padrão da linguagem C;
- *-std=c11*: utiliza o padrão C11 da linguagem C;
- *-O2*: habilita otimizações do compilador;
- *-Iinclude*: indica o diretório onde estão localizados os arquivos de cabeçalho;
- *-lm*: realiza a ligação com a biblioteca matemática da linguagem C.

Para remover o executável gerado anteriormente e realizar uma nova compilação, utilizam-se:

Recompilando o programa

```
make clean
make
```

Para executar o algoritmo de Simulated Annealing, é necessário informar os vértices de origem e destino. Opcionalmente, podem ser fornecidos o tempo máximo de execução e a semente utilizada pelo gerador de números pseudoaleatórios.

A estrutura geral do comando é:

Estrutura do comando

```
./tarefa08 origem destino [tempo_maximo] [semente] < arquivo.gr
```

Os argumentos utilizados possuem os seguintes significados:

- `origem`: vértice inicial do caminho, sendo um argumento obrigatório;
- `destino`: vértice final do caminho, também obrigatório;
- `tempo_maximo`: parâmetro opcional que define o limite de tempo da execução, em segundos. Caso não seja informado, assume o valor de 60 segundos;
- `semente`: parâmetro opcional utilizado para inicializar o gerador de números pseudoaleatórios. Caso não seja informado, é utilizado o horário atual do sistema.

Para executar o algoritmo considerando o vértice 1 como origem e o vértice 4 como destino, utilizando os parâmetros padrão, utiliza-se:

Executando a Tarefa 8

```
./tarefa08 1 4 < tests/contraexemplo.gr
```

Para definir um limite de tempo de 10 segundos, utiliza-se:

Execução com limite de tempo

```
./tarefa08 1 4 10 < tests/contraexemplo.gr
```

Também é possível informar uma semente fixa, permitindo reproduzir a sequência de números pseudoaleatórios utilizada pelo algoritmo.

Por exemplo, para executar o programa com limite de 10 segundos e semente igual a 12345:

Execução com semente fixa

```
./tarefa08 1 4 10 12345 < tests/contraexemplo.gr
```

O programa possui os seguintes parâmetros internos de configuração do Simulated Annealing:

- Temperatura inicial: $100,0$;
- Temperatura mínima: $0,01$;
- Taxa de resfriamento: $0,995$;
- Iterações por temperatura: 20;
- Tempo máximo padrão: 60 segundos.

Esses parâmetros controlam o processo de busca e o resfriamento gradual da temperatura durante a execução do algoritmo.

O grafo é fornecido pela entrada padrão por meio do operador `<`, que redireciona o conteúdo do arquivo de entrada para o programa.

O Simulated Annealing é uma meta-heurística que utiliza um processo de busca probabilística para explorar diferentes soluções do problema.

Diferentemente de uma estratégia puramente gulosa, o algoritmo pode aceitar soluções de custo superior ao da solução atual, dependendo da temperatura e dos critérios de aceitação implementados.

Esse comportamento permite explorar outras regiões do espaço de busca, buscando reduzir a possibilidade de ficar restrito a ótimos locais.

Ao final da execução, os resultados são apresentados pela função `imprimirResultadoSA()`, permitindo analisar a solução produzida pelo algoritmo.

Para executar o teste definido no Makefile, utiliza-se:

Executando os testes

```
make test
```

Esse comando compila o programa e executa automaticamente a seguinte instrução:

Comando executado pelo teste

```
./tarefa08 1 4 5 12345 < tests/contraexemplo.gr
```

Nesse teste, o algoritmo utiliza o vértice 1 como origem, o vértice 4 como destino, um limite de 5 segundos e uma semente fixa igual a 12345.

A utilização de uma semente fixa facilita a comparação entre execuções, pois permite reproduzir a sequência pseudoaleatória, desde que as demais condições de execução sejam mantidas.

O teste permite observar o comportamento da meta-heurística em uma instância específica do problema de caminhos mínimos.

## 3.9 Tarefa 9 – Algoritmo de Floyd–Warshall

Para a compilação do programa, deve-se acessar o diretório `tarefa09` e executar o comando:

Compilando a Tarefa 9

```
make
```

O Makefile utiliza o compilador GCC e os arquivos `main.c`, `grafo.c` e `floyd_warshall.c`, gerando o executável `tarefa09`.

Na compilação são utilizadas as seguintes opções:

- *-Wall*: habilita avisos sobre possíveis problemas no código;
- *-Wextra*: habilita avisos adicionais durante a compilação;
- *-Wpedantic*: verifica a conformidade do código com o padrão da linguagem C;
- *-std=c11*: utiliza o padrão C11 da linguagem C;
- *-O2*: habilita otimizações do compilador;
- *-Iinclude*: indica o diretório onde estão localizados os arquivos de cabeçalho.

Para remover o executável gerado anteriormente e realizar uma nova compilação, utilizam-se:

Recompilando o programa

```
make clean
make
```

O algoritmo de Floyd–Warshall calcula as menores distâncias entre todos os pares de vértices de um grafo ponderado.

O programa permite duas formas de execução: a apresentação da matriz completa de distâncias mínimas ou a consulta do caminho mínimo entre dois vértices específicos.

A estrutura geral do comando é:

Estrutura do comando

```
./tarefa09 [origem destino] < arquivo.gr
```

Os argumentos utilizados possuem os seguintes significados:

- `origem`: vértice inicial do caminho que se deseja consultar;
- `destino`: vértice final do caminho que se deseja consultar.

Os argumentos são opcionais em conjunto. Dessa forma, o programa deve ser executado sem argumentos ou com os dois vértices informados simultaneamente.

Quando nenhum argumento é fornecido, o programa apresenta a matriz completa de distâncias mínimas entre todos os pares de vértices.

Para executar o algoritmo dessa forma, utiliza-se:

Executando a Tarefa 9

```
./tarefa09 < tests/exemplo.gr
```

Para consultar a distância mínima e o caminho entre dois vértices específicos, devem ser informados os vértices de origem e destino.

Por exemplo, para consultar o caminho do vértice 1 até o vértice 4, utiliza-se:

Definindo origem e destino

```
./tarefa09 1 4 < tests/exemplo.gr
```

Também é possível realizar a consulta no sentido contrário, utilizando o vértice 4 como origem e o vértice 1 como destino:

Consultando o caminho inverso

```
./tarefa09 4 1 < tests/exemplo.gr
```

Essa possibilidade permite consultar as distâncias em diferentes sentidos, o que é especialmente importante em grafos direcionados, nos quais a distância entre dois vértices pode depender do sentido considerado.

O grafo é fornecido pela entrada padrão por meio do operador `<`, que redireciona o conteúdo do arquivo de entrada para o programa.

Os vértices informados devem ser inteiros positivos e pertencer ao intervalo de vértices existente no grafo.

Quando os vértices de origem e destino são informados, o programa apresenta a distância mínima entre eles e o caminho correspondente.

Caso não exista um caminho entre os vértices, a distância é apresentada como `INF`, acompanhada de uma mensagem indicando que os vértices não estão conectados por um caminho.

O algoritmo de Floyd–Warshall utiliza a técnica de programação dinâmica para calcular as distâncias mínimas entre todos os pares de vértices.

Sua complexidade de tempo é $O(V^3)$, enquanto a complexidade de espaço é $O(V^2)$, sendo $V$ a quantidade de vértices do grafo.

Essas características tornam o algoritmo adequado para o cálculo de caminhos mínimos entre todos os pares de vértices, embora seu custo computacional e de memória possa ser elevado em grafos de grande dimensão.

Além dos resultados calculados, o programa apresenta o tempo de execução do algoritmo, em segundos, utilizando a função `clock()` da biblioteca padrão da linguagem C.

A medição considera apenas a execução da função `floydWarshall()`, não incluindo a leitura do grafo ou a impressão dos resultados.

Para executar os testes definidos no Makefile, utiliza-se:

Executando os testes

```
make test
```

Esse comando compila o programa e realiza automaticamente três execuções:

- Execução sem argumentos, apresentando a matriz completa de distâncias mínimas;
- Consulta do caminho mínimo entre os vértices 1 e 4;
- Consulta do caminho mínimo entre os vértices 4 e 1.

Os comandos executados pelo teste são:

Comandos executados pelo teste

```
./tarefa09 < tests/exemplo.gr
./tarefa09 1 4 < tests/exemplo.gr
./tarefa09 4 1 < tests/exemplo.gr
```

Esses testes permitem verificar a execução do algoritmo tanto para o cálculo das distâncias entre todos os pares de vértices quanto para consultas individuais de caminhos.

O alvo `test` realiza as execuções, mas não compara automaticamente os resultados com arquivos de saída esperados.

# 4 Tarefa 1 – Multiplicação de Matrizes

## 4.1 Descrição do problema

A primeira tarefa consiste na implementação de um algoritmo para realizar a multiplicação de duas matrizes quadradas de ordem $n$, utilizando a técnica de divisão e conquista. Dadas duas matrizes $A$ e $B$, ambas de dimensão $n \times n$, o objetivo é calcular a matriz resultante $C = A \cdot B$, também de ordem $n$. Cada elemento da matriz resultante é definido por:

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}, \qquad 1 \leq i, j \leq n. \tag{1}$$

A entrada do programa é composta por um inteiro positivo $n$, que representa a ordem das matrizes, seguido dos $n^2$ elementos da matriz $A$ e dos $n^2$ elementos da matriz $B$. A saída consiste nos elementos da matriz resultante $C$, organizados em $n$ linhas e $n$ colunas. A solução deve utilizar uma abordagem recursiva baseada em divisão e conquista, decompondo o problema original em multiplicações de submatrizes de menor dimensão.

## 4.2 Estratégia de solução

Para resolver o problema, foi utilizada a técnica de divisão e conquista, que consiste em dividir um problema em subproblemas menores, resolvê-los recursivamente e combinar seus resultados para obter a solução original.

Na multiplicação de matrizes, essa estratégia é aplicada dividindo cada matriz quadrada em quatro submatrizes de mesma dimensão. Considerando duas matrizes $A$ e $B$, de ordem $n$, suas decomposições podem ser representadas por:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \qquad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}. \tag{2}$$

Cada submatriz possui dimensão $\frac{n}{2} \times \frac{n}{2}$, considerando inicialmente que $n$ é uma potência de dois. A matriz resultante $C = A \cdot B$ também pode ser representada por quatro submatrizes:

$$C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}. \tag{3}$$

Os blocos da matriz resultante são calculados pelas seguintes expressões:

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}, \tag{4}$$
$$C_{12} = A_{11}B_{12} + A_{12}B_{22}, \tag{5}$$
$$C_{21} = A_{21}B_{11} + A_{22}B_{21}, \tag{6}$$
$$C_{22} = A_{21}B_{12} + A_{22}B_{22}. \tag{7}$$

Dessa forma, cada etapa da recursão realiza oito multiplicações de submatrizes de ordem $n/2$, além das operações de soma e combinação dos resultados parciais.

O caso base da recursão ocorre quando as matrizes possuem ordem $n = 1$. Nesse caso, a multiplicação é realizada diretamente entre seus únicos elementos, encerrando a chamada recursiva.

Após o cálculo dos quatro blocos resultantes, eles são combinados para formar a matriz produto da etapa atual.

### 4.2.1 Tratamento de matrizes de ordem arbitrária

Como a divisão recursiva utiliza submatrizes de mesma dimensão, foi implementado um procedimento para tratar matrizes cuja ordem não é uma potência de dois.

Inicialmente, determina-se a menor potência de dois maior ou igual à ordem original $n$:

$$m = 2^{\lceil \log_2 n \rceil}. \tag{8}$$

Em seguida, são criadas duas matrizes auxiliares de ordem $m$, preenchidas inicialmente com zeros. Os elementos das matrizes originais são copiados para as posições correspondentes dessas matrizes auxiliares. Esse procedimento, denominado preenchimento com zeros (*zero-padding*), permite que as matrizes sejam divididas sucessivamente em quatro blocos de mesma dimensão. Após a multiplicação recursiva, são extraídos os $n \times n$ elementos da região superior esquerda da matriz resultante, obtendo-se o produto correspondente às matrizes originais. Assim, a solução permite multiplicar matrizes quadradas de qualquer ordem positiva, mantendo a estratégia recursiva de divisão e conquista.

## 4.3 Implementação

A implementação foi desenvolvida na linguagem C, utilizando um Tipo Abstrato de Dados (TAD) para representar e manipular matrizes quadradas.

O código foi organizado em três arquivos principais: `matriz.h`, responsável pela definição da estrutura e declaração das funções; `matriz.c`, que contém a implementação das operações sobre matrizes; e `main.c`, responsável pela leitura dos dados, execução da multiplicação e apresentação do resultado.

### 4.3.1 Estrutura de dados

A estrutura `Matriz` armazena a ordem da matriz e um ponteiro para seus elementos, representados por um vetor de ponteiros para linhas alocadas dinamicamente.

```
1 typedef struct {
2     int n;
3     int **dados;
4 } Matriz;
```

Código 1: Definição do TAD Matriz.

O campo `n` representa a quantidade de linhas e colunas, enquanto `dados` permite acessar os elementos por meio da expressão `m->dados[i][j]`.

A alocação dinâmica permite criar matrizes de diferentes dimensões durante a execução, o que é necessário para o funcionamento da abordagem recursiva.

### 4.3.2 Operações básicas

Foram implementadas funções para criação, liberação de memória, leitura e impressão de matrizes.

A função `createMatriz` recebe a ordem da matriz e realiza a alocação dinâmica de sua estrutura e de seus elementos, inicializados com zero.

A função `deleteMatriz` libera a memória alocada, enquanto `readMatriz` e `printMatriz` realizam, respectivamente, a leitura dos elementos e a apresentação da matriz.

Essas operações são utilizadas tanto pelo programa principal quanto pelas funções auxiliares da multiplicação.

### 4.3.3 Funções auxiliares

Para implementar a estratégia de divisão e conquista, foram desenvolvidas três funções auxiliares:

- `subMatriz`: cria uma nova matriz contendo uma região quadrada da matriz original, definida pela posição inicial e pela dimensão desejada.
- `somaMatriz`: recebe duas matrizes de mesma ordem e retorna uma nova matriz contendo a soma de seus elementos correspondentes.
- `juntaMatriz`: combina quatro submatrizes em uma única matriz quadrada, posicionando cada bloco no quadrante correspondente.

Essas funções permitem dividir as matrizes de entrada, combinar os produtos parciais e reconstruir o resultado de cada chamada recursiva.

### 4.3.4 Multiplicação recursiva

A função `multiplicaRecursivo` implementa a etapa principal do algoritmo de divisão e conquista.

A função recebe duas matrizes de mesma ordem e retorna uma nova matriz correspondente ao produto entre elas.

O caso base ocorre quando a ordem das matrizes é igual a um. Nessa situação, a multiplicação é realizada diretamente, conforme apresentado no Código 2.

```
1 if (n == 1) {
2     Matriz *C = createMatriz(1);
3
4     if (C != NULL)
5         C->dados[0][0] = A->dados[0][0] * B->dados[0][0];
6
7     return C;
8 }
```

Código 2: Caso base da multiplicação recursiva.

Para matrizes de ordem superior a um, são criadas quatro submatrizes de cada operando. Em seguida, são realizadas oito chamadas recursivas para calcular os produtos necessários à construção dos quatro blocos da matriz resultante. Os produtos são somados dois a dois, conforme as expressões apresentadas na Seção 4, e os quatro blocos resultantes são combinados pela função `juntaMatriz`. A implementação também realiza a liberação das matrizes temporárias após sua utilização, evitando que as alocações intermediárias permaneçam ocupando memória desnecessariamente.

### 4.3.5 Função principal de multiplicação

A função `multiplicaMatriz` funciona como interface para a multiplicação recursiva.

Inicialmente, verifica as matrizes recebidas e determina a menor potência de dois maior ou igual à ordem original.

Quando necessário, são criadas matrizes auxiliares maiores, preenchidas com zeros, para permitir a divisão recursiva em quadrantes de mesma dimensão.

Após a execução de `multiplicaRecursivo`, a função constrói a matriz resultante com a dimensão original, copiando os elementos correspondentes e liberando as estruturas temporárias.

Dessa forma, o programa permite multiplicar matrizes quadradas de ordem arbitrária, mantendo a lógica recursiva concentrada em uma função específica.

### 4.3.6 Programa principal

O arquivo `main.c` coordena a execução do programa, realizando a leitura da ordem das matrizes, a criação das estruturas, a leitura dos elementos e a chamada da função `multiplicaMatriz`.

Após a multiplicação, o resultado é impresso e a memória alocada é liberada.

A implementação verifica possíveis falhas de leitura e alocação, encerrando a execução quando não é possível prosseguir corretamente.

## 4.4 Análise de complexidade

A análise de complexidade do algoritmo considera o número de operações realizadas durante a multiplicação recursiva de duas matrizes quadradas de ordem $n$, bem como a quantidade de memória auxiliar necessária para sua execução.

### 4.4.1 Complexidade de tempo

Em cada chamada da função `multiplicaRecursivo`, o algoritmo divide as duas matrizes em quatro submatrizes de ordem $n/2$, realiza oito multiplicações recursivas e combina os resultados obtidos.

A criação das submatrizes exige a cópia de elementos das matrizes originais, resultando em um custo de $\Theta(n^2)$.

As oito multiplicações recursivas são realizadas sobre matrizes de ordem $n/2$, produzindo o termo $8T(n/2)$.

Por fim, as operações de soma e combinação dos blocos resultantes também possuem custo $\Theta(n^2)$.

Dessa forma, a relação de recorrência do algoritmo é dada por:

$$T(n) = \begin{cases} \Theta(1), & \text{se } n = 1, \\ 8T(n/2) + \Theta(n^2), & \text{se } n > 1. \end{cases} \tag{9}$$

Para resolver essa recorrência, aplica-se o Teorema Mestre, considerando sua forma geral:

$$T(n) = aT(n/b) + f(n). \tag{10}$$

Identificam-se os seguintes parâmetros:

$$a = 8, \qquad b = 2, \qquad f(n) = \Theta(n^2). \tag{11}$$

Calculando o termo de comparação do Teorema Mestre:

$$n^{\log_b a} = n^{\log_2 8} = n^3. \tag{12}$$

Como $f(n) = \Theta(n^2)$, verifica-se que:

$$f(n) = O(n^{3-\varepsilon}), \tag{13}$$

para $\varepsilon = 1$.

Portanto, a recorrência satisfaz o primeiro caso do Teorema Mestre, resultando em:

$$\boxed{T(n) = \Theta(n^3)}. \tag{14}$$

Assim, o algoritmo apresenta complexidade de tempo cúbica em relação à ordem das matrizes.

### 4.4.2 Complexidade de espaço

A implementação utiliza alocação dinâmica para armazenar as matrizes originais, as submatrizes e os resultados intermediários das operações recursivas.

Em cada chamada, são criadas matrizes auxiliares cuja quantidade total de elementos é proporcional a $n^2$.

Embora sejam realizadas oito multiplicações recursivas, essas chamadas são executadas sequencialmente. Dessa forma, a memória necessária para uma chamada recursiva pode ser reutilizada após seu encerramento.

O espaço máximo utilizado pela recursão pode ser representado pela relação:

$$S(n) = S(n/2) + \Theta(n^2). \tag{15}$$

Expandindo a recorrência, obtém-se:

$$S(n) = \Theta\left(n^2 + \frac{n^2}{4} + \frac{n^2}{16} + \cdots\right). \tag{16}$$

A expressão corresponde a uma progressão geométrica de razão 1/4. Como a soma da série geométrica é limitada por uma constante, conclui-se que:

$$\boxed{S(n) = \Theta(n^2)}. \tag{17}$$

A profundidade da recursão é $\Theta(\log n)$, sendo o espaço ocupado pela pilha de chamadas assintoticamente inferior ao espaço utilizado pelas matrizes auxiliares.

### 4.4.3 Análise do preenchimento com zeros

Para matrizes cuja ordem não é uma potência de dois, a implementação utiliza matrizes auxiliares de ordem:

$$m = 2^{\lceil \log_2 n \rceil}. \tag{18}$$

Para todo $n \geq 1$, verifica-se que:

$$n \leq m < 2n. \tag{19}$$

Consequentemente, $m = \Theta(n)$.

A criação e a cópia dos elementos para as matrizes auxiliares possuem custo $\Theta(m^2)$, enquanto a multiplicação recursiva apresenta custo $\Theta(m^3)$.

Como $m = \Theta(n)$, o preenchimento com zeros não altera as ordens assintóticas de tempo e espaço.

Portanto, considerando o algoritmo completo, obtêm-se as seguintes complexidades:

$$\boxed{T(n) = \Theta(n^3)} \tag{20}$$

$$\boxed{S(n) = \Theta(n^2)}. \tag{21}$$

## 4.5 Experimentos e resultados

Para verificar o funcionamento do algoritmo de multiplicação de matrizes, foram realizados testes com diferentes dimensões e configurações de entrada.

Os testes tiveram como objetivo avaliar a correção dos resultados produzidos pela implementação, incluindo o caso base da recursão, matrizes de ordem que não é potência de dois e matrizes de dimensões maiores.

### 4.5.1 Metodologia dos testes

Os testes foram organizados em arquivos de entrada e saída, armazenados no diretório `tests` do projeto.

Para cada caso, foi definido um arquivo contendo a ordem das matrizes e seus elementos, juntamente com outro arquivo contendo o resultado esperado da multiplicação.

A execução dos testes foi automatizada por meio de um `Makefile`, utilizando o comando:

```
make test
```

O procedimento executa o programa para cada arquivo de entrada e compara a saída produzida com o resultado esperado, utilizando a ferramenta `diff`.

Um teste é considerado aprovado quando os dois arquivos apresentam o mesmo conteúdo.

### 4.5.2 Casos de teste

Foram utilizados quatro casos de teste, apresentados na Tabela 1.

Tabela 1: Casos de teste da multiplicação de matrizes.

| Teste | Ordem | Descrição | Resultado |
|---|---|---|---|
| 01 | 2 | Multiplicação de duas matrizes com elementos constantes. | Aprovado |
| 02 | 1 | Verificação do caso base da recursão. | Aprovado |
| 03 | 3 | Multiplicação com preenchimento de zeros. | Aprovado |
| 04 | 4 | Multiplicação de uma matriz pela matriz identidade. | Aprovado |

### 4.5.3 Teste 01 – Matrizes de ordem 2

O primeiro teste utiliza duas matrizes de ordem 2, definidas por:

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \qquad B = \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix}. \tag{22}$$

O resultado esperado é:

$$C = A \cdot B = \begin{bmatrix} 4 & 4 \\ 4 & 4 \end{bmatrix}. \tag{23}$$

Esse teste verifica a multiplicação recursiva de matrizes cuja ordem é uma potência de dois.

### 4.5.4 Teste 02 – Caso base

O segundo teste utiliza matrizes de ordem 1:

$$A = \begin{bmatrix} 5 \end{bmatrix}, \qquad B = \begin{bmatrix} 3 \end{bmatrix}. \tag{24}$$

O resultado esperado é:

$$C = \begin{bmatrix} 15 \end{bmatrix}. \tag{25}$$

Esse teste verifica o caso base da recursão, no qual a multiplicação é realizada diretamente entre os únicos elementos das matrizes.

### 4.5.5 Teste 03 – Ordem não potência de dois

O terceiro teste utiliza matrizes de ordem 3:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \qquad B = \begin{bmatrix} 9 & 8 & 7 \\ 6 & 5 & 4 \\ 3 & 2 & 1 \end{bmatrix}. \tag{26}$$

O resultado esperado é:

$$C = \begin{bmatrix} 30 & 24 & 18 \\ 84 & 69 & 54 \\ 138 & 114 & 90 \end{bmatrix}. \tag{27}$$

Esse caso verifica o procedimento de preenchimento com zeros, pois as matrizes são ampliadas internamente para ordem 4 antes da multiplicação recursiva.

### 4.5.6 Teste 04 – Matriz identidade

O quarto teste utiliza uma matriz de ordem 4 multiplicada pela matriz identidade.

A matriz de entrada é:

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}. \tag{28}$$

Como a segunda matriz é a identidade $I_4$, o resultado esperado é:

$$A \cdot I_4 = A. \tag{29}$$

Esse teste verifica se o algoritmo preserva corretamente os elementos da matriz quando multiplicada pela identidade.

### 4.5.7 Análise dos resultados

Os quatro testes funcionais foram executados com sucesso, produzindo resultados iguais aos valores esperados. Os testes contemplam diferentes situações de execução, incluindo o caso base da recursão, matrizes com ordem potência de dois e matrizes que exigem preenchimento com zeros. Os resultados obtidos fornecem evidências da correção da implementação para os casos avaliados. Entretanto, esses testes não constituem uma demonstração formal de correção para todas as entradas possíveis. A análise assintótica apresentada na Seção 3.4 estabelece o comportamento teórico do algoritmo, enquanto os testes realizados verificam seu funcionamento em entradas específicas.

## 4.6 Considerações da tarefa

A implementação da multiplicação de matrizes por divisão e conquista permitiu aplicar os conceitos de recursividade, decomposição de problemas e análise de complexidade estudados na disciplina de Projeto e Análise de Algoritmos. A utilização de um Tipo Abstrato de Dados (TAD) para representar as matrizes contribuiu para a organização do código, separando as operações de criação, leitura, impressão e liberação de memória da lógica de multiplicação.

A estratégia adotada consiste na divisão das matrizes em quatro submatrizes, seguida da realização de oito multiplicações recursivas e da combinação dos resultados parciais. Uma decisão importante da implementação foi a utilização de preenchimento com zeros para permitir a multiplicação de matrizes cuja ordem não é uma potência de dois. Dessa forma, o algoritmo pode receber matrizes quadradas de diferentes dimensões sem modificar sua estrutura recursiva. A análise teórica demonstrou que a solução apresenta complexidade de tempo $\Theta(n^3)$ e complexidade de espaço $\Theta(n^2)$. Os testes funcionais realizados apresentaram os resultados esperados para os casos avaliados, incluindo matrizes de ordem 1, matrizes de ordem potência de dois e matrizes que exigem preenchimento com zeros.

Como limitação, destaca-se a necessidade de alocar e copiar diversas submatrizes temporárias durante a execução, o que introduz custos adicionais de memória e processamento. Embora a estratégia de divisão e conquista organize a solução de forma recursiva, a implementação com oito multiplicações por etapa mantém a mesma ordem de complexidade de tempo do algoritmo convencional de multiplicação de matrizes. Portanto, a solução desenvolvida atende ao objetivo de aplicar a técnica de divisão e conquista à multiplicação de matrizes, permitindo relacionar a implementação prática com sua análise matemática de complexidade.

# 5 Tarefa 2 – Algoritmo de Dijkstra utilizando vetores

## 5.1 Descrição do problema

A segunda tarefa consiste na implementação do algoritmo de Dijkstra para determinar os caminhos mínimos a partir de um vértice de origem em um grafo direcionado e ponderado, utilizando vetores como estrutura de armazenamento.

Um grafo pode ser representado por $G = (V, E)$, em que $V$ corresponde ao conjunto de vértices e $E$ ao conjunto de arestas. Cada aresta $(u, v) \in E$ possui um peso não negativo $w(u, v)$, que representa o custo de deslocamento entre os vértices $u$ e $v$.

Dado um vértice de origem $s \in V$, o objetivo é determinar, para cada vértice $v \in V$, a menor distância possível entre $s$ e $v$.

O custo de um caminho $P = (v_0, v_1, \ldots, v_k)$ é definido pela soma dos pesos das arestas que o compõem:

$$w(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1}). \tag{30}$$

A distância mínima entre a origem $s$ e um vértice $v$ é definida por:

$$\delta(s, v) = \min_{P \in \mathcal{P}(s,v)} w(P), \tag{31}$$

em que $\mathcal{P}(s, v)$ representa o conjunto de caminhos entre $s$ e $v$. Quando não existe um caminho entre esses vértices, considera-se que $\delta(s, v) = \infty$.

A entrada do programa utiliza o formato DIMACS para problemas de caminhos mínimos. Nesse formato, são informados a quantidade de vértices, a quantidade de arestas e os respectivos vértices de origem, destino e peso de cada aresta.

A saída apresenta, para cada destino alcançável, a distância mínima calculada e a sequência de vértices que compõe o caminho correspondente. Para destinos inalcançáveis, a distância é representada como infinita.

A implementação deve permitir o processamento de grafos de diferentes dimensões, considerando a utilização de vetores para armazenar as informações do grafo e os dados necessários à execução do algoritmo.

## 5.2 Estratégia de solução

Para resolver o problema dos caminhos mínimos, foi utilizado o algoritmo de Dijkstra, que emprega uma estratégia gulosa para determinar as menores distâncias entre um vértice de origem e os demais vértices de um grafo com pesos não negativos.

A estratégia consiste em manter, para cada vértice, uma estimativa da menor distância conhecida a partir da origem. Inicialmente, a distância do vértice de origem é definida como zero, enquanto as distâncias dos demais vértices são consideradas infinitas.

Em cada iteração, o algoritmo seleciona o vértice ainda não visitado que apresenta a menor distância estimada. Como os pesos das arestas são não negativos, essa escolha permite estabelecer definitivamente a menor distância até o vértice selecionado.

Após a seleção, são examinadas as arestas que partem desse vértice, verificando se existe um caminho de menor custo até seus vértices adjacentes. Esse procedimento é denominado relaxamento de arestas.

Considerando uma aresta $(u, v)$ de peso $w(u, v)$, a distância estimada de $v$ é atualizada quando:

$$d[u] + w(u, v) < d[v]. \tag{32}$$

Nesse caso, realiza-se a atualização:

$$d[v] = d[u] + w(u, v). \tag{33}$$

Além da atualização da distância, o vértice $u$ é registrado como predecessor de $v$, permitindo reconstruir posteriormente o caminho mínimo encontrado.

O processo é repetido até que todos os vértices alcançáveis tenham sido visitados. Caso não exista um vértice não visitado com distância finita, o algoritmo é encerrado, pois os vértices restantes não são alcançáveis a partir da origem.

### 5.2.1 Representação do grafo

Para armazenar o grafo, foi utilizada uma representação baseada em vetores indexados, na qual os vértices e as arestas são mantidos em estruturas alocadas dinamicamente. Cada vértice armazena o índice de sua primeira aresta de saída. As arestas, por sua vez, armazenam o vértice de destino, o peso e o índice da próxima aresta associada à mesma origem.

Essa organização permite percorrer as arestas de saída de um vértice sem a necessidade de armazenar uma matriz de adjacência. A representação foi escolhida por permitir o armazenamento de grafos esparsos com consumo de memória proporcional à quantidade de vértices e arestas.

### 5.2.2 Seleção dos vértices

A seleção do próximo vértice a ser processado é realizada por meio de uma busca sequencial em um vetor de distâncias.

Em cada iteração, todos os vértices são examinados para identificar aquele que ainda não foi visitado e apresenta a menor distância estimada.

Essa abordagem dispensa a utilização de estruturas auxiliares mais complexas, como filas de prioridade, mas exige uma varredura dos vértices a cada seleção.

Consequentemente, a etapa de seleção apresenta custo linear em relação ao número de vértices, influenciando diretamente a complexidade total do algoritmo.

### 5.2.3 Reconstrução dos caminhos

Para permitir a apresentação dos caminhos mínimos, foi utilizado um vetor de predecessores. Sempre que o relaxamento de uma aresta produz uma distância menor para determinado vértice, seu predecessor é atualizado. Após a execução de Dijkstra, o caminho entre a origem e um destino pode ser reconstruído percorrendo os predecessores a partir do destino até alcançar a origem.

Como essa reconstrução ocorre na ordem inversa, os vértices são armazenados temporariamente e posteriormente apresentados na sequência correta. Dessa forma, a solução permite obter tanto as distâncias mínimas quanto os caminhos correspondentes para os vértices alcançáveis.

## 5.3 Implementação

A implementação foi desenvolvida utilizando estruturas de dados alocadas dinamicamente para representar o grafo e armazenar as informações necessárias à execução do algoritmo de Dijkstra.

O código foi organizado em arquivos de cabeçalho, responsáveis pela definição das estruturas e declaração das funções, e arquivos de implementação, que contêm as operações sobre o grafo, o algoritmo de caminhos mínimos e o programa principal.

Os arquivos `vertice.h` e `aresta.h` definem as estruturas utilizadas para representar os vértices e as arestas. O arquivo `grafo.h` define a estrutura do grafo e declara suas operações, enquanto `grafo.c` implementa as funções de criação, inserção de arestas, leitura e liberação de memória.

Os arquivos `dijkstra.h` e `dijkstra.c` contêm, respectivamente, as declarações e a implementação das funções relacionadas ao cálculo dos caminhos mínimos. Por fim, o arquivo `main.c` coordena a leitura dos argumentos, o processamento do grafo e a apresentação dos resultados.

### 5.3.1 Estruturas de dados

Para representar o grafo, foram definidas estruturas específicas para vértices, arestas e para o próprio grafo.

A estrutura `Vertice` armazena o identificador de um vértice e o índice de sua primeira aresta de saída, conforme apresentado no Código 3.

```
1 typedef struct {
2     int id;
3     int primeiraAresta;
```

```
} Vertice;
```

Código 3: Estrutura utilizada para representar um vértice.

O campo `id` identifica o vértice, enquanto `primeiraAresta` indica a posição da primeira aresta de saída no vetor de arestas do grafo.

Quando um vértice não possui arestas de saída, esse campo recebe o valor $-1$.

A estrutura `Aresta`, apresentada no Código 4, armazena o destino, o peso e o índice da próxima aresta associada à mesma origem.

```
typedef struct {
    int destino;
    int peso;
    int proxima;
} Aresta;
```

Código 4: Estrutura utilizada para representar uma aresta.

O campo `destino` identifica o vértice de chegada, enquanto `peso` representa o custo da aresta.

O campo `proxima` permite percorrer as demais arestas de saída do mesmo vértice, utilizando índices do vetor. O valor $-1$ indica o término dessa sequência.

A estrutura `Grafo` reúne os vetores de vértices e arestas, além das informações necessárias para controlar suas dimensões.

```
typedef struct {
    int numVertices;
    int numArestas;
    int capacidadeArestas;
    Vertice *vertices;
    Aresta *arestas;
} Grafo;
```

Código 5: Estrutura utilizada para representar o grafo.

O campo `numVertices` representa a quantidade de vértices do grafo, enquanto `numArestas` armazena a quantidade de arestas inseridas.

O campo `capacidadeArestas` indica a capacidade máxima do vetor de arestas alocado inicialmente.

Os ponteiros `vertices` e `arestas` permitem acessar as estruturas armazenadas dinamicamente.

Como os identificadores dos vértices no formato DIMACS começam em 1, o vetor de vértices é alocado com $V + 1$ posições, permitindo o acesso direto pelo identificador.

Além da representação do grafo, foi definida a estrutura `ResultadoDijkstra`, utilizada para armazenar os resultados do algoritmo.

```
typedef struct {
    int numVertices;
    int origem;
    int64_t *distancia;
    int *visitado;
    int *predecessor;
} ResultadoDijkstra;
```

Código 6: Estrutura utilizada para armazenar os resultados de Dijkstra.

O campo `distancia` armazena as menores distâncias conhecidas a partir do vértice de origem, utilizando inteiros de 64 bits para representar os custos acumulados.

O vetor `visitado` identifica os vértices cujas menores distâncias já foram estabelecidas pelo algoritmo.

O vetor `predecessor` registra o vértice anterior no caminho mínimo encontrado, permitindo reconstruir posteriormente a sequência de vértices entre a origem e cada destino.

Os campos `numVertices` e `origem` armazenam, respectivamente, a quantidade de vértices e o identificador do vértice inicial utilizado na execução.

Essa organização separa a representação do grafo dos resultados do processamento, permitindo que as estruturas sejam manipuladas e liberadas de forma independente.

### 5.3.2 Operações básicas do grafo

Para realizar a construção e manipulação do grafo, foram implementadas funções responsáveis pela criação da estrutura, inserção de arestas, leitura dos dados de entrada e liberação da memória alocada.

A função `criarGrafo` recebe a quantidade de vértices e a capacidade máxima de arestas, realizando a alocação dinâmica das estruturas necessárias. O vetor de vértices é alocado com $V + 1$ posições, permitindo que os identificadores sejam utilizados diretamente como índices. Inicialmente, cada vértice recebe seu identificador e o campo `primeiraAresta` é definido como $-1$, indicando que não existem arestas de saída registradas. O vetor de arestas é alocado com capacidade definida a partir das informações presentes no cabeçalho do arquivo de entrada.

A função `inserirAresta` recebe os identificadores dos vértices de origem e destino, além do peso da aresta, e realiza sua inserção na estrutura do grafo.

Antes da inserção, são verificadas a validade dos vértices, a não negatividade do peso e a disponibilidade de espaço no vetor de arestas.

Cada nova aresta é armazenada na próxima posição disponível do vetor, utilizando o campo `numArestas` como índice.

A associação entre as arestas de saída de um mesmo vértice é realizada por meio dos campos `primeiraAresta` e `proxima`, conforme apresentado no Código 7.

```
int indice = grafo->numArestas;

grafo->arestas[indice].destino = destino;
grafo->arestas[indice].peso = peso;
grafo->arestas[indice].proxima =
    grafo->vertices[origem].primeiraAresta;

grafo->vertices[origem].primeiraAresta = indice;
grafo->numArestas++;
```

Código 7: Inserção de uma aresta utilizando índices.

Nesse procedimento, a nova aresta passa a ocupar a primeira posição da sequência de arestas de saída do vértice de origem.

O campo `proxima` da nova aresta armazena o índice da aresta que anteriormente ocupava essa posição, preservando a ligação com as demais arestas do mesmo vértice. Essa estratégia permite inserir uma aresta em tempo constante, sem deslocar os elementos já armazenados no vetor.

A função `lerGrafo` é responsável pela leitura dos dados fornecidos pela entrada padrão, utilizando o formato DIMACS. Nesse formato, as linhas iniciadas por `c` representam comentários, enquanto a linha iniciada por `p sp` informa a quantidade de vértices e arestas do grafo.

As linhas iniciadas por `a` descrevem as arestas, apresentando os identificadores dos vértices de origem e destino e o respectivo peso.

Um exemplo simplificado desse formato é apresentado no Código 8.

```
c Grafo de exemplo
p sp 5 6
a 1 2 10
a 1 3 3
a 3 2 4
a 2 4 2
a 3 4 8
a 4 5 1
```

Código 8: Exemplo de entrada no formato DIMACS.

A leitura é realizada linha a linha. Após identificar o cabeçalho do problema, a função cria o grafo com as dimensões informadas e utiliza `inserirAresta` para registrar as arestas encontradas.

Durante o processamento, são verificadas possíveis inconsistências nos dados, incluindo identificadores inválidos, pesos negativos e divergências entre a quantidade declarada e a quantidade de arestas efetivamente inseridas.

Por fim, a função `destruirGrafo` realiza a liberação dos vetores de vértices e arestas, seguida da liberação da própria estrutura `Grafo`.

Essa operação permite recuperar a memória alocada dinamicamente após o processamento, evitando que as estruturas permaneçam ocupando recursos desnecessariamente. A separação dessas operações em funções específicas contribui para a organização do código e permite que a representação do grafo seja utilizada independentemente da implementação do algoritmo de Dijkstra.

### 5.3.3 Implementação do algoritmo de Dijkstra

A função `dijkstra` implementa o procedimento principal para determinar os caminhos mínimos a partir de um vértice de origem em um grafo com pesos não negativos.

A função recebe um ponteiro para a estrutura `Grafo` e o identificador do vértice de origem, retornando uma estrutura `ResultadoDijkstra` contendo as distâncias calculadas, os predecessores e as informações dos vértices visitados.

**Inicialização dos vetores** Inicialmente, são alocados os vetores necessários ao processamento do algoritmo.

O vetor `distancia` armazena as menores distâncias conhecidas a partir da origem. Seus elementos são inicializados com `INT64_MAX`, utilizado para representar uma distância infinita.

O vetor `predecessor` é inicializado com $-1$, indicando que ainda não existe um predecessor definido para cada vértice.

O vetor `visitado` é inicializado com zero, indicando que nenhum vértice foi processado.

Após essas inicializações, a distância do vértice de origem é definida como zero, conforme apresentado no Código 9.

```
for (int i = 1; i <= grafo->numVertices; i++) {
    resultado->distancia[i] = INT64_MAX;
    resultado->predecessor[i] = -1;
    resultado->visitado[i] = 0;
}

resultado->distancia[origem] = 0;
```

Código 9: Inicialização dos vetores do algoritmo de Dijkstra.

Essa configuração estabelece as condições iniciais necessárias para o cálculo das menores distâncias, considerando que apenas a origem possui distância conhecida no início da execução.

**Seleção do vértice de menor distância** Em cada iteração, o algoritmo procura o vértice ainda não visitado que apresenta a menor distância estimada.

Essa seleção é realizada por meio de uma busca sequencial, percorrendo todos os vértices armazenados no grafo.

O procedimento é apresentado no Código 10.

```
int u = -1;
int64_t menorDistancia = INT64_MAX;

for (int v = 1; v <= n; v++) {
            if (!resultado->visitado[v] && resultado->distancia[v] <
                menorDistancia) {
                menorDistancia = resultado->distancia[v];
                u = v;
            }
        }

if (u == -1)
    break;

resultado->visitado[u] = 1;
```

Código 10: Seleção do vértice de menor distância.

A variável `u` armazena o identificador do vértice selecionado, enquanto `menor` mantém a menor distância encontrada durante a busca. Caso nenhum vértice não visitado possua distância finita, a variável `u` permanece igual a $-1$, encerrando o processamento.

Quando um vértice é selecionado, sua posição no vetor `visitado` é marcada com o valor 1, indicando que sua menor distância foi estabelecida definitivamente.

**Relaxamento das arestas** Após selecionar um vértice, o algoritmo percorre suas arestas de saída para verificar se é possível reduzir as distâncias estimadas dos vértices adjacentes.

A navegação é realizada utilizando o índice da primeira aresta do vértice e o campo `proxima` das estruturas `Aresta`.

O procedimento de relaxamento é apresentado no Código 11.

```
int indice = grafo->vertices[u].primeiraAresta;

 while (indice != -1) {
            Aresta aresta = grafo->arestas[indice];

            int v = aresta.destino;
            int peso = aresta.peso;

            if (!resultado->visitado[v] && resultado->distancia[u] <=
                INT64_MAX - peso) {
                int64_t novaDistancia = resultado->distancia[u] + peso;

                if (novaDistancia < resultado->distancia[v]) {
                    resultado->distancia[v] = novaDistancia;
                    resultado->predecessor[v] = u;
                }
            }

            indice = aresta.proxima;
        }
```

Código 11: Percurso e relaxamento das arestas.

Para cada aresta $(u, v)$, verifica-se se o caminho que passa pelo vértice `u` apresenta custo inferior à distância anteriormente registrada para `v`.

Quando essa condição é satisfeita, a distância de `v` é atualizada e seu predecessor passa a ser o vértice `u`.

A condição que utiliza `INT64_MAX - peso` evita que a soma entre a distância acumulada e o peso da aresta ultrapasse o limite do tipo inteiro utilizado.

Além disso, o relaxamento considera apenas vértices ainda não visitados, pois as menores distâncias dos vértices já processados foram estabelecidas anteriormente.

**Encerramento do algoritmo** O processamento continua enquanto existirem vértices não visitados com distância finita, respeitando o limite de iterações correspondente à quantidade de vértices do grafo. Ao final, a estrutura `ResultadoDijkstra` contém as menores distâncias encontradas a partir da origem e os predecessores necessários para reconstruir os caminhos correspondentes. Os vértices inalcançáveis permanecem com distância igual a `INT64_MAX`, permitindo identificá-los durante a apresentação dos resultados. A função retorna um ponteiro para a estrutura de resultados, que pode ser utilizada pelas funções de impressão e posteriormente liberada da memória.

### 5.3.4 Reconstrução dos caminhos

Após a execução do algoritmo de Dijkstra, as menores distâncias a partir do vértice de origem são armazenadas no vetor `distancia`. Entretanto, para apresentar a sequência de vértices que compõe cada caminho mínimo, é necessário utilizar as informações registradas no vetor `predecessor`.

A função `imprimirCaminho` é responsável por reconstruir e apresentar o caminho entre o vértice de origem e um destino específico.

O procedimento inicia no vértice de destino e percorre sucessivamente seus predecessores até alcançar o vértice de origem.

Durante esse percurso, os identificadores dos vértices são armazenados em um vetor auxiliar, pois a sequência obtida encontra-se na ordem inversa do caminho original.

O procedimento de reconstrução é apresentado no Código 12.

```
int atual = destino;
int tamanho = 0;

while (atual != -1 && tamanho < resultado->numVertices) {
    caminho[tamanho++] = atual;
    atual = resultado->predecessor[atual];
}
```

Código 12: Reconstrução do caminho por predecessores.

Após o preenchimento do vetor `caminho`, seus elementos são percorridos em ordem inversa, permitindo apresentar a sequência correta de vértices, desde a origem até o destino. Por exemplo, considerando um caminho mínimo formado pelos vértices $1 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 5$, a reconstrução por predecessores produz inicialmente a sequência $5, 4, 2, 3, 1$.

A inversão dessa sequência permite obter o caminho na ordem correta. Antes da reconstrução, a função verifica se o destino é válido e se existe um caminho a partir da origem. Quando a distância armazenada para o destino corresponde a `INT64_MAX`, considera-se que o vértice é inalcançável, não sendo possível apresentar um caminho até ele.

### 5.3.5 Apresentação de todos os caminhos

Além da reconstrução de um caminho específico, foi implementada a função `imprimirTodosCaminhos`, responsável por apresentar os resultados para todos os vértices de destino, exceto a própria origem.

A função percorre os identificadores dos vértices em ordem crescente, consultando as distâncias calculadas e reconstruindo os respectivos caminhos por meio do vetor de predecessores.

Para os vértices alcançáveis, a saída utiliza o formato:

```
[origem,destino](distancia) caminho
```

Código 13: Formato de saída dos caminhos mínimos.

Como exemplo, considerando o grafo de teste utilizado na implementação, o caminho mínimo entre os vértices 1 e 5 é apresentado da seguinte forma:

```
[1,5](10) 1 3 2 4 5
```

Código 14: Exemplo de saída de um caminho mínimo.

Nesse formato, os valores entre colchetes identificam os vértices de origem e destino, o valor entre parênteses representa a distância mínima calculada e a sequência seguinte apresenta os vértices que compõem o caminho. Para vértices inalcançáveis, a distância é representada por `INF`, indicando que não existe um caminho entre a origem e o destino.

A padronização da saída permite comparar os resultados produzidos pelo programa com os caminhos mínimos disponibilizados nos arquivos de referência utilizados durante a etapa de validação.

Dessa forma, as funções de reconstrução e impressão complementam o algoritmo de Dijkstra, permitindo apresentar não apenas as distâncias mínimas, mas também os caminhos correspondentes.

### 5.3.6 Programa principal

O arquivo `main.c` é responsável por coordenar a execução do programa, realizando a interpretação dos argumentos fornecidos pela linha de comando, a leitura do grafo, a execução do algoritmo de Dijkstra e a apresentação dos resultados.

O programa permite informar o vértice de origem e, opcionalmente, um vértice de destino específico. Quando nenhum argumento é fornecido, o vértice 1 é utilizado como origem padrão.

A interpretação dos argumentos é realizada por meio da função `strtol`, que permite converter os valores recebidos como texto para números inteiros, verificando possíveis erros de conversão.

Após a validação dos argumentos, a função `lerGrafo` é chamada para construir a representação do grafo a partir dos dados fornecidos pela entrada padrão.

Em seguida, são verificadas as condições necessárias para a execução, incluindo a validade dos identificadores dos vértices de origem e destino.

Com o grafo carregado, a função `dijkstra` é executada para calcular os caminhos mínimos a partir do vértice de origem selecionado.

**Medição do tempo de execução** Para avaliar o desempenho da implementação, foi utilizada a função `clock`, disponibilizada pela biblioteca `time.h` da linguagem C.

A medição é realizada imediatamente antes e depois da chamada da função `dijkstra`, permitindo registrar o tempo de CPU consumido especificamente pelo algoritmo.

O procedimento utilizado é apresentado no Código 15.

```
clock_t inicio = clock();

ResultadoDijkstra *resultado =
    dijkstra(grafo, origem);

clock_t fimTempo = clock();

double tempo =
    (fimTempo - inicio) / (double)CLOCKS_PER_SEC;
```

Código 15: Medição do tempo de execução do algoritmo.

A diferença entre os valores retornados por `clock` representa a quantidade de unidades de tempo de CPU consumidas durante o intervalo medido.

A divisão por `CLOCKS_PER_SEC` converte essa diferença para segundos, permitindo registrar os tempos utilizados na análise experimental.

Como a medição envolve apenas a chamada do algoritmo, não são contabilizadas diretamente as etapas de leitura do grafo e impressão dos caminhos.

O tempo calculado é apresentado na saída de erro padrão, utilizando `stderr`, para evitar interferências no formato da saída principal do programa.

**Apresentação dos resultados** Após a execução do algoritmo, o programa verifica se foi informado um vértice de destino específico.

Quando um destino é fornecido, são apresentadas a menor distância calculada e a sequência de vértices que compõe o caminho correspondente.

Caso nenhum destino seja informado, a função `imprimirTodosCaminhos` é utilizada para apresentar os caminhos mínimos entre a origem e todos os demais vértices do grafo.

Os resultados são enviados para a saída padrão, permitindo seu redirecionamento para arquivos de texto utilizados na validação.

Por exemplo, a execução do programa com o grafo de Roma pode ser realizada por meio do seguinte comando:

```
./tarefa02.exe < tests/rome99c.gr > resultado_roma.txt
```

Código 16: Execução do programa com o grafo de Roma.

Nesse procedimento, o arquivo `rome99c.gr` é utilizado como entrada e os caminhos mínimos calculados são armazenados em `resultado_roma.txt`.

O tempo de execução do algoritmo permanece disponível no terminal, pois é apresentado separadamente pela saída de erro padrão.

**Liberação de memória** Ao final do processamento, o programa realiza a liberação das estruturas alocadas dinamicamente.

A função `destruirResultadoDijkstra` libera os vetores de distâncias, predecessores e vértices visitados, além da estrutura que armazena os resultados.

Em seguida, a função `destruirGrafo` libera os vetores de vértices e arestas, bem como a própria estrutura do grafo.

A implementação também verifica possíveis falhas durante a leitura dos dados, a validação dos argumentos e a execução do algoritmo, encerrando o programa quando não é possível prosseguir corretamente. Dessa forma, o programa principal integra as operações de leitura, processamento, apresentação dos resultados e gerenciamento de memória, mantendo a implementação do algoritmo de Dijkstra separada das operações de entrada e saída.

## 5.4 Análise de complexidade

A análise de complexidade considera o número de vértices $V$ e o número de arestas $E$ do grafo, avaliando a quantidade de operações realizadas pelo algoritmo de Dijkstra e a memória necessária para sua execução.

A implementação utiliza vetores indexados para representar o grafo e uma busca sequencial para selecionar o próximo vértice a ser processado.

### 5.4.1 Complexidade de tempo

O algoritmo de Dijkstra é executado por meio de um laço principal que realiza, no máximo, $V$ iterações, uma vez que cada vértice pode ser selecionado e marcado como visitado apenas uma vez.

Em cada iteração, o algoritmo percorre o vetor de vértices para identificar aquele que ainda não foi visitado e possui a menor distância estimada.

Como essa busca examina até $V$ vértices, seu custo é $\Theta(V)$ por iteração.

Considerando as $V$ iterações, o custo total da seleção dos vértices é:

$$T_{\text{selecao}}(V) = V \cdot \Theta(V) = \Theta(V^2). \tag{34}$$

Após selecionar um vértice, o algoritmo percorre suas arestas de saída e realiza o procedimento de relaxamento.

Cada aresta é examinada, no máximo, uma vez durante a execução, pois as arestas de saída de um vértice são percorridas apenas quando esse vértice é selecionado.

Como cada relaxamento envolve um número constante de operações, o custo total dessa etapa é:

$$T_{\text{relaxamento}}(V, E) = O(E). \tag{35}$$

A inicialização dos vetores de distâncias, predecessores e vértices visitados exige uma varredura dos $V$ vértices, apresentando custo $\Theta(V)$. Portanto, o tempo total do algoritmo pode ser representado pela expressão:

$$T(V, E) = \Theta(V) + O(V^2) + O(E). \tag{36}$$

Considerando o termo dominante, obtém-se:

$$\boxed{T(V, E) = O(V^2 + E)}. \tag{37}$$

Como a seleção do vértice de menor distância é realizada por busca sequencial, a implementação apresenta custo quadrático em relação ao número de vértices no pior caso.

Para grafos direcionados simples, em que $E = O(V^2)$, a expressão pode ser simplificada para:

$$\boxed{T(V) = O(V^2)}. \tag{38}$$

A utilização de uma fila de prioridade poderia reduzir o custo da seleção dos vértices. Entretanto, a implementação desenvolvida utiliza exclusivamente a busca sequencial, conforme a estratégia adotada para esta tarefa.

### 5.4.2 Complexidade de espaço

A implementação utiliza alocação dinâmica para armazenar os vértices, as arestas e os vetores auxiliares necessários ao algoritmo.

A representação do grafo possui um vetor de $V+1$ posições para armazenar os vértices e um vetor com capacidade para $E$ arestas.

Dessa forma, o espaço necessário para armazenar o grafo é:

$$S_{\text{grafo}}(V, E) = \Theta(V + E). \tag{39}$$

O algoritmo de Dijkstra utiliza três vetores auxiliares:

- `distancia`: armazena as distâncias estimadas dos $V$ vértices.
- `visitado`: identifica os vértices que já foram processados.
- `predecessor`: armazena os predecessores utilizados na reconstrução dos caminhos.

Cada um desses vetores possui tamanho proporcional à quantidade de vértices.

Consequentemente, o espaço auxiliar utilizado pelo algoritmo é:

$$S_{\text{Dijkstra}}(V) = \Theta(V). \tag{40}$$

Considerando a representação do grafo e os vetores auxiliares, o espaço total utilizado pela implementação é:

$$S(V, E) = \Theta(V + E) + \Theta(V). \tag{41}$$

Portanto:

$$\boxed{S(V, E) = \Theta(V + E)}. \tag{42}$$

Essa representação apresenta uma vantagem em relação à matriz de adjacência, que exige $\Theta(V^2)$ posições independentemente da quantidade de arestas existentes.

Para grafos esparsos, nos quais a quantidade de arestas é significativamente menor que $V^2$, a utilização de vetores indexados permite reduzir o consumo de memória.

### 5.4.3 Complexidade da reconstrução dos caminhos

Após a execução do algoritmo, os caminhos mínimos podem ser reconstruídos utilizando o vetor de predecessores.

Para reconstruir um caminho entre a origem e um destino, é necessário percorrer a sequência de predecessores até alcançar o vértice inicial.

Considerando um caminho formado por $k$ arestas, o procedimento realiza $k+1$ acessos ao vetor de predecessores.

Assim, o custo da reconstrução de um caminho é:

$$T_{\text{caminho}}(k) = \Theta(k + 1). \tag{43}$$

Como um caminho mínimo pode ser representado sem ciclos, seu número de arestas é limitado por $V - 1$.

Portanto, no pior caso, a reconstrução de um único caminho apresenta:

$$T_{\text{caminho}}(V) = O(V). \tag{44}$$

Quando são reconstruídos e apresentados os caminhos para todos os destinos, o custo total pode atingir $O(V^2)$, pois cada caminho pode conter até $V$ vértices.

O vetor auxiliar utilizado para armazenar temporariamente um caminho possui tamanho proporcional ao número de vértices, resultando em espaço auxiliar $O(V)$.

Dessa forma, a reconstrução dos caminhos não altera a ordem assintótica quadrática do tempo total no pior caso, considerando a implementação com busca sequencial.

### 5.4.4 Complexidade das operações do grafo

Além do algoritmo de Dijkstra, foram analisadas as operações utilizadas na construção e manipulação do grafo.

A função `criarGrafo` realiza a alocação dos vetores e a inicialização das informações dos vértices, apresentando custo de tempo $O(V)$.

A função `inserirAresta` armazena cada nova aresta na próxima posição disponível do vetor e atualiza os índices necessários para manter o encadeamento das arestas de saída.

Como essas operações não dependem da quantidade de vértices ou arestas já inseridas, cada inserção apresenta custo $\Theta(1)$.

A função `lerGrafo` realiza a leitura das informações dos vértices e das arestas do arquivo de entrada.

Considerando a leitura de um arquivo DIMACS com $V$ vértices e $E$ arestas, e desconsiderando o custo das linhas de comentário, o processamento apresenta custo $O(V + E)$.

Por fim, a função `destruirGrafo` libera os vetores de vértices e arestas e a própria estrutura do grafo.

A análise dessas operações demonstra que a representação por vetores indexados permite construir e armazenar o grafo de maneira eficiente, sem a necessidade de utilizar uma matriz de adjacência.

Considerando a construção do grafo, a execução de Dijkstra e a apresentação dos caminhos mínimos, a complexidade de tempo no pior caso permanece limitada por:

$$\boxed{T(V, E) = O(V^2 + E)}. \tag{45}$$

O espaço total utilizado permanece proporcional à quantidade de vértices e arestas:

$$\boxed{S(V, E) = \Theta(V + E)}. \tag{46}$$

## 5.5 Experimentos e resultados

Para avaliar o funcionamento e o desempenho da implementação do algoritmo de Dijkstra, foram realizados experimentos utilizando grafos de diferentes dimensões.

Os testes tiveram como objetivos verificar a correção dos caminhos mínimos calculados, avaliar o tempo de execução do algoritmo e observar o comportamento da implementação em função do crescimento do número de vértices e arestas.

### 5.5.1 Metodologia experimental

Os experimentos foram realizados utilizando grafos direcionados e ponderados, representados no formato DIMACS.

Inicialmente, foi utilizado um grafo pequeno, construído para verificar o funcionamento das operações básicas, a atualização das distâncias e a reconstrução dos caminhos mínimos.

Posteriormente, foram utilizados dois grafos de redes rodoviárias de dimensões distintas: um correspondente à cidade de Roma e outro à rede rodoviária de Nova York.

As características dos grafos utilizados são apresentadas na Tabela 2.

Tabela 2: Características dos grafos utilizados nos experimentos.

| **Grafo** | **Vértices** | **Arestas** |
|---|---:|---:|
| Roma | 3.353 | 8.859 |
| Nova York | 264.346 | 730.100 |

O grafo de Roma foi utilizado para avaliar o algoritmo em uma rede de menor dimensão, enquanto o grafo de Nova York permitiu observar seu comportamento em uma rede significativamente maior.

Em ambos os casos, o vértice de origem utilizado nos experimentos foi o vértice 1.

**Procedimento de execução** A implementação foi desenvolvida na linguagem C e compilada utilizando o compilador GCC, com as opções de compilação `-Wall`, `-Wextra`, `-std=c11` e `-O2`.

Os experimentos foram realizados a partir da execução do programa em terminal, utilizando o redirecionamento da entrada padrão para carregar os arquivos DIMACS.

Para cada grafo, o algoritmo foi executado cinco vezes, mantendo as mesmas condições de entrada.

O tempo de execução foi medido por meio da função `clock`, considerando exclusivamente o intervalo correspondente à execução da função `dijkstra`.

Dessa forma, os tempos registrados não incluem diretamente a leitura do arquivo de entrada nem a impressão dos caminhos mínimos.

As medições foram expressas em segundos e utilizadas para calcular a média aritmética e o desvio-padrão amostral dos tempos observados.

A média aritmética é definida pela Equação 47.

$$\bar{t} = \frac{1}{n} \sum_{i=1}^{n} t_i, \tag{47}$$

em que $t_i$ representa o tempo registrado na execução $i$ e $n$ corresponde à quantidade de execuções realizadas.

O desvio-padrão amostral é calculado pela Equação 48.

$$s = \sqrt{\frac{\sum_{i=1}^{n} (t_i - \bar{t})^2}{n-1}}. \tag{48}$$

Essas medidas permitem avaliar tanto o tempo médio de execução quanto a variação entre as repetições realizadas.

**Validação dos resultados** Além da avaliação de desempenho, foi considerado um procedimento de validação dos caminhos mínimos calculados.

Inicialmente, o grafo pequeno foi utilizado para verificar manualmente as distâncias e os caminhos encontrados. Para o grafo de Roma, foi utilizado como referência o arquivo `spathsRome99c.gr`, contendo informações de caminhos mínimos associados ao grafo utilizado nos testes. Foi preparado um script em Python para auxiliar a comparação entre os resultados gerados pela implementação e os dados de referência.

O procedimento permite verificar a correspondência entre as distâncias calculadas e as distâncias esperadas, além de avaliar a consistência dos caminhos apresentados. A comparação das distâncias é especialmente importante porque diferentes caminhos podem apresentar o mesmo custo mínimo. Dessa forma, a validação não deve depender exclusivamente da igualdade entre as sequências de vértices, mas também do custo total dos caminhos.

Os resultados quantitativos dos experimentos são apresentados nas subsubseções seguintes.

### 5.5.2 Resultados dos experimentos

Os experimentos foram realizados com os grafos de Roma e Nova York, utilizando cinco execuções independentes para cada conjunto de dados. Os tempos registrados correspondem exclusivamente à execução do algoritmo de Dijkstra, sem considerar as etapas de leitura do grafo e apresentação dos caminhos mínimos.

**Resultados para o grafo de Roma** O primeiro conjunto de experimentos utilizou o grafo de Roma, composto por 3.353 vértices e 8.859 arestas.

Os tempos obtidos nas cinco execuções são apresentados na Tabela 3.

Tabela 3: Tempos de execução do algoritmo de Dijkstra para o grafo de Roma.

| **Execução** | **Tempo (s)** |
|---|---|
| 1 | 0,008882 |
| 2 | 0,008872 |
| 3 | 0,008771 |
| 4 | 0,008606 |
| 5 | 0,006677 |
| Média | 0,008362 |
| Desvio-padrão | 0,000948 |

A média dos tempos registrados foi de aproximadamente 0,008362 segundos, enquanto o desvio-padrão amostral foi de aproximadamente 0,000948 segundos. Os resultados indicam que a implementação conseguiu processar o grafo de Roma em poucos milissegundos. Observa-se que as quatro primeiras execuções apresentaram tempos próximos, enquanto a quinta execução registrou um valor inferior.

**Resultados para o grafo de Nova York** O segundo conjunto de experimentos utilizou o grafo de Nova York, composto por 264.346 vértices e 730.100 arestas.

Os tempos registrados são apresentados na Tabela 4.

Tabela 4: Tempos de execução do algoritmo de Dijkstra para o grafo de Nova York.

| **Execução** | **Tempo (s)** |
|---|---|
| 1 | 44,318441 |
| 2 | 42,966839 |
| 3 | 46,213813 |
| 4 | 43,582669 |
| 5 | 43,038672 |
| Média | 44,024087 |
| Desvio-padrão | 1,338396 |

Para o grafo de Nova York, o tempo médio de execução foi de aproximadamente 44,024087 segundos, com desvio-padrão amostral de aproximadamente 1,338396 segundos.

Os tempos registrados apresentaram maior variação absoluta em comparação aos resultados obtidos para o grafo de Roma.

A menor medição foi de 42,966839 segundos, enquanto a maior foi de 46,213813 segundos.

O aumento significativo no tempo de execução está associado à maior dimensão do grafo e ao custo da busca sequencial utilizada para selecionar os vértices durante o algoritmo.

**Resumo dos resultados** A Tabela 5 apresenta uma comparação dos principais resultados obtidos nos experimentos.

Tabela 5: Resumo dos tempos de execução dos grafos avaliados.

| **Métrica** | **Roma** | **Nova York** |
|---|---|---|
| Vértices | 3.353 | 264.346 |
| Arestas | 8.859 | 730.100 |
| Execuções | 5 | 5 |
| Tempo médio (s) | 0,008362 | 44,024087 |
| Desvio-padrão (s) | 0,000948 | 1,338396 |

Os resultados demonstram uma diferença expressiva entre os tempos de processamento dos dois grafos avaliados. Enquanto o grafo de Roma foi processado em uma fração de segundo, o grafo de Nova York exigiu aproximadamente 44 segundos de tempo de CPU, em média. Essa diferença evidencia a importância de considerar o número de vértices e a estratégia de seleção utilizada pelo algoritmo ao trabalhar com grafos de grande dimensão.

Uma análise mais detalhada da relação entre os tempos observados e a complexidade teórica da implementação é apresentada a seguir.

### 5.5.3 Comparação entre os grafos de Roma e Nova York

A comparação entre os resultados obtidos para os grafos de Roma e Nova York permite analisar o comportamento da implementação do algoritmo de Dijkstra diante do aumento da quantidade de vértices e arestas.

O grafo de Roma possui 3.353 vértices e 8.859 arestas, enquanto o grafo de Nova York possui 264.346 vértices e 730.100 arestas.

A razão entre as quantidades de vértices dos dois grafos é dada por:

$$R_V = \frac{V_{\text{NY}}}{V_{\text{Roma}}} = \frac{264346}{3353} \approx 78{,}84. \tag{49}$$

De maneira semelhante, a razão entre as quantidades de arestas é:

$$R_E = \frac{E_{\text{NY}}}{E_{\text{Roma}}} = \frac{730100}{8859} \approx 82{,}41. \tag{50}$$

Portanto, o grafo de Nova York possui aproximadamente 78,84 vezes mais vértices e 82,41 vezes mais arestas que o grafo de Roma.

Entretanto, o crescimento do tempo de execução foi consideravelmente superior.

Utilizando os tempos médios obtidos nos experimentos, a razão entre os tempos é calculada por:

$$R_T = \frac{\bar{t}_{\text{NY}}}{\bar{t}_{\text{Roma}}} = \frac{44{,}024087}{0{,}0083616} \approx 5265. \tag{51}$$

Assim, o tempo médio de execução para o grafo de Nova York foi aproximadamente 5.265 vezes maior que o tempo médio registrado para Roma.

Essa diferença pode ser relacionada à complexidade de tempo da implementação:

$$T(V, E) = O(V^2 + E). \tag{52}$$

Como a seleção do próximo vértice é realizada por busca sequencial, essa operação pode exigir até $V$ comparações em cada uma das $V$ iterações.

Consequentemente, o custo quadrático associado à quantidade de vértices tende a apresentar grande influência no tempo de execução para grafos de maiores dimensões.

Considerando apenas o crescimento do termo quadrático, a razão teórica entre os custos é dada por:

$$R_{V^2} = \left(\frac{V_{\text{NY}}}{V_{\text{Roma}}}\right)^2. \tag{53}$$

Substituindo os valores:

$$R_{V^2} = \left(\frac{264346}{3353}\right)^2 \approx 6215. \tag{54}$$

A Tabela 6 apresenta um resumo das razões de crescimento.

Tabela 6: Comparação das razões de crescimento entre os grafos.

| **Grandeza** | **Razão aproximada** |
|---|---|
| Número de vértices | 78,84 |
| Número de arestas | 82,41 |
| Tempo médio de execução | 5.265 |
| Termo quadrático $V^2$ | 6.215 |

Observa-se que o crescimento experimental do tempo de execução apresenta a mesma ordem de grandeza do crescimento do termo quadrático $V^2$.

Esse comportamento é compatível com a análise de complexidade realizada anteriormente, na qual a busca sequencial dos vértices apresenta custo $O(V^2)$.

Entretanto, os valores não precisam coincidir exatamente, pois a complexidade assintótica descreve o comportamento do algoritmo em função do tamanho da entrada, desconsiderando constantes multiplicativas e fatores específicos da execução. Além disso, o número de vértices efetivamente processados, a estrutura das conexões do grafo e as condições do ambiente computacional podem influenciar os tempos observados. Deve-se considerar também que a comparação utiliza apenas dois grafos de dimensões diferentes, não sendo suficiente para determinar experimentalmente uma função exata de crescimento.

Apesar dessa limitação, os resultados são consistentes com o comportamento esperado para a implementação de Dijkstra com seleção sequencial dos vértices. A utilização de uma estrutura de fila de prioridade, como um heap binário, poderia reduzir o custo da seleção dos vértices, permitindo uma implementação com complexidade de tempo $O((V + E) \log V)$.

Dessa forma, os experimentos evidenciam que a escolha da estrutura utilizada para selecionar os vértices exerce influência significativa sobre o desempenho do algoritmo, principalmente em grafos com grande quantidade de vértices.

### 5.5.4 Análise dos resultados

Os experimentos realizados permitiram avaliar o comportamento da implementação do algoritmo de Dijkstra em grafos com diferentes quantidades de vértices e arestas. Os resultados demonstraram uma diferença significativa no tempo de execução entre os grafos de Roma e Nova York. Para o grafo de Roma, o algoritmo apresentou tempo médio de aproximadamente 0,008362 segundos, enquanto para o grafo de Nova York o tempo médio foi de aproximadamente 44,024087 segundos.

Esse aumento expressivo está relacionado principalmente à estratégia utilizada para selecionar o próximo vértice a ser processado. Na implementação desenvolvida, essa operação é realizada por meio de uma busca sequencial no vetor de vértices, apresentando custo $O(V)$ por seleção. Como essa busca pode ser repetida até $V$ vezes, seu custo acumulado pode atingir $O(V^2)$. Dessa forma, o aumento da quantidade de vértices produz um impacto significativo no tempo de execução, mesmo quando o número de arestas cresce de maneira aproximadamente proporcional.

**Influência da representação do grafo** A utilização de vetores indexados para representar as listas de adjacência permitiu armazenar os grafos com espaço proporcional a $V + E$. Essa característica é especialmente relevante para os grafos utilizados nos experimentos, pois ambos apresentam uma quantidade de arestas relativamente pequena em comparação ao número máximo de conexões possíveis entre seus vértices. Uma representação baseada em matriz de adjacência exigiria espaço proporcional a $V^2$, independentemente da quantidade de arestas existentes.

A representação por vetores indexados também permite percorrer diretamente as arestas de saída de cada vértice, sem examinar posições correspondentes a arestas inexistentes.

Entretanto, essa estrutura não elimina o custo quadrático da implementação, pois a seleção do vértice de menor distância continua sendo realizada por busca sequencial. Assim, a representação do grafo contribui principalmente para a eficiência do armazenamento e do percurso das arestas, enquanto o método de seleção dos vértices determina o principal custo assintótico da implementação.

**Variação dos tempos de execução** As cinco execuções realizadas para cada grafo apresentaram variações nos tempos registrados. No grafo de Roma, o desvio-padrão amostral foi de aproximadamente 0,000948 segundos. Para o grafo de Nova York, o desvio-padrão amostral foi de aproximadamente 1,338396 segundos.

Essas diferenças podem estar associadas a fatores do ambiente computacional, como o escalonamento de processos, a utilização dos recursos do sistema e o comportamento da memória durante a execução.

Como a medição foi realizada utilizando a função `clock`, os valores representam o tempo de CPU consumido pelo trecho analisado, não necessariamente o tempo total decorrido entre o início e o fim do programa. Além disso, a medição foi limitada à execução da função `dijkstra`, não incluindo diretamente o tempo necessário para carregar o arquivo DIMACS ou imprimir os resultados.

Essa separação permite avaliar o desempenho do algoritmo sem incluir o custo das operações de entrada e saída.

**Limitações dos experimentos** Embora os resultados obtidos sejam compatíveis com a análise teórica, os experimentos apresentam algumas limitações.

A avaliação de desempenho foi realizada utilizando apenas dois grafos de redes rodoviárias, com dimensões significativamente diferentes. Consequentemente, não é possível determinar uma função exata de crescimento do tempo de execução apenas com essas medições. Além disso, as características estruturais dos grafos, como a distribuição das arestas e a quantidade de vértices alcançáveis a partir da origem, podem influenciar o número efetivo de operações realizadas.

A utilização de um conjunto maior de grafos, com quantidades progressivamente crescentes de vértices e arestas, permitiria avaliar o comportamento experimental de maneira mais detalhada. Também seria possível comparar a implementação desenvolvida com outras versões do algoritmo, especialmente aquelas que utilizam filas de prioridade para selecionar os vértices.

**Considerações sobre o desempenho** De maneira geral, os experimentos demonstraram que a implementação apresenta tempos reduzidos para grafos de menor dimensão, mas seu desempenho é significativamente afetado pelo crescimento da quantidade de vértices.

A comparação entre Roma e Nova York apresentou um crescimento experimental do tempo de execução compatível com a influência do termo quadrático presente na análise de complexidade. A representação do grafo por vetores indexados mostrou-se adequada para armazenar redes rodoviárias esparsas, mantendo o consumo de memória proporcional à quantidade de vértices e arestas. Entretanto, a busca sequencial utilizada na seleção dos vértices constitui uma limitação importante para o processamento de grafos de grande dimensão. Portanto, os resultados reforçam a importância da escolha das estruturas de dados e das estratégias de implementação na eficiência dos algoritmos de caminhos mínimos.

## 5.6 Considerações da tarefa

Nesta tarefa, foi desenvolvido um programa em linguagem C para determinar os caminhos mínimos a partir de um vértice de origem, utilizando o algoritmo de Dijkstra em grafos direcionados com pesos não negativos.

A implementação utilizou uma representação do grafo baseada em vetores indexados, na qual os vértices e as arestas são armazenados em estruturas alocadas dinamicamente. Essa abordagem permitiu representar as listas de adjacência por meio de índices, evitando a utilização de ponteiros individuais para conectar as arestas de um mesmo vértice.

O algoritmo foi implementado utilizando uma estratégia gulosa, selecionando a cada iteração o vértice não visitado com a menor distância estimada e realizando o relaxamento de suas arestas de saída. Além do cálculo das distâncias mínimas, foi implementado o armazenamento dos predecessores, permitindo reconstruir e apresentar os caminhos encontrados entre a origem e os demais vértices. A análise de complexidade demonstrou que a implementação apresenta tempo $O(V^2 + E)$, devido principalmente à busca sequencial utilizada para selecionar os vértices.

O espaço total necessário é $\Theta(V + E)$, considerando a representação do grafo e os vetores auxiliares utilizados pelo algoritmo.

Nos experimentos realizados, foram utilizados grafos de redes rodoviárias correspondentes a Roma e Nova York, com diferentes quantidades de vértices e arestas.

O tempo médio de execução foi de aproximadamente 0,008362 segundos para o grafo de Roma e 44,024087 segundos para o grafo de Nova York. A diferença entre os tempos observados foi compatível com a influência do custo quadrático da seleção sequencial dos vértices, conforme identificado na análise teórica. Os resultados também evidenciaram a importância da representação do grafo por vetores indexados, especialmente em redes esparsas, nas quais o armazenamento de uma matriz de adjacência exigiria uma quantidade significativamente maior de memória.

Entretanto, a implementação apresenta limitações de desempenho para grafos de grande dimensão, em razão do custo associado à seleção dos vértices. Como possibilidade de melhoria, poderia ser utilizada uma fila de prioridade baseada em heap binário, reduzindo a complexidade de tempo para $O((V + E) \log V)$. Outra possibilidade seria ampliar os experimentos para grafos de diferentes dimensões e características, permitindo uma avaliação mais detalhada do comportamento do algoritmo.

De maneira geral, a tarefa permitiu aplicar conceitos relacionados a algoritmos gulosos, estruturas de dados para representação de grafos, análise de complexidade e avaliação experimental de desempenho.

A implementação também demonstrou como diferentes decisões sobre estruturas de dados e operações fundamentais podem influenciar a eficiência de um algoritmo, mesmo quando sua estratégia principal permanece inalterada.

# 6 Tarefa 3 – Algoritmo de Dijkstra utilizando Listas Encadeadas

## 6.1 Descrição do problema

A terceira tarefa consiste na implementação do algoritmo de Dijkstra para determinar os caminhos mínimos em um grafo direcionado e ponderado, utilizando listas encadeadas como estrutura de representação das adjacências. O problema é equivalente ao apresentado na Tarefa 2, diferenciando-se principalmente pela estrutura de dados utilizada para armazenar as arestas do grafo. Considerando um grafo $G = (V, E)$, em que $V$ representa o conjunto de vértices e $E$ o conjunto de arestas, cada aresta $(u, v) \in E$ possui um peso não negativo $w(u, v)$. Dado um vértice de origem $s \in V$, o objetivo é determinar a menor distância entre $s$ e os demais vértices alcançáveis, além de reconstruir os caminhos mínimos correspondentes.

Nesta implementação, cada vértice possui uma lista encadeada contendo suas arestas de saída. Cada nó da lista armazena o identificador do vértice de destino, o peso da aresta e um ponteiro para a próxima aresta. Diferentemente da representação por vetores indexados utilizada na Tarefa 2, as arestas são alocadas individualmente na memória e conectadas por ponteiros. O algoritmo deve utilizar busca sequencial para selecionar o próximo vértice de menor distância, sem o emprego de filas de prioridade, mantendo a mesma estratégia de processamento da implementação anterior. A entrada do programa utiliza o formato DIMACS, contendo a quantidade de vértices, a quantidade de arestas e as informações de origem, destino e peso de cada aresta. A saída apresenta as distâncias mínimas calculadas e os respectivos caminhos, representados pela sequência de vértices entre a origem e cada destino. Para vértices inalcançáveis, a distância é representada como infinita.

Além da implementação, a tarefa propõe a comparação entre diferentes formas de representação de grafos, considerando o tempo de execução, o espaço necessário para armazenamento e o custo de acesso às adjacências. Essa análise busca identificar as vantagens e limitações das listas encadeadas em relação aos vetores indexados e às matrizes de adjacência, especialmente para grafos esparsos e densos.

## 6.2 Estratégia de solução

Para resolver o problema, foi utilizada a mesma estratégia gulosa do algoritmo de Dijkstra apresentada na Tarefa 2, modificando apenas a representação das adjacências do grafo. A principal alteração consiste na utilização de listas encadeadas, nas quais cada vértice mantém um ponteiro para sua primeira aresta de saída. As demais arestas são acessadas sequencialmente por meio de ponteiros. Essa representação permite armazenar somente as arestas existentes, sem a necessidade de reservar espaço para todas as possíveis conexões entre os vértices.

### 6.2.1 Representação do grafo

O grafo é representado por um vetor de vértices, no qual cada elemento contém um ponteiro para o início de sua lista encadeada de adjacência. Cada nó da lista armazena o destino, o peso da aresta e um ponteiro para o próximo nó. A inserção de uma aresta é realizada no início da lista correspondente ao vértice de origem, permitindo que essa operação seja executada em tempo constante.

Diferentemente da Tarefa 2, na qual as conexões eram estabelecidas por índices de um vetor de arestas, a nova representação utiliza ponteiros para conectar os nós alocados dinamicamente.

### 6.2.2 Execução do algoritmo

O processamento mantém os mesmos vetores auxiliares de distâncias, predecessores e vértices visitados utilizados na implementação anterior. A seleção do próximo vértice continua sendo realizada por busca sequencial, sem utilização de fila de prioridade.

A principal diferença ocorre durante o relaxamento das arestas. Após selecionar um vértice, o algoritmo percorre sua lista encadeada de adjacência, seguindo os ponteiros até alcançar o final da lista. Para cada aresta encontrada, verifica-se se o caminho passando pelo vértice atual apresenta custo inferior à distância anteriormente registrada para o destino. Dessa forma, a lógica de Dijkstra permanece inalterada, permitindo comparar as duas implementações com foco nas diferenças entre as estruturas de armazenamento.

## 6.3 Implementação

A implementação foi desenvolvida em linguagem C, mantendo a organização modular utilizada na Tarefa 2. Os arquivos de cabeçalho e de implementação foram preservados, sendo modificadas principalmente as estruturas responsáveis pela representação do grafo e pelo percurso das adjacências.

### 6.3.1 Estruturas de dados

A principal modificação foi realizada nas estruturas `Vertice` e `Aresta`, que passaram a utilizar ponteiros para representar as listas encadeadas.

A estrutura `Aresta` armazena o destino, o peso e um ponteiro para a próxima aresta da lista.

```
typedef struct Aresta {
    int destino;
    int peso;
    struct Aresta *proxima;
} Aresta;
```

Código 17: Estrutura de uma aresta na lista encadeada.

A estrutura `Vertice` mantém o identificador do vértice e um ponteiro para sua primeira aresta de saída.

```
typedef struct {
    int id;
    Aresta *primeiraAresta;
} Vertice;
```

Código 18: Estrutura de um vértice com lista de adjacência.

A estrutura `Grafo` armazena a quantidade de vértices, a quantidade de arestas e um vetor de vértices.

```
typedef struct {
    int numVertices;
    int numArestas;
    Vertice *vertices;
} Grafo;
```

Código 19: Estrutura do grafo com listas encadeadas.

Diferentemente da Tarefa 2, não existe um vetor único para armazenar todas as arestas. Cada aresta é alocada individualmente e associada à lista do seu vértice de origem.

### 6.3.2 Inserção de arestas

A função `inserirAresta` foi adaptada para realizar a alocação dinâmica de cada nova aresta.

Após a alocação, o ponteiro `proxima` recebe o endereço da primeira aresta atual, e o ponteiro do vértice é atualizado para a nova aresta.

```
Aresta *novaAresta = malloc(sizeof(Aresta));

if (novaAresta == NULL)
    return 0;

novaAresta->destino = destino;
novaAresta->peso = peso;
novaAresta->proxima =
    grafo->vertices[origem].primeiraAresta;

grafo->vertices[origem].primeiraAresta = novaAresta;
grafo->numArestas++;
```

Código 20: Inserção de uma aresta no início da lista.

Esse procedimento possui custo constante, pois não é necessário percorrer a lista para inserir uma nova aresta. A função `destruirGrafo` também foi modificada para percorrer as listas de adjacência e liberar individualmente cada nó alocado.

### 6.3.3 Adaptação do algoritmo de Dijkstra

A estrutura `ResultadoDijkstra` e os procedimentos de inicialização, seleção de vértices e reconstrução dos caminhos foram mantidos.

A principal alteração na função `dijkstra` ocorreu no percurso das arestas de saída de cada vértice. Na implementação anterior, as arestas eram acessadas por índices de um vetor. Nesta versão, o percurso é realizado diretamente por ponteiros.

```
Aresta *aresta =
    grafo->vertices[u].primeiraAresta;

while (aresta != NULL) {
    int v = aresta->destino;
    int peso = aresta->peso;

    if (!resultado->visitado[v] &&
        resultado->distancia[u] <= INT64_MAX - peso &&
        resultado->distancia[u] + peso <
            resultado->distancia[v]) {

        resultado->distancia[v] =
            resultado->distancia[u] + peso;

        resultado->predecessor[v] = u;
    }

    aresta = aresta->proxima;
}
```

Código 21: Percurso e relaxamento das listas encadeadas.

O processamento continua até que o ponteiro `aresta` seja igual a `NULL`, indicando o final da lista de adjacência. Essa modificação mantém a lógica de relaxamento da Tarefa 2, alterando somente a forma de acesso às arestas.

### 6.3.4 Programa principal

O arquivo `main.c` manteve a mesma organização da Tarefa 2, realizando a leitura do grafo, a execução do algoritmo, a apresentação dos caminhos mínimos e a liberação da memória. A entrada continua utilizando o formato DIMACS, permitindo executar as mesmas instâncias de teste e comparar os resultados das duas implementações.

## 6.4 Análise de complexidade

A análise de complexidade considera o custo das operações realizadas sobre as listas encadeadas e o processamento do algoritmo de Dijkstra. Embora a representação do grafo tenha sido modificada em relação à Tarefa 2, a estratégia de seleção dos vértices e de relaxamento das arestas permanece a mesma.

### 6.4.1 Complexidade de tempo

A criação do grafo exige a inicialização dos $V$ vértices, apresentando custo $\Theta(V)$. Cada aresta é inserida no início de uma lista encadeada, com custo $\Theta(1)$ por inserção. Assim, a construção completa do grafo possui complexidade:

$$T_{\text{construcao}}(V, E) = \Theta(V + E). \tag{55}$$

Durante a execução de Dijkstra, a seleção do vértice não visitado de menor distância continua sendo realizada por busca sequencial, com custo $O(V)$ por iteração.

Como podem ocorrer até $V$ iterações, o custo total dessa etapa é $O(V^2)$.

O relaxamento das arestas é realizado percorrendo as listas encadeadas de adjacência. Como cada vértice é processado no máximo uma vez, cada aresta é examinada no máximo uma vez, resultando em custo $O(E)$.

Portanto, a complexidade temporal do algoritmo permanece:

$$\boxed{T(V, E) = O(V^2 + E)}. \tag{56}$$

Essa complexidade é equivalente à obtida na Tarefa 2, pois a alteração da representação das arestas não modifica o custo assintótico da seleção dos vértices nem do relaxamento das adjacências.

### 6.4.2 Complexidade de espaço

A representação por listas encadeadas utiliza um vetor com $V$ vértices e um nó alocado dinamicamente para cada uma das $E$ arestas. Cada nó armazena o destino, o peso e um ponteiro para a próxima aresta, ocupando uma quantidade constante de memória.

Dessa forma, o espaço necessário para armazenar o grafo é:

$$\boxed{S_{\text{grafo}}(V, E) = \Theta(V + E)}. \tag{57}$$

O algoritmo de Dijkstra utiliza três vetores auxiliares para armazenar distâncias, predecessores e informações dos vértices visitados, resultando em espaço auxiliar $\Theta(V)$.

Considerando a representação do grafo e as estruturas auxiliares, a complexidade espacial total é:

$$\boxed{S_{\text{total}}(V, E) = \Theta(V + E)}. \tag{58}$$

### 6.4.3 Comparação entre representações

As representações por vetores indexados e listas encadeadas apresentam a mesma complexidade assintótica de espaço, $\Theta(V+E)$, sendo adequadas para o armazenamento de grafos esparsos. Entretanto, existem diferenças em seus custos constantes. Na representação por vetores, as arestas são armazenadas em posições contíguas de memória, enquanto nas listas encadeadas cada aresta é alocada individualmente.

A utilização de ponteiros e alocações individuais pode aumentar o consumo de memória e prejudicar a localidade de acesso aos dados. Por outro lado, as listas encadeadas permitem inserir novas arestas sem a necessidade de reservar posições em um vetor de arestas. Uma matriz de adjacência, por sua vez, exige espaço $\Theta(V^2)$, independentemente da quantidade de arestas existentes.

Embora permita consultar diretamente a existência de uma aresta em tempo constante, essa representação pode apresentar elevado consumo de memória para grafos esparsos.

Assim, a escolha da representação depende da densidade do grafo, das operações realizadas e dos custos práticos de armazenamento e acesso às adjacências.

## 6.5 Experimentos e resultados

Para avaliar o desempenho da implementação por listas encadeadas, foram utilizadas as mesmas instâncias de grafos da Tarefa 2, permitindo comparar as duas representações sob condições semelhantes. Os experimentos foram realizados com os grafos de Roma e Nova York, disponíveis no formato DIMACS.

### 6.5.1 Metodologia dos testes

Inicialmente, a implementação foi validada utilizando um grafo pequeno com cinco vértices e seis arestas, verificando as distâncias mínimas e os caminhos produzidos pelo algoritmo.

Em seguida, foram realizados experimentos com duas instâncias de maior dimensão, apresentadas na Tabela 7.

Para cada instância, o algoritmo foi executado cinco vezes, registrando o tempo de processamento em segundos.

A medição foi realizada utilizando a função `clock()`, considerando somente a execução do algoritmo de Dijkstra, sem incluir a leitura do arquivo ou a impressão dos caminhos. Os tempos foram utilizados para calcular a média aritmética e o desvio-padrão amostral de cada implementação.

Tabela 7: Instâncias utilizadas nos experimentos.

| Instância | Vértices | Arestas |
|---|---|---|
| Roma | 3.353 | 8.859 |
| Nova York | 264.346 | 730.100 |

### 6.5.2 Resultados experimentais

A Tabela 8 apresenta os tempos registrados nas cinco execuções do grafo de Roma.

Tabela 8: Tempos de execução para o grafo de Roma (s).

| Execução | Tarefa 2 | Tarefa 3 |
|---|---|---|
| 1 | 0,008882 | 0,010191 |
| 2 | 0,008872 | 0,008271 |
| 3 | 0,008771 | 0,006622 |
| 4 | 0,008606 | 0,009292 |
| 5 | 0,006677 | 0,008857 |
| Média | 0,008362 | 0,008647 |
| Desvio-padrão | 0,000948 | 0,001331 |

Para o grafo de Nova York, os tempos obtidos são apresentados na Tabela 9.

Tabela 9: Tempos de execução para o grafo de Nova York (s).

| Execução | Tarefa 2 | Tarefa 3 |
|---|---|---|
| 1 | 44,318441 | 43,072907 |
| 2 | 42,966839 | 41,407636 |
| 3 | 46,213813 | 41,679474 |
| 4 | 43,582669 | 42,715275 |
| 5 | 43,038672 | 41,228662 |
| Média | 44,024087 | 42,020791 |
| Desvio-padrão | 1,338396 | 0,822979 |

### 6.5.3 Análise comparativa dos resultados

Os resultados mostram que as duas implementações apresentaram tempos de execução próximos, embora tenham sido observadas diferenças entre as instâncias avaliadas.

No grafo de Roma, a implementação por vetores indexados apresentou tempo médio de aproximadamente 0,008362 segundos, enquanto a implementação por listas encadeadas obteve 0,008647 segundos.

Isso corresponde a um tempo médio aproximadamente 3,41% maior para as listas encadeadas.

Entretanto, essa diferença é pequena em relação à variabilidade observada nas execuções, não permitindo afirmar uma superioridade consistente de uma representação para essa instância. No grafo de Nova York, a implementação por listas encadeadas apresentou tempo médio de 42,020791 segundos, enquanto a implementação por vetores indexados obteve 44,024087 segundos. Nesse caso, as listas encadeadas apresentaram uma redução aproximada de 4,55% no tempo médio de execução.

Apesar dessa vantagem observada, os resultados não demonstram que listas encadeadas sejam sempre mais eficientes, pois o desempenho também depende das características do grafo e do ambiente de execução. Como ambas as implementações utilizam busca sequencial para selecionar o próximo vértice, o custo quadrático dessa operação permanece predominante para instâncias com muitos vértices.

Assim, os experimentos indicam que a mudança na representação das adjacências pode influenciar o desempenho prático, mesmo quando a complexidade assintótica do algoritmo permanece inalterada.

## 6.6 Considerações da tarefa

A implementação do algoritmo de Dijkstra utilizando listas encadeadas permitiu analisar a influência da representação do grafo sobre o desempenho e a organização das estruturas de dados. Em relação à Tarefa 2, a principal modificação foi a substituição dos índices utilizados para conectar as arestas por ponteiros, permitindo a construção de listas de adjacência com nós alocados dinamicamente.

A análise teórica demonstrou que essa alteração não modifica a complexidade temporal do algoritmo, que permanece $O(V^2 + E)$, nem a complexidade espacial total, que permanece $\Theta(V + E)$. Nos experimentos realizados, a implementação por vetores indexados apresentou um tempo médio ligeiramente menor para o grafo de Roma, enquanto a representação por listas encadeadas obteve melhor desempenho médio para o grafo de Nova York. Esses resultados indicam que a representação das adjacências pode influenciar o desempenho prático, mesmo quando as implementações apresentam a mesma complexidade assintótica.

As duas representações são adequadas para grafos esparsos, pois armazenam apenas as arestas existentes. Entretanto, os vetores indexados apresentam a vantagem de utilizar armazenamento contíguo, enquanto as listas encadeadas oferecem maior flexibilidade para inserções individuais de arestas. Em comparação, as matrizes de adjacência permitem consultar diretamente a existência de uma aresta, mas exigem espaço $\Theta(V^2)$, tornando-se menos adequadas para grafos grandes e esparsos.

Portanto, para as instâncias avaliadas, as representações por vetores indexados e listas encadeadas constituem alternativas viáveis. A escolha entre elas deve considerar não apenas a complexidade assintótica, mas também a organização da memória, as operações realizadas e as características do grafo.

# 7 Tarefa 4 – Algoritmo de Dijkstra utilizando Fila de Prioridade

## 7.1 Descrição do problema

A quarta tarefa consiste em implementar uma nova versão do algoritmo de Dijkstra, utilizando uma fila de prioridade para otimizar a seleção do vértice de menor distância. O problema de caminhos mínimos permanece o mesmo das Tarefas 2 e 3, assim como o formato de entrada DIMACS e a apresentação das distâncias e dos caminhos encontrados.

A principal diferença está na substituição da busca sequencial por uma fila de prioridade, implementada por meio de um heap binário mínimo (*min-heap*). Além disso, devem ser contabilizadas as operações realizadas durante a execução, incluindo inserções, remoções, atualizações de distância e vértices processados.

O objetivo é avaliar o impacto dessa estrutura no desempenho do algoritmo, comparando os resultados com as implementações anteriores, especialmente em grafos de grandes dimensões.

## 7.2 Estratégia de solução

A solução mantém a estratégia gulosa do algoritmo de Dijkstra e a representação do grafo por listas encadeadas utilizada na Tarefa 3.

A principal alteração ocorre na seleção do próximo vértice a ser processado. Em vez de percorrer sequencialmente o vetor de distâncias, utiliza-se um min-heap, no qual a prioridade de cada vértice corresponde à menor distância conhecida a partir da origem. Inicialmente, apenas o vértice de origem é inserido na fila, com prioridade zero. A cada iteração, o vértice de menor prioridade é removido e suas arestas de saída são examinadas.

Quando um caminho de menor custo é encontrado, a distância do vértice de destino é atualizada. Caso ele ainda não esteja na fila, é inserido; caso contrário, sua prioridade é reduzida. Dessa forma, a fila permite selecionar e atualizar os vértices de maneira mais eficiente, eliminando a necessidade de realizar uma busca sequencial a cada iteração.

## 7.3 Implementação

A implementação foi desenvolvida em C, aproveitando as estruturas de grafo e as funções de leitura, impressão e liberação de memória da Tarefa 3. Foram adicionados os arquivos `fila_prioridade.h` e `fila_prioridade.c`, responsáveis pela definição e implementação do TAD `FilaPrioridade`.

A estrutura utiliza um vetor de elementos do heap, um vetor auxiliar de posições e campos para controlar seu tamanho e capacidade. Cada elemento armazena o identificador de um vértice e sua prioridade.

```
typedef struct {
    int vertice;
    int64_t prioridade;
} ElementoHeap;

typedef struct {
    ElementoHeap *elementos;
    int *posicao;
    int tamanho;
    int capacidade;
} FilaPrioridade;
```

Código 22: Estruturas utilizadas na fila de prioridade.

O vetor `posicao` permite localizar diretamente um vértice no heap, possibilitando a atualização de sua prioridade sem percorrer sequencialmente a estrutura.

As principais operações implementadas são `inserir`, `extrairMinimo`, `diminuirPrioridade` e `filaVazia`. As operações de inserção, remoção e atualização utilizam procedimentos auxiliares para preservar a propriedade de heap mínimo.

Na função `dijkstra`, a busca sequencial foi substituída pela operação `extrairMinimo`. Durante o relaxamento das arestas, são utilizadas as operações `inserir` e `diminuirPrioridade`, conforme a presença do vértice na fila.

Também foram adicionados à estrutura `ResultadoDijkstra` quatro contadores para registrar o número de inserções, remoções, atualizações de distância e vértices processados. Essas informações são apresentadas pelo programa principal, juntamente com o tempo de execução, para permitir a análise experimental.

## 7.4 Análise de complexidade

A análise considera um grafo direcionado $G = (V, E)$, em que $V$ representa o número de vértices e $E$ o número de arestas. A principal diferença em relação às implementações anteriores está na utilização de uma fila de prioridade baseada em heap binário mínimo.

### 7.4.1 Complexidade das operações do heap

O heap binário é organizado como uma árvore binária completa, armazenada em um vetor. Sua altura é proporcional a $\log V$.

As operações de inserção e remoção do elemento de menor prioridade podem exigir a reorganização dos elementos ao longo de um caminho da árvore, apresentando custo $O(\log V)$. A operação `diminuirPrioridade` também apresenta custo $O(\log V)$, pois utiliza o vetor auxiliar `posicao` para localizar o vértice em tempo constante e, quando necessário, reposicioná-lo no heap. A verificação de fila vazia é realizada em tempo constante, $O(1)$.

### 7.4.2 Complexidade de tempo

Inicialmente, a criação dos vetores auxiliares e da fila de prioridade apresenta custo $O(V)$. Durante a execução de Dijkstra, cada vértice alcançável é inserido e removido da fila no máximo uma vez. Como cada operação possui custo $O(\log V)$, o custo total dessas operações é:

$$O(V \log V). \tag{59}$$

Além disso, cada aresta é examinada no máximo uma vez durante o processamento de seu vértice de origem. Quando ocorre um relaxamento bem-sucedido, pode ser necessário atualizar a prioridade de um vértice no heap, operação com custo $O(\log V)$. Como existem $E$ arestas, o custo total associado ao exame das arestas e às atualizações de prioridade é:

$$O(E \log V). \tag{60}$$

Somando os custos, obtém-se:

$$T(V, E) = O(V + V \log V + E \log V). \tag{61}$$

Portanto, para $V \geq 2$, a complexidade de tempo da implementação é:

$$\boxed{T(V, E) = O((V + E) \log V)}. \tag{62}$$

Essa complexidade representa uma melhoria em relação às implementações anteriores, nas quais a seleção sequencial do próximo vértice resulta em custo $O(V^2 + E)$.

### 7.4.3 Complexidade de espaço

A representação do grafo por listas de adjacência encadeadas requer espaço $O(V + E)$ para armazenar os vértices e as arestas. Os vetores de distâncias, predecessores e vértices visitados utilizam espaço $O(V)$. A fila de prioridade armazena no máximo $V$ elementos, enquanto o vetor auxiliar de posições também possui tamanho proporcional a $V$.

Assim, o espaço auxiliar utilizado pelo algoritmo de Dijkstra é:

$$\boxed{S_{\text{aux}}(V) = O(V)}. \tag{63}$$

Considerando também o armazenamento do grafo, a complexidade total de espaço é:

$$\boxed{S(V, E) = O(V + E)}. \tag{64}$$

### 7.4.4 Comparação com as implementações anteriores

A Tabela 10 apresenta as complexidades das três implementações do algoritmo de Dijkstra desenvolvidas no trabalho.

Tabela 10: Comparação das implementações de Dijkstra.

| Tarefa | Seleção do mínimo | Tempo total |
|---|---|---|
| 2 – Vetores | $O(V)$ | $O(V^2 + E)$ |
| 3 – Listas | $O(V)$ | $O(V^2 + E)$ |
| 4 – Min-heap | $O(\log V)$ | $O((V + E) \log V)$ |

Nas Tarefas 2 e 3, a busca sequencial pelo vértice de menor distância é realizada repetidamente, produzindo um custo quadrático em relação ao número de vértices. Na Tarefa 4, essa busca é substituída pela remoção do mínimo do heap, reduzindo o custo de seleção para $O(\log V)$. Essa alteração é especialmente vantajosa em grafos esparsos, nos quais o número de arestas é proporcional ao número de vértices, isto é, $E = O(V)$.

Nesse caso, a complexidade da Tarefa 4 é limitada por:

$$T(V, E) = O(V \log V), \tag{65}$$

enquanto as implementações anteriores mantêm o custo quadrático $O(V^2)$.

## 7.5 Experimentos e resultados

Os experimentos da Tarefa 4 tiveram como objetivo verificar o funcionamento do algoritmo de Dijkstra com fila de prioridade e avaliar seu desempenho em comparação com as Tarefas 2 e 3.

Foram utilizados grafos de teste pequenos e duas instâncias de redes viárias no formato DIMACS: Roma, com 3.353 vértices e 8.859 arestas, e Nova York, com 264.346 vértices e 730.100 arestas.

### 7.5.1 Testes de funcionamento

Inicialmente, foram executados testes com um grafo pequeno, um grafo desconexo e um grafo contendo arestas de peso zero. Os resultados obtidos foram compatíveis com as distâncias esperadas. No grafo pequeno, a saída da Tarefa 4 foi comparada com a da Tarefa 3 utilizando o comando `diff`, sem apresentar diferenças. A comparação também foi realizada no grafo de Roma, produzindo saídas idênticas para as duas implementações.

No grafo de Nova York, as saídas apresentaram diferenças na comparação textual. Como podem existir diferentes caminhos mínimos com o mesmo custo, ainda é necessário verificar se as distâncias calculadas pelas duas implementações são iguais.

### 7.5.2 Metodologia dos experimentos

Para avaliar o desempenho, cada grafo foi processado cinco vezes, utilizando o vértice 1 como origem. O tempo foi medido com a função `clock()`, considerando apenas a execução do algoritmo de Dijkstra, sem incluir a leitura do grafo ou a impressão dos caminhos. Os programas foram compilados em C com a opção de otimização `-O2`. Os testes foram executados em ambiente Linux por meio do WSL.

Para cada conjunto de execuções, foram calculados a média aritmética e o desvio-padrão amostral dos tempos.

Tabela 11: Tempos de execução da Tarefa 4.

| Execução | Roma (s) | Nova York (s) |
|---|---|---|
| 1 | 0,000324 | 0,049182 |
| 2 | 0,000500 | 0,056541 |
| 3 | 0,000319 | 0,050694 |
| 4 | 0,000720 | 0,052872 |
| 5 | 0,000330 | 0,053205 |
| Média | 0,0004386 | 0,0524988 |
| Desvio-padrão | 0,0001748 | 0,0027945 |

### 7.5.3 Tempos de execução

A Tabela 11 apresenta os tempos obtidos nas cinco execuções de cada grafo.

O grafo de Roma apresentou tempo médio de aproximadamente 0,439 milissegundo, enquanto Nova York apresentou tempo médio de aproximadamente 52,499 milissegundos. Embora Nova York possua uma quantidade significativamente maior de vértices e arestas, o algoritmo conseguiu processar essa instância em um tempo inferior a um décimo de segundo.

### 7.5.4 Operações da fila de prioridade

Além do tempo de execução, foram contabilizadas as operações realizadas durante o processamento.

A Tabela 12 apresenta os resultados observados, que permaneceram constantes nas cinco execuções de cada grafo.

Tabela 12: Operações registradas na Tarefa 4.

| Operação | Roma | Nova York |
|---|---|---|
| Inserções | 3.353 | 264.346 |
| Remoções | 3.353 | 264.346 |
| Atualizações de distância | 3.747 | 299.020 |
| Vértices processados | 3.353 | 264.346 |

Observa-se que cada vértice foi inserido e removido da fila uma única vez. O número de atualizações de distância foi superior ao de inserções porque alguns vértices tiveram suas prioridades reduzidas após a descoberta de caminhos de menor custo.

Esses resultados são compatíveis com a utilização de um heap indexado, que permite atualizar prioridades sem inserir cópias adicionais de um mesmo vértice.

### 7.5.5 Comparação de desempenho

A Tabela 13 compara os tempos médios das três implementações desenvolvidas.

Tabela 13: Comparação dos tempos médios de Dijkstra.

| Tarefa | Roma (s) | Nova York (s) |
|---|---|---|
| 2 – Vetores | 0,0083616 | 44,0240868 |
| 3 – Listas | 0,0086466 | 42,0207908 |
| 4 – Min-heap | 0,0004386 | 0,0524988 |

Para quantificar a melhoria, foi calculado o fator de aceleração (*speedup*), definido pela razão entre o tempo médio da implementação anterior e o tempo médio da Tarefa 4:

$$S = \frac{T_{\text{anterior}}}{T_{\text{heap}}}. \tag{66}$$

A Tabela 14 apresenta os fatores de aceleração obtidos.

Em Roma, a implementação com heap apresentou um ganho de aproximadamente 19 vezes em relação às versões anteriores.

Tabela 14: Fatores de aceleração da Tarefa 4.

| Comparação | Roma | Nova York |
|---|---|---|
| Tarefa 2 / Tarefa 4 | 19,06 | 838,57 |
| Tarefa 3 / Tarefa 4 | 19,71 | 800,41 |

Em Nova York, o ganho foi ainda mais expressivo, alcançando aproximadamente 800 vezes em relação à Tarefa 3. Essa diferença está relacionada à complexidade das operações de seleção do próximo vértice. Nas implementações anteriores, essa seleção exige uma busca sequencial com custo $O(V)$. Na Tarefa 4, a utilização do heap reduz o custo da operação para $O(\log V)$, contribuindo para a melhoria do desempenho em grafos com muitos vértices.

### 7.5.6 Análise dos resultados

Os resultados experimentais mostram que a utilização de uma fila de prioridade baseada em heap binário reduziu significativamente o tempo de execução do algoritmo de Dijkstra.

O ganho foi especialmente expressivo no grafo de Nova York, caracterizado por uma grande quantidade de vértices e um número relativamente pequeno de arestas por vértice. Esse comportamento é coerente com a análise teórica, segundo a qual a implementação com heap apresenta complexidade $O((V + E) \log V)$, enquanto as implementações anteriores apresentam $O(V^2 + E)$.

Nos testes de Roma, os tempos muito pequenos produziram variações relativas mais elevadas entre as execuções. Portanto, os resultados de Nova York oferecem uma comparação experimental mais representativa do ganho obtido. De maneira geral, os experimentos demonstram que a escolha da estrutura de dados utilizada para selecionar os vértices influencia diretamente a eficiência do algoritmo, mesmo quando sua estratégia fundamental permanece inalterada.

## 7.6 Considerações da tarefa

A quarta tarefa permitiu avaliar o impacto da utilização de uma fila de prioridade na eficiência do algoritmo de Dijkstra. Mantendo a representação do grafo por listas encadeadas da Tarefa 3, a principal alteração foi a substituição da busca sequencial pelo uso de um heap binário mínimo para selecionar o próximo vértice.

A implementação utilizou um heap indexado, permitindo inserir, remover e atualizar prioridades em tempo $O(\log V)$. Com essa estrutura, a complexidade de tempo do algoritmo passou de $O(V^2+E)$, nas implementações anteriores, para $O((V+E) \log V)$, mantendo complexidade de espaço total $O(V + E)$. Os experimentos demonstraram uma melhoria expressiva no desempenho. No grafo de Roma, a implementação com heap foi aproximadamente 19,7 vezes mais rápida que a Tarefa 3. No grafo de Nova York, o fator de aceleração foi de aproximadamente 800,4 vezes, evidenciando a vantagem da estrutura em grafos grandes e esparsos. Os testes funcionais apresentaram os resultados esperados para os casos pequenos, e a comparação com a Tarefa 3 produziu saídas idênticas no grafo de Roma. Para Nova York, ainda é necessário verificar se as diferenças nos caminhos impressos correspondem apenas a alternativas de mesmo custo mínimo. Como limitação, a implementação com heap exige operações adicionais de manutenção da estrutura e um vetor auxiliar para controlar as posições dos vértices. Entretanto, os resultados demonstraram que esse custo é compensado pela redução do tempo necessário para selecionar os vértices durante a execução.

Conclui-se que a escolha de estruturas de dados adequadas pode melhorar significativamente o desempenho de um algoritmo, mesmo sem modificar sua estratégia fundamental de solução.

# 8 Tarefa 5 – Caminho Mínimo por Busca Completa

## 8.1 Descrição do problema

A quinta tarefa consiste na implementação de um algoritmo para determinar o caminho de menor custo entre dois vértices de um grafo direcionado e ponderado, utilizando a estratégia de busca completa. Diferentemente das tarefas anteriores, nas quais foi utilizado o algoritmo de Dijkstra, a solução deve explorar os caminhos simples possíveis entre os vértices de origem e destino, calculando seus custos e identificando aquele que apresenta o menor valor.

Considere um grafo $G = (V, E)$, em que $V$ representa o conjunto de vértices e $E$ o conjunto de arestas. Cada aresta $(u, v) \in E$ possui um peso não negativo $w(u, v)$, que representa o custo de deslocamento entre os vértices. Dados dois vértices $s, t \in V$, correspondentes à origem e ao destino, respectivamente, o objetivo é encontrar um caminho $P = (v_0, v_1, \ldots, v_k)$, com $v_0 = s$ e $v_k = t$, que minimize a soma dos pesos das arestas percorridas.

O custo de um caminho é definido por:

$$w(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1}). \tag{67}$$

Dessa forma, o problema consiste em determinar:

$$\delta(s, t) = \min_{P \in \mathcal{P}_s(s,t)} w(P), \tag{68}$$

em que $\mathcal{P}_s(s, t)$ representa o conjunto de caminhos simples entre a origem $s$ e o destino $t$.

Um caminho é considerado simples quando não apresenta repetição de vértices. Essa restrição impede a formação de ciclos durante a exploração e limita a quantidade de vértices presentes em cada caminho. A entrada do programa utiliza o formato DIMACS, empregado nas tarefas anteriores, contendo a quantidade de vértices, a quantidade de arestas e os respectivos pesos. Os vértices de origem e destino são informados por argumentos de execução, juntamente com um limite de tempo opcional.

A saída apresenta o menor custo encontrado, a sequência de vértices correspondente, a quantidade de caminhos explorados e o número de chamadas recursivas realizadas. Também são informados o tempo de execução e o estado de conclusão da busca, permitindo identificar situações em que a exploração foi interrompida antes de examinar todos os caminhos possíveis.

Além de verificar a correção da solução, a tarefa tem como objetivo analisar o crescimento do custo computacional da busca completa em grafos de diferentes dimensões e comparar seu desempenho com o algoritmo de Dijkstra implementado na Tarefa 4.

## 8.2 Estratégia de solução

Para resolver o problema do caminho mínimo, foi utilizada a estratégia de busca completa, implementada por meio de recursão e da técnica de *backtracking*.

A abordagem consiste em explorar os caminhos simples existentes entre os vértices de origem e destino, calculando o custo acumulado de cada percurso e armazenando a melhor solução encontrada. Diferentemente do algoritmo de Dijkstra, que utiliza uma estratégia gulosa para determinar as menores distâncias, a busca completa examina diferentes possibilidades de caminhos, sem utilizar estimativas de distância para selecionar o próximo vértice. A exploração é realizada em profundidade, percorrendo recursivamente as arestas de saída de cada vértice. Ao alcançar o destino, o caminho construído é avaliado e a execução retorna para explorar outras possibilidades.

### 8.2.1 Exploração recursiva dos caminhos

A busca tem início no vértice de origem, com custo acumulado igual a zero e um caminho contendo apenas o vértice inicial. Em cada chamada recursiva, o algoritmo examina as arestas de saída do vértice atual, verificando quais vértices adjacentes podem ser adicionados ao caminho.

Para cada vértice adjacente ainda não visitado, é realizada uma nova chamada recursiva, acrescentando o peso da aresta ao custo acumulado.

Considerando um caminho parcial $P = (v_0, v_1, \ldots, v_k)$, cujo custo acumulado é $c(P)$, a extensão desse caminho por uma aresta $(v_k, u)$ produz o novo custo:

$$c(P') = c(P) + w(v_k, u). \tag{69}$$

Esse procedimento é repetido até que o vértice de destino seja alcançado ou não existam novos vértices que possam ser explorados.

Quando o destino é alcançado, o algoritmo contabiliza um caminho completo e verifica se seu custo é inferior ao melhor valor registrado.

Após a avaliação, a execução retorna à chamada anterior, permitindo que outros caminhos sejam examinados.

### 8.2.2 Controle dos vértices visitados

Para garantir que os caminhos explorados sejam simples, foi utilizado um vetor de controle que identifica os vértices presentes no caminho atual.

Inicialmente, todos os vértices são considerados não visitados. Ao adicionar um vértice ao caminho, sua posição no vetor é marcada, impedindo que ele seja utilizado novamente durante a mesma sequência de chamadas recursivas.

Quando a exploração de um determinado vértice é encerrada, sua marcação é removida antes do retorno à chamada anterior. Esse procedimento caracteriza a técnica de *backtracking*, pois permite desfazer as decisões tomadas durante a construção de um caminho e explorar outras alternativas. Dessa forma, um mesmo vértice pode participar de diferentes caminhos, desde que não seja repetido dentro de um único percurso. A utilização desse controle também impede que a busca entre em ciclos, limitando a profundidade da recursão a, no máximo, $|V|$ vértices.

### 8.2.3 Atualização do melhor caminho

Durante a execução, o algoritmo mantém o menor custo encontrado e a sequência de vértices correspondente. Inicialmente, o melhor custo é definido como infinito, indicando que nenhum caminho entre a origem e o destino foi encontrado. Sempre que a busca alcança o destino, o custo acumulado do caminho atual é comparado com o melhor custo registrado anteriormente.

A atualização ocorre quando:

$$c(P_{\text{atual}}) < c(P_{\text{melhor}}). \tag{70}$$

Quando essa condição é satisfeita, o custo do novo caminho é armazenado, juntamente com sua sequência de vértices. Caso o custo seja maior ou igual ao melhor valor conhecido, a solução anterior é preservada. Ao concluir a exploração de todos os caminhos simples possíveis, o menor custo registrado corresponde à solução ótima do problema. Como os pesos das arestas são não negativos, existe um caminho mínimo simples sempre que o destino é alcançável a partir da origem.

### 8.2.4 Controle do tempo de execução

Como a quantidade de caminhos simples pode crescer rapidamente com o número de vértices, foi implementado um mecanismo de controle do tempo de execução.

O algoritmo recebe um limite de tempo configurável, utilizado para interromper a exploração quando o processamento ultrapassa o intervalo estabelecido. Durante a busca recursiva, o tempo decorrido é verificado para determinar se a execução pode continuar. Quando o limite é atingido, a exploração é interrompida e o resultado é identificado como uma busca não concluída.

Nessa situação, o programa mantém o melhor caminho encontrado até o momento da interrupção, caso algum caminho tenha sido identificado.

Entretanto, como nem todas as possibilidades foram examinadas, não é possível garantir que esse caminho corresponda à solução ótima.

Quando a busca termina sem atingir o limite de tempo, todos os caminhos simples relevantes foram examinados, permitindo garantir a optimalidade da solução encontrada.

O controle de tempo possibilita avaliar experimentalmente as limitações da busca completa, especialmente em grafos com maior número de vértices e arestas.

## 8.3 Implementação

A implementação da busca completa foi desenvolvida na linguagem C, utilizando a representação de grafos empregada na Tarefa 4.

O código foi organizado em arquivos de cabeçalho e implementação, separando as operações de manipulação do grafo, a execução da busca recursiva e o programa principal.

Os arquivos `grafo.h` e `grafo.c` são responsáveis pela representação do grafo, leitura dos dados no formato DIMACS e liberação da memória alocada.

Os arquivos `busca_completa.h` e `busca_completa.c` contêm as estruturas e funções relacionadas à exploração dos caminhos. Por fim, o arquivo `main.c` realiza a leitura dos argumentos, executa o algoritmo e apresenta os resultados obtidos.

### 8.3.1 Representação do grafo

A representação utilizada mantém os vértices e as arestas em estruturas alocadas dinamicamente, permitindo percorrer as arestas de saída de cada vértice.

Cada vértice armazena a referência à sua primeira aresta, enquanto cada aresta contém o identificador do destino, seu peso e a referência à próxima aresta de saída.

Essa organização permite explorar os vértices adjacentes sem percorrer todas as arestas do grafo a cada chamada recursiva.

A leitura dos dados utiliza o formato DIMACS, mantendo a compatibilidade com os grafos processados nas tarefas anteriores.

### 8.3.2 Estrutura de resultados

Para armazenar as informações produzidas pela busca completa, foi definida a estrutura `ResultadoBuscaCompleta`.

Essa estrutura reúne os identificadores dos vértices de origem e destino, o menor custo encontrado, a sequência de vértices do melhor caminho e as métricas utilizadas na avaliação experimental.

Sua definição é apresentada no Código 23.

```
typedef struct {
    int origem;
    int destino;
    int numVertices;
    int64_t melhorCusto;
    int *melhorCaminho;
    int tamanhoMelhorCaminho;
    uint64_t numCaminhosExplorados;
    uint64_t numChamadasRecursivas;
    int buscaConcluida;
} ResultadoBuscaCompleta;
```

Código 23: Estrutura de resultados da busca completa.

O campo `melhorCusto` armazena o menor custo identificado durante a exploração, utilizando um inteiro de 64 bits.

O vetor `melhorCaminho` contém a sequência de vértices correspondente, enquanto `tamanhoMelhorCaminho` indica a quantidade de vértices presentes nessa sequência.

Os campos `numCaminhosExplorados` e `numChamadasRecursivas` registram, respectivamente, a quantidade de caminhos que alcançaram o destino e o número de chamadas realizadas pela função recursiva.

Por fim, `buscaConcluida` indica se a exploração terminou normalmente ou foi interrompida pelo limite de tempo.

### 8.3.3 Função recursiva de busca

A exploração dos caminhos é realizada por uma função recursiva que recebe as informações necessárias para continuar a busca a partir de determinado vértice.

Durante a execução, são mantidos um vetor de vértices visitados, um vetor contendo o caminho atual e uma variável com o custo acumulado.

A cada chamada, o algoritmo verifica se o vértice atual corresponde ao destino.

Quando essa condição é satisfeita, um caminho completo é contabilizado e seu custo é comparado com o melhor valor conhecido. Caso o novo caminho apresente menor custo, sua sequência de vértices é copiada para o vetor que armazena a melhor solução.

Quando o destino ainda não foi alcançado, a função percorre as arestas de saída do vértice atual e realiza novas chamadas recursivas para os vértices adjacentes que não estejam marcados como visitados.

### 8.3.4 Controle de visitas e backtracking

O controle de visitas é fundamental para impedir a formação de ciclos durante a exploração. Antes de realizar uma chamada recursiva para um vértice adjacente, o algoritmo verifica sua marcação no vetor `visitado`. Caso o vértice ainda não pertença ao caminho atual, ele é marcado e adicionado à sequência de vértices. Após o retorno da chamada recursiva, a marcação é removida, permitindo que o mesmo vértice participe de outros caminhos. O procedimento central pode ser representado pelo Código 24.

```
1 if (!visitado[proximo]) {
2     visitado[proximo] = 1;
3     caminhoAtual[profundidade] = proximo;
4
5     buscaRecursiva(...);
6
7     visitado[proximo] = 0;
8 }
```

Código 24: Controle de visitas durante a busca recursiva.

O código apresentado é uma representação simplificada da lógica de exploração, omitindo os parâmetros da chamada recursiva. A remoção da marcação após o retorno permite desfazer a escolha realizada e continuar a exploração por outros caminhos.

Esse procedimento garante que cada percurso construído seja um caminho simples.

### 8.3.5 Atualização da melhor solução

Sempre que o destino é alcançado, o custo do caminho atual é comparado com o menor custo registrado. Inicialmente, o melhor custo é representado por `INT64_MAX`, indicando que ainda não foi encontrada uma solução.

Quando um caminho apresenta custo inferior ao melhor valor, o algoritmo atualiza `melhorCusto` e armazena a sequência correspondente em `melhorCaminho`. A cópia da sequência é necessária porque o vetor do caminho atual é reutilizado durante a exploração das demais possibilidades.

Dessa forma, o melhor caminho encontrado permanece armazenado mesmo após o retorno das chamadas recursivas.

### 8.3.6 Controle do tempo de execução

Para evitar execuções excessivamente longas, a implementação permite definir um limite máximo de tempo.

A medição é realizada utilizando a função `clock()`, disponibilizada pela biblioteca `time.h`. O tempo de CPU decorrido é calculado pela diferença entre o valor atual do relógio e o instante inicial da execução.

Essa diferença é convertida para segundos por meio da constante `CLOCKS_PER_SEC`. Quando o tempo decorrido atinge o limite estabelecido, a busca é interrompida e o campo `buscaConcluida` recebe o valor zero. Caso a exploração termine sem atingir esse limite, o campo indica que a busca foi concluída. Essa funcionalidade permite executar experimentos com grafos de diferentes dimensões, evitando que instâncias computacionalmente inviáveis permaneçam em execução por tempo indeterminado.

### 8.3.7 Função principal e apresentação dos resultados

O arquivo `main.c` coordena a execução do programa.

Inicialmente, são interpretados os argumentos informados pelo usuário, incluindo os vértices de origem e destino e o limite de tempo.

Em seguida, o grafo é carregado a partir da entrada padrão, utilizando a função de leitura do formato DIMACS. Após a construção do grafo, a função `buscaCompleta` é executada para realizar a exploração dos caminhos. Ao término do processamento, são apresentados o menor custo, a sequência de vértices do melhor caminho, a quantidade de caminhos explorados, o número de chamadas recursivas, o tempo de execução e o estado de conclusão da busca. Por fim, a memória utilizada pelos resultados e pela representação do grafo é liberada. A separação entre leitura, processamento e apresentação dos resultados contribui para a organização do código e permite reutilizar a implementação em diferentes experimentos.

## 8.4 Análise de complexidade

A análise de complexidade da busca completa considera a quantidade de caminhos simples que podem existir entre dois vértices de um grafo direcionado, bem como o número de operações necessárias para explorar esses caminhos. Diferentemente do algoritmo de Dijkstra, que utiliza uma estratégia polinomial para determinar os caminhos mínimos, a busca completa pode precisar examinar uma quantidade fatorial de possibilidades. Para a análise, considera-se um grafo $G = (V, E)$, em que $V$ representa o número de vértices e $E$ o número de arestas.

### 8.4.1 Quantidade de caminhos simples

O custo da busca completa depende diretamente da quantidade de caminhos simples existentes entre os vértices de origem e destino. Para analisar o pior caso, considera-se um grafo direcionado completo, no qual existe uma aresta entre cada par de vértices distintos. Fixando os vértices de origem $s$ e destino $t$, restam $V - 2$ vértices que podem ser utilizados como intermediários.

Um caminho simples pode conter nenhum, um ou até $V - 2$ vértices intermediários, sem repetição.

Para um caminho contendo exatamente $k$ vértices intermediários, a quantidade de sequências possíveis corresponde ao número de arranjos de $V - 2$ elementos tomados $k$ a $k$:

$$A(V-2,k) = \frac{(V-2)!}{(V-2-k)!}. \tag{71}$$

Somando todas as possibilidades, obtém-se a quantidade total de caminhos simples entre $s$ e $t$:

$$P(V) = \sum_{k=0}^{V-2} \frac{(V-2)!}{(V-2-k)!}. \tag{72}$$

Colocando $(V-2)!$ em evidência:

$$P(V) = (V-2)! \sum_{j=0}^{V-2} \frac{1}{j!}. \tag{73}$$

Como a soma $\sum_{j=0}^{V-2} 1/j!$ é limitada inferiormente por uma constante positiva e superiormente pelo número de Euler $e$, conclui-se que:

$$\boxed{P(V) = \Theta((V-2)!)}. \tag{74}$$

Portanto, no pior caso, a quantidade de caminhos simples apresenta crescimento fatorial em relação ao número de vértices.

### 8.4.2 Complexidade de tempo

A função recursiva explora os vértices adjacentes que ainda não pertencem ao caminho atual. Em cada chamada, são realizadas operações de controle, como verificação do limite de tempo, atualização das métricas e consulta ao vetor de visitados. Além disso, a função percorre as arestas de saída do vértice atual para identificar possíveis continuações do caminho.

No pior caso, um vértice pode possuir até $V-1$ arestas de saída, resultando em um custo $O(V)$ para o processamento de uma chamada recursiva. Como a exploração ocorre sobre diferentes caminhos parciais, a quantidade de chamadas também pode apresentar crescimento fatorial.

Assim, uma cota superior conservadora para o tempo de execução é:

$$T(V) = O(V \cdot V!). \tag{75}$$

Essa cota considera o custo de percorrer as adjacências durante a enumeração das possibilidades.

Para grafos direcionados completos, a quantidade de chamadas recursivas e de caminhos explorados cresce fatorialmente. Portanto, o algoritmo apresenta comportamento combinatório no pior caso.

O limite de tempo utilizado na implementação não modifica essa complexidade teórica, pois apenas interrompe a execução antes que todas as possibilidades sejam examinadas.

### 8.4.3 Complexidade de espaço

A implementação utiliza vetores auxiliares para controlar os vértices visitados, armazenar o caminho atual e registrar o melhor caminho encontrado.

Cada um desses vetores possui tamanho proporcional ao número de vértices do grafo, resultando em consumo de memória $O(V)$. Além disso, a busca é realizada recursivamente, utilizando a pilha de chamadas do programa.

Como um vértice não pode ser repetido dentro do mesmo caminho, a profundidade máxima da recursão é limitada a $V$.

Consequentemente, a memória utilizada pela pilha recursiva também é $O(V)$.

Somando os vetores auxiliares e a pilha de chamadas, obtém-se:

$$\boxed{S_{\text{aux}}(V) = O(V)}. \tag{76}$$

Considerando também o armazenamento do grafo por listas de adjacência, cujo consumo é $O(V+E)$, o espaço total da implementação é:

$$\boxed{S_{\text{total}}(V, E) = O(V + E)}. \tag{77}$$

Dessa forma, embora o tempo de execução possa apresentar crescimento fatorial, a memória auxiliar utilizada pela busca permanece linear em relação ao número de vértices.

### 8.4.4 Comparação teórica com Dijkstra

Para avaliar a eficiência da busca completa, considera-se a implementação de Dijkstra desenvolvida na Tarefa 4, que utiliza uma fila de prioridade baseada em heap binário.

Nessa implementação, as operações de inserção, remoção e atualização da fila possuem custo logarítmico em relação à quantidade de vértices.

Considerando o processamento dos vértices e o relaxamento das arestas, a complexidade temporal de Dijkstra é:

$$T_{\text{Dijkstra}}(V, E) = O((V + E) \log V). \tag{78}$$

Em comparação, a busca completa apresenta uma cota superior de tempo dada por:

$$T_{\text{Busca}}(V) = O(V \cdot V!). \tag{79}$$

Enquanto Dijkstra possui complexidade polinomial, a busca completa pode necessitar de um número fatorial de operações. Essa diferença se torna especialmente significativa em grafos com muitos vértices e elevada densidade de arestas, nos quais existe uma grande quantidade de caminhos simples possíveis. Embora ambas as estratégias possam encontrar o caminho de menor custo, Dijkstra utiliza propriedades dos pesos não negativos para evitar a enumeração de todas as soluções. Por outro lado, a busca completa examina sistematicamente os caminhos possíveis, permitindo observar experimentalmente o crescimento combinatório do problema.

## 8.5 Experimentos e resultados

Para avaliar o funcionamento e o desempenho da busca completa, foram realizados experimentos com grafos de diferentes dimensões.

Os testes tiveram como objetivo verificar o crescimento do tempo de execução, a quantidade de caminhos explorados e o número de chamadas recursivas realizadas. Também foi realizada uma comparação com o algoritmo de Dijkstra implementado na Tarefa 4, utilizando os custos dos caminhos encontrados para verificar a correção dos resultados.

### 8.5.1 Metodologia dos experimentos

Os experimentos foram automatizados por meio de scripts desenvolvidos em Python, responsáveis pela execução dos programas, coleta das métricas e organização dos resultados em arquivos CSV.

Na avaliação de desempenho, foram utilizadas oito instâncias com quantidades de vértices variando de 5 a 40. Cada instância foi executada cinco vezes, totalizando 40 execuções. Para cada execução, foram registrados o tempo de processamento, a quantidade de caminhos completos explorados, o número de chamadas recursivas, o menor custo encontrado e o estado de conclusão da busca.

Os tempos médios foram calculados pela média aritmética das cinco execuções de cada instância:

$$\overline{T} = \frac{1}{r} \sum_{i=1}^{r} T_i, \tag{80}$$

em que $r = 5$ representa o número de repetições e $T_i$ corresponde ao tempo registrado na execução $i$. Além da avaliação de desempenho, foi realizado um conjunto separado de testes comparativos com Dijkstra, considerando instâncias de 5, 10, 15, 20 e 25 vértices.

### 8.5.2 Resultados de desempenho

A Tabela 15 apresenta os resultados obtidos na avaliação de desempenho da busca completa.

Os valores de tempo correspondem às médias de cinco execuções, convertidas para milissegundos.

As quantidades de caminhos e chamadas recursivas também correspondem às médias registradas.

Tabela 15: Resultados experimentais da busca completa.

| $V$ | $E$ | Tempo (ms) | Caminhos | Chamadas |
|---|---|---|---|---|
| 5 | 7 | 0,004 | 1 | 5 |
| 10 | 19 | 0,013 | 3 | 13 |
| 15 | 29 | 0,017 | 4 | 22 |
| 20 | 40 | 0,023 | 5 | 38 |
| 25 | 51 | 0,038 | 6 | 59 |
| 30 | 65 | 0,079 | 10 | 126 |
| 35 | 80 | 0,153 | 16 | 220 |
| 40 | 93 | 0,965 | 136 | 1651 |

Todas as 40 execuções desse conjunto foram concluídas, apresentando taxa de conclusão de 100%. O menor custo encontrado foi igual a 596 em todas as instâncias desse conjunto de experimentos. Observa-se que o aumento da quantidade de vértices e arestas foi acompanhado pelo crescimento do número de caminhos explorados e de chamadas recursivas. Na instância com 5 vértices, foram registrados apenas um caminho completo e cinco chamadas recursivas. Já na instância com 40 vértices, foram explorados 136 caminhos, com 1651 chamadas recursivas.

O tempo médio de execução aumentou de 0,004 ms para 0,965 ms, representando um crescimento de aproximadamente 241 vezes. Entretanto, os tempos absolutos permaneceram inferiores a um milissegundo, indicando que as instâncias avaliadas ainda apresentaram baixo custo computacional para a implementação.

Como os grafos utilizados possuem quantidades relativamente pequenas de arestas em relação ao número máximo possível, a quantidade de caminhos simples permaneceu limitada. Assim, os resultados demonstram o crescimento do esforço computacional nas instâncias avaliadas, mas não representam diretamente o comportamento do pior caso teórico.

### 8.5.3 Comparação com Dijkstra

Para verificar os custos encontrados pela busca completa, foi realizada uma comparação com o algoritmo de Dijkstra desenvolvido na Tarefa 4. Os resultados são apresentados na Tabela 16.

Tabela 16: Comparação dos custos com Dijkstra.

| $V$ | Busca completa | Dijkstra | Concluída |
|---|---|---|---|
| 5 | 10 | 10 | Sim |
| 10 | 7 | 7 | Sim |
| 15 | 5 | 5 | Sim |
| 20 | 59 | 8 | Não |
| 25 | 81 | 5 | Não |

Nas instâncias com 5, 10 e 15 vértices, a busca completa foi concluída e apresentou custos idênticos aos calculados por Dijkstra.

Esses resultados fornecem evidências da correção da implementação para os casos avaliados. Nas instâncias com 20 e 25 vértices, a busca completa foi interrompida antes de examinar todos os caminhos possíveis. Consequentemente, os custos encontrados foram superiores aos obtidos por Dijkstra. Para 20 vértices, a busca completa encontrou custo 59, enquanto Dijkstra encontrou custo 8. Para 25 vértices, os custos foram 81 e 5, respectivamente. Essas diferenças não indicam necessariamente um erro na implementação, pois as buscas interrompidas não garantem a identificação do caminho ótimo.

Os resultados demonstram a importância de distinguir execuções concluídas de execuções interrompidas durante a avaliação experimental.

### 8.5.4 Análise dos resultados

Os experimentos realizados permitiram observar o comportamento da busca completa em diferentes configurações de grafos. No primeiro conjunto de testes, o aumento do número de vértices foi acompanhado pelo crescimento do tempo de execução e da quantidade de chamadas recursivas. Entretanto, todas as execuções foram concluídas rapidamente, pois a quantidade de caminhos simples existentes nas instâncias permaneceu relativamente pequena. No segundo conjunto de testes, a busca completa apresentou resultados equivalentes aos de Dijkstra nas instâncias em que a exploração foi concluída.

Nas instâncias interrompidas, os custos encontrados foram superiores aos valores ótimos calculados por Dijkstra. Esse comportamento evidencia uma limitação da enumeração exaustiva: a obtenção de uma solução ótima depende da exploração de todos os caminhos simples possíveis. Em contraste, Dijkstra utiliza uma estratégia especializada que evita essa enumeração, apresentando complexidade polinomial para grafos com pesos não negativos. Portanto, os resultados experimentais são compatíveis com a análise teórica: a busca completa pode ser adequada para instâncias pequenas ou com poucos caminhos possíveis, mas sua utilização torna-se limitada quando a quantidade de alternativas cresce significativamente.

## 8.6 Considerações da tarefa

A implementação do algoritmo de caminho mínimo por busca completa permitiu aplicar os conceitos de recursividade, exploração em profundidade e enumeração de soluções estudados na disciplina de Projeto e Análise de Algoritmos. A estratégia desenvolvida consiste em explorar os caminhos simples existentes entre dois vértices de um grafo direcionado e ponderado, calculando o custo de cada percurso e armazenando a melhor solução encontrada. Para evitar a formação de ciclos, foi utilizado um vetor de controle dos vértices visitados. A técnica de *backtracking* permitiu desfazer as escolhas realizadas durante a exploração recursiva, possibilitando a construção de diferentes caminhos sem repetição de vértices. A implementação também incorporou mecanismos para contabilizar os caminhos explorados, registrar o número de chamadas recursivas e controlar o tempo máximo de execução. Essas funcionalidades possibilitaram avaliar experimentalmente o comportamento do algoritmo em diferentes instâncias.

A análise teórica demonstrou que a quantidade de caminhos simples pode apresentar crescimento fatorial em relação ao número de vértices. Consequentemente, a busca completa possui um custo computacional significativamente superior ao de algoritmos especializados, como Dijkstra, em instâncias

com grande quantidade de caminhos possíveis. Em relação ao consumo de memória, a implementação utiliza espaço auxiliar $O(V)$, devido aos vetores de controle e à pilha de chamadas recursivas. Considerando também o armazenamento do grafo, o espaço total é $O(V + E)$. Nos experimentos de desempenho, foram avaliadas instâncias com 5 a 40 vértices. Todas as execuções desse conjunto foram concluídas, apresentando crescimento do tempo de processamento e do número de chamadas recursivas à medida que as dimensões dos grafos aumentaram. Na comparação com Dijkstra, as instâncias de 5, 10 e 15 vértices apresentaram custos idênticos aos valores ótimos, confirmando o funcionamento da busca completa nos casos avaliados. Entretanto, nas instâncias de 20 e 25 vértices, a execução foi interrompida antes da exploração de todos os caminhos. Nessas situações, os custos encontrados foram superiores aos calculados por Dijkstra, evidenciando a impossibilidade de garantir a optimalidade de uma busca não concluída. Como principal limitação, destaca-se o crescimento combinatório da quantidade de caminhos simples, que pode tornar a exploração exaustiva computacionalmente inviável em grafos maiores ou mais densos. Dessa forma, a busca completa constitui uma abordagem importante para compreender o problema de caminho mínimo e avaliar o custo da enumeração de soluções. Entretanto, para instâncias de maiores dimensões, algoritmos de complexidade polinomial, como Dijkstra, apresentam vantagens significativas de eficiência. Portanto, a tarefa permitiu relacionar a análise teórica de complexidade aos resultados experimentais, demonstrando a influência da quantidade de caminhos possíveis sobre o desempenho de um algoritmo de busca exaustiva.

# 9 Tarefa 6 – Caminho Mínimo utilizando Backtracking

## 9.1 Descrição do Problema

Esta tarefa consiste em determinar o caminho de menor custo entre dois vértices de um grafo direcionado e ponderado, utilizando a técnica de backtracking.

A implementação parte da busca completa desenvolvida na Tarefa 5, incorporando estratégias de poda para evitar a exploração de caminhos que não podem melhorar a melhor solução encontrada.

O objetivo é comparar o desempenho da busca completa com duas versões de backtracking, considerando o tempo de execução, o número de chamadas recursivas, a quantidade de caminhos explorados e o número de podas realizadas.

## 9.2 Estratégia de Solução

Foram implementadas três estratégias para a resolução do problema:

1. **Busca completa:** explora todos os caminhos simples entre os vértices de origem e destino, sem aplicar podas.
2. **Backtracking com poda por custo:** interrompe a exploração de um ramo quando o custo acumulado é maior ou igual ao custo da melhor solução já encontrada.
3. **Backtracking com limite inferior:** utiliza as distâncias mínimas até o destino, calculadas previamente pelo algoritmo de Dijkstra, para identificar ramos que não podem produzir uma solução melhor.

Na primeira estratégia de poda, um ramo é descartado quando:

$$custoAtual \geq melhorCusto$$

Na segunda estratégia, considera-se também uma estimativa do custo restante até o destino:

$$custoAtual + distanciaAteDestino[atual] \geq melhorCusto$$

Como os pesos das arestas são não negativos, essas condições permitem descartar ramos sem eliminar soluções de custo inferior ao melhor valor conhecido.

O algoritmo mantém um vetor de vértices visitados, impedindo a formação de ciclos durante a exploração. Ao retornar de uma chamada recursiva, o vértice é desmarcado, permitindo sua utilização em outros caminhos.

## 9.3 Implementação

A implementação foi desenvolvida na linguagem C, utilizando a estrutura de grafos com listas de adjacência da tarefa anterior. O programa permite selecionar três modos de execução: busca completa, backtracking com poda por custo e backtracking com poda por limite inferior. A função `explorarCaminhos` realiza a busca recursiva, armazenando o caminho atual, o custo acumulado e os vértices visitados. Durante a execução, são contabilizados o número de chamadas recursivas, os caminhos completos explorados e a quantidade de podas realizadas.

Na primeira estratégia, a exploração é interrompida quando o custo acumulado não pode melhorar a melhor solução conhecida. Na segunda estratégia, a função `calcularDistancias` utiliza Dijkstra para determinar os menores custos restantes até o destino. Essas distâncias são utilizadas como limites inferiores durante a exploração recursiva. O programa também permite estabelecer um limite máximo de tempo, registrando se a busca foi concluída ou interrompida. Os resultados são apresentados no terminal, incluindo o melhor caminho encontrado, seu custo e as métricas de desempenho.

## 9.4 Análise de Complexidade

A busca completa apresenta complexidade exponencial no pior caso, pois pode ser necessário explorar um número exponencial ou fatorial de caminhos simples.

Em grafos direcionados completos, a quantidade de caminhos simples entre dois vértices pode crescer na ordem de $O(V!)$. As estratégias de backtracking reduzem a quantidade de caminhos efetivamente explorados, mas não alteram a complexidade fatorial no pior caso. A poda por custo apresenta custo constante para verificar sua condição, enquanto a poda por limite inferior exige um pré-processamento com Dijkstra.

Na implementação utilizada, Dijkstra percorre os vértices e procura arestas de entrada no grafo original, apresentando complexidade $O(V^2 + VE)$. Assim, o custo total da segunda estratégia inclui esse pré-processamento e a exploração recursiva. O espaço auxiliar utilizado pela busca recursiva é $O(V)$, considerando o vetor de visitados, o caminho atual e a pilha de recursão. A segunda estratégia também armazena um vetor de distâncias de tamanho $O(V)$.

## 9.5 Validação dos Resultados

Para verificar a correção dos algoritmos, os custos encontrados foram comparados com os valores ótimos calculados por uma implementação independente do algoritmo de Dijkstra. Foram verificadas 135 execuções, correspondentes a 15 instâncias de grafos, três algoritmos e três repetições. Em 117 execuções, o custo encontrado coincidiu com o valor ótimo. As 18 divergências ocorreram exclusivamente na busca completa, em instâncias nas quais a execução foi interrompida pelo limite de tempo estabelecido.

Nas execuções concluídas, os três algoritmos encontraram os custos ótimos. Os resultados demonstram que as estratégias de poda preservaram a correção da solução e permitiram encontrar caminhos ótimos em instâncias nas quais a busca completa não conseguiu terminar no tempo disponível.

## 9.6 Experimentos e Resultados

Os experimentos foram realizados utilizando grafos direcionados e ponderados, com diferentes quantidades de vértices e densidades de arestas. Foram comparadas três abordagens: busca completa, backtracking com poda por custo e backtracking com poda por custo e limite inferior. Para cada instância, foram registrados o tempo de execução, a quantidade de chamadas recursivas, o número de caminhos explorados e a quantidade de podas realizadas.

As execuções foram submetidas a um limite de tempo, permitindo avaliar também o comportamento dos algoritmos em instâncias nas quais a exploração completa se torna computacionalmente inviável.

Tabela 17: Tempo médio de execução dos algoritmos

| V | Densidade | Completa | Poda 1 | Podas 1+2 |
|---|---|---|---|---|
| 10 | 20% | 8,00 $\mu$s | 4,00 $\mu$s | 28,80 $\mu$s |
| 15 | 20% | 432,20 $\mu$s | 15,40 $\mu$s | 57,40 $\mu$s |
| 20 | 20% | 36,71 ms | 8,40 $\mu$s | 99,00 $\mu$s |
| 10 | 50% | 2,31 ms | 15,80 $\mu$s | 29,80 $\mu$s |
| 15 | 50% | 6,561 s | 22,60 $\mu$s | 73,00 $\mu$s |
| 20 | 50% | 60,000 s | 98,40 $\mu$s | 130,00 $\mu$s |
| 10 | 80% | 67,74 ms | 62,20 $\mu$s | 37,40 $\mu$s |
| 15 | 80% | 60,000 s | 135,40 $\mu$s | 73,20 $\mu$s |
| 20 | 80% | 60,000 s | 185,40 $\mu$s | 143,80 $\mu$s |

## 9.7 Discussão dos Resultados

Os resultados demonstram que a utilização de podas reduziu significativamente a quantidade de chamadas recursivas em comparação com a busca completa. A redução foi especialmente expressiva nas instâncias com maior número de vértices e maior densidade, nas quais a quantidade de caminhos possíveis cresce rapidamente. A estratégia que utiliza o limite inferior calculado por Dijkstra apresentou, em geral, maior redução no número de chamadas recursivas do que a poda baseada apenas no

Tabela 18: Redução das chamadas recursivas

| V | Densidade | Poda 1 | Podas 1+2 |
|---|---|---|---|
| 10 | 20% | 46,6667% | 66,6667% |
| 15 | 20% | 95,2381% | 97,6190% |
| 20 | 20% | 99,9714% | 99,9893% |
| 10 | 50% | 98,8107% | 99,5821% |
| 15 | 50% | 99,9993% | 99,9999% |
| 20 | 50% | 99,9994% | 99,9999% |
| 10 | 80% | 99,8068% | 99,9630% |
| 15 | 80% | 99,9995% | 99,9999% |
| 20 | 80% | 99,9988% | 99,9998% |

custo acumulado. Entretanto, uma redução no número de chamadas não implica necessariamente menor tempo de execução, pois a segunda estratégia possui o custo adicional do pré-processamento. Nas instâncias em que a busca completa foi interrompida pelo limite de tempo, suas métricas representam uma exploração parcial do espaço de soluções.

A validação independente com Dijkstra confirmou que os algoritmos com poda encontraram os custos ótimos nas instâncias avaliadas.

## 9.8 Considerações da tarefa

A implementação demonstrou que o backtracking com estratégias de poda pode reduzir significativamente o esforço computacional necessário para encontrar o caminho mínimo. A poda baseada no custo acumulado apresentou melhorias expressivas em relação à busca completa, enquanto a utilização de um limite inferior calculado por Dijkstra permitiu reduzir ainda mais a quantidade de chamadas recursivas. Entretanto, a segunda estratégia possui um custo adicional de pré-processamento, que pode não compensar em instâncias pequenas. A validação independente confirmou que os algoritmos com poda encontraram os custos ótimos nas instâncias testadas. Conclui-se que as podas tornam a exploração recursiva mais eficiente na prática, embora não eliminem a complexidade fatorial do problema no pior caso.

## 9.9 Questões Teóricas

**1. As estratégias de poda preservam a solução ótima?** Sim. Como os pesos das arestas são não negativos, o custo de um caminho não diminui durante sua extensão. Assim, quando o custo acumulado é maior ou igual ao melhor custo conhecido, continuar a exploração não poderá produzir uma solução estritamente melhor.

Da mesma forma, a distância mínima calculada por Dijkstra representa um limite inferior para o custo restante até o destino. Portanto, as duas estratégias preservam o custo ótimo da solução.

**2. Qual é a complexidade no pior caso?** A complexidade permanece fatorial no pior caso, pois o algoritmo pode precisar explorar uma quantidade de caminhos simples que cresce na ordem de $O(V!)$. As podas reduzem o espaço efetivamente explorado em muitas instâncias, mas não garantem uma melhoria na ordem assintótica do pior caso.

**3. Em quais situações as podas são mais eficientes?** As podas tendem a ser mais eficientes quando o grafo possui muitos caminhos alternativos e uma solução de baixo custo é encontrada no início da exploração. Nessas condições, diversos ramos podem ser descartados antes de alcançar o destino. A poda por limite inferior também é vantajosa quando as distâncias restantes permitem identificar antecipadamente caminhos que não podem melhorar a solução.

**4. Como a qualidade da solução inicial influencia o desempenho?** Uma solução inicial de baixo custo estabelece um limite superior mais restritivo para a busca. Quanto menor esse limite, maior a possibilidade de descartar caminhos cujo custo acumulado, ou custo estimado total, não possa produzir uma solução melhor.

Por outro lado, uma solução inicial de custo elevado permite menos podas, aumentando a quantidade de caminhos explorados.

# 10 Tarefa 7 – Estratégia Gulosa para o Problema de Caminho Mínimo

## 10.1 Descrição do Problema

O objetivo desta tarefa consiste em implementar uma estratégia gulosa para encontrar um caminho entre dois vértices de um grafo direcionado e ponderado.

Diferentemente do algoritmo de Dijkstra, a estratégia proposta não garante a obtenção do caminho mínimo, pois suas decisões são tomadas considerando apenas informações locais. O algoritmo deve encontrar um caminho entre a origem e o destino, quando possível, retornando a sequência de vértices percorridos, o custo acumulado e o número de vértices visitados.

## 10.2 Estratégia de Solução

A estratégia implementada consiste em selecionar, a cada iteração, a aresta de menor peso entre aquelas que partem do vértice atual e conduzem a vértices ainda não visitados. Inicialmente, o vértice de origem é marcado como visitado e inserido no caminho. Em seguida, são examinadas as arestas de saída do vértice corrente. A aresta válida de menor peso é selecionada, e seu vértice de destino passa a ser o novo vértice corrente. Esse procedimento é repetido até que o destino seja alcançado ou não existam mais arestas válidas para continuar o percurso.

Em caso de empate entre arestas de mesmo peso, é escolhido o vértice de menor identificador. A estratégia não realiza retrocesso, de modo que uma escolha anterior não pode ser desfeita. Para verificar a qualidade das soluções encontradas, foi utilizada a implementação do algoritmo de Dijkstra desenvolvida na Tarefa 4.

## 10.3 Implementação

A implementação foi desenvolvida em linguagem C, reutilizando a estrutura de grafos das tarefas anteriores. O grafo é armazenado por meio de listas de adjacência, permitindo percorrer as arestas de saída de cada vértice. A função principal da estratégia gulosa recebe o grafo, os vértices de origem e destino e o tempo máximo permitido para execução. Durante o processamento, são armazenados o caminho percorrido, o custo acumulado e a quantidade de vértices visitados.

A implementação também utiliza um vetor de visitados para impedir que o algoritmo retorne a vértices anteriormente percorridos. Os testes foram realizados utilizando grafos no formato DIMACS, mantendo a compatibilidade com as implementações anteriores. Para a comparação experimental, foi reutilizado o executável do algoritmo de Dijkstra da Tarefa 4, sem necessidade de uma nova implementação.

## 10.4 Análise de Complexidade

Considere um grafo com $V$ vértices e $E$ arestas, representado por listas de adjacência. Como cada vértice pode ser visitado no máximo uma vez, o algoritmo guloso realiza no máximo $V$ iterações. Em cada iteração, são examinadas as arestas de saída do vértice corrente para selecionar a aresta válida de menor peso. Como nenhum vértice é processado mais de uma vez, cada lista de adjacência é examinada no máximo uma vez durante a execução.

Consequentemente, a complexidade de tempo no pior caso é:

$$T(V, E) = O(V + E).$$

O algoritmo utiliza um vetor para identificar os vértices visitados e outro para armazenar o caminho encontrado. Portanto, sua complexidade de espaço auxiliar é:

$$S(V) = O(V).$$

Embora apresente baixa complexidade computacional, a estratégia gulosa não garante a solução ótima nem a chegada ao vértice de destino.

## 10.5 Experimentos e Resultados

Os experimentos utilizaram 15 instâncias sintéticas de grafos direcionados, com 5, 10, 15, 20 e 25 vértices e densidades de 20%, 50% e 80%.

Cada algoritmo foi executado 30 vezes por instância, totalizando 900 execuções. Foram analisados o custo dos caminhos encontrados, o erro relativo em comparação com Dijkstra, a quantidade de vértices visitados e os tempos de execução.

Tabela 19: Classificação das soluções encontradas pela estratégia gulosa

| Classificação | Instâncias | Percentual |
|---|---|---|
| Solução ótima | 4 | 26,67% |
| Solução subótima | 9 | 60,00% |
| Destino não encontrado | 2 | 13,33% |
| Total | 15 | 100,00% |

O algoritmo guloso conseguiu alcançar o destino em 13 das 15 instâncias analisadas. Entretanto, apenas quatro soluções apresentaram custo igual ao obtido por Dijkstra.

Em nove instâncias, o algoritmo encontrou caminhos válidos, porém com custo superior ao ótimo. Nas duas instâncias restantes, a estratégia gulosa não conseguiu alcançar o destino, apesar da existência de um caminho no grafo.

## 10.6 Contraexemplo da Estratégia Gulosa

Considere um grafo direcionado com quatro vértices, origem 1 e destino 4, contendo as seguintes arestas:

Tabela 20: Arestas do grafo utilizado como contraexemplo

| Origem | Destino | Peso |
|---|---|---|
| 1 | 2 | 1 |
| 1 | 3 | 4 |
| 2 | 4 | 100 |
| 3 | 4 | 4 |

Ao iniciar no vértice 1, o algoritmo guloso seleciona a aresta $(1, 2)$, de peso 1, pois ela possui peso menor que a aresta $(1, 3)$, de peso 4.

No vértice 2, o algoritmo segue para o destino 4, utilizando uma aresta de peso 100.

Assim, o caminho encontrado é:

$$P_{\text{guloso}} = (1, 2, 4)$$

com custo:

$$C_{\text{guloso}} = 1 + 100 = 101.$$

Entretanto, o caminho mínimo é:

$$P_{\text{ótimo}} = (1, 3, 4)$$

com custo:

$$C_{\text{ótimo}} = 4 + 4 = 8.$$

O erro relativo é calculado por:

$$Erro = \frac{C_{\text{guloso}} - C_{\text{ótimo}}}{C_{\text{ótimo}}}.$$

Substituindo os valores:

$$Erro = \frac{101 - 8}{8} = 11{,}625.$$

Portanto, o erro percentual corresponde a:

$$Erro_{\%} = 1162{,}5\%.$$

Esse exemplo demonstra que selecionar a aresta de menor peso em cada etapa não garante a obtenção do caminho de menor custo total.

Tabela 21: Comparação dos tempos médios de execução em microssegundos

| Vértices | Densidade | Guloso | Dijkstra |
|---|---|---|---|
| 5 | 20% | 2,97 | 2,87 |
| 10 | 20% | 1,90 | 2,43 |
| 15 | 20% | 2,37 | 2,63 |
| 20 | 20% | 2,50 | 5,80 |
| 25 | 20% | 5,47 | 4,43 |
| 5 | 50% | 2,37 | 1,67 |
| 10 | 50% | 2,23 | 2,47 |
| 15 | 50% | 2,83 | 5,67 |
| 20 | 50% | 5,77 | 4,40 |
| 25 | 50% | 6,27 | 7,20 |
| 5 | 80% | 1,60 | 2,03 |
| 10 | 80% | 4,27 | 2,77 |
| 15 | 80% | 5,20 | 3,20 |
| 20 | 80% | 2,87 | 4,97 |
| 25 | 80% | 4,23 | 10,13 |

Os tempos registrados apresentaram valores muito baixos, geralmente na ordem de microssegundos.

Em algumas instâncias, o algoritmo guloso apresentou menor tempo médio de execução. Em outras, Dijkstra obteve tempos inferiores. Devido à pequena duração das execuções e à variabilidade observada, não é possível estabelecer uma superioridade consistente de desempenho apenas com esses resultados. Entretanto, os experimentos demonstram que o algoritmo de Dijkstra possui a vantagem de garantir o caminho mínimo, enquanto a estratégia gulosa implementada não oferece essa garantia.

## 10.7 Diferença entre a Estratégia Gulosa e o Algoritmo de Dijkstra

Embora ambos os algoritmos utilizem decisões gulosas, seus critérios de seleção são diferentes. A estratégia implementada nesta tarefa escolhe a aresta de menor peso que parte do vértice corrente, considerando apenas o custo local da próxima movimentação.

Por outro lado, Dijkstra seleciona o vértice não processado com a menor distância acumulada a partir da origem. Além disso, Dijkstra realiza o relaxamento das arestas, atualizando as melhores distâncias conhecidas para os demais vértices. Como os pesos das arestas são não negativos, quando um vértice é selecionado com a menor distância provisória, nenhum caminho que passe por vértices ainda não processados poderá produzir uma distância menor até ele.

Essa propriedade permite que sua distância seja considerada definitiva. Portanto, Dijkstra garante a obtenção dos menores caminhos em grafos com pesos não negativos, enquanto a estratégia gulosa local pode tomar decisões que conduzem a caminhos subótimos ou que não alcançam o destino.

## 10.8 Considerações da tarefa

A Tarefa 7 demonstrou as limitações de uma estratégia gulosa baseada exclusivamente na escolha da aresta de menor peso.

Embora o algoritmo apresente complexidade de tempo $O(V+E)$, suas decisões locais não garantem a obtenção do caminho mínimo. Nos experimentos realizados, apenas 4 das 15 instâncias apresentaram soluções ótimas, enquanto 9 produziram soluções subótimas e 2 não permitiram que o algoritmo guloso

alcançasse o destino. O contraexemplo apresentado confirmou formalmente essa limitação. A comparação com Dijkstra evidenciou que o critério utilizado para realizar a escolha gulosa é fundamental para a correção do algoritmo.

Conclui-se que estratégias gulosas podem apresentar baixo custo computacional, mas sua capacidade de encontrar soluções ótimas depende das propriedades do problema e do critério de seleção adotado.

# 11 Tarefa 8 – Caminho Mínimo utilizando Simulated Annealing

## 11.1 Descrição do problema

A oitava tarefa consiste na implementação de uma metaheurística para o problema do caminho mínimo em grafos direcionados e ponderados. Dado um grafo $G = (V, E)$, um vértice de origem $s \in V$ e um vértice de destino $t \in V$, o objetivo é encontrar um caminho válido entre $s$ e $t$ que apresente o menor custo possível.

O custo de um caminho $P = (v_0, v_1, \ldots, v_k)$ é definido pela soma dos pesos das arestas percorridas:

$$C(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1}). \tag{81}$$

O problema de otimização pode ser representado por:

$$\min_{P \in \mathcal{P}(s,t)} C(P), \tag{82}$$

em que $\mathcal{P}(s, t)$ representa o conjunto de caminhos válidos entre a origem e o destino. Diferentemente dos algoritmos exatos utilizados nas tarefas anteriores, uma metaheurística não garante necessariamente a obtenção da solução ótima. A abordagem escolhida foi o *Simulated Annealing*, também denominado Recozimento Simulado, que utiliza mecanismos probabilísticos para explorar o espaço de soluções. Os resultados são comparados com o algoritmo de Dijkstra implementado na Tarefa 4, utilizado como referência para os custos ótimos.

## 11.2 Estratégia de solução

O Simulated Annealing é uma metaheurística inspirada no processo físico de recozimento, no qual um material é aquecido e resfriado gradualmente para alcançar configurações energeticamente favoráveis. No contexto da otimização, cada configuração representa uma solução candidata e sua energia corresponde ao valor da função objetivo. O algoritmo mantém uma solução atual e gera soluções vizinhas. Soluções melhores são aceitas, enquanto soluções piores podem ser aceitas com determinada probabilidade. Essa característica permite escapar de mínimos locais durante a busca.

### 11.2.1 Representação das soluções

Cada solução é representada por uma sequência de vértices que forma um caminho da origem até o destino. A representação utilizada é:

$$P = (s, v_1, v_2, \ldots, t). \tag{83}$$

Para garantir a validade das soluções, os caminhos gerados devem respeitar as arestas existentes no grafo. A implementação utiliza caminhos simples, nos quais um vértice não pode aparecer mais de uma vez. Como os pesos considerados são não negativos, existe um caminho mínimo simples sempre que o destino é alcançável. Além da sequência de vértices, são armazenados os pesos das arestas utilizadas, permitindo calcular corretamente o custo acumulado, inclusive quando existem arestas paralelas.

### 11.2.2 Função de avaliação

A função de avaliação, também denominada função de aptidão ou *fitness*, corresponde ao custo total do caminho:

$$f(P) = C(P). \tag{84}$$

Como o problema é de minimização, caminhos com menores custos são considerados soluções de melhor qualidade. A implementação trabalha somente com soluções completas e válidas entre a origem e o destino. Tentativas que não produzem caminhos válidos são descartadas.

### 11.2.3 Construção da solução inicial

Inicialmente, é necessário construir um caminho válido entre a origem e o destino. Para isso, foi utilizada uma busca em profundidade aleatorizada, que percorre as arestas de saída dos vértices em ordem aleatória. Um vetor de vértices visitados impede a formação de ciclos no caminho. Quando uma escolha não permite continuar a construção, a busca realiza retrocesso, retornando ao vértice anterior. Também são utilizados limites de chamadas recursivas e de tempo, evitando buscas excessivamente longas. Caso a construção inicial não encontre uma solução dentro dos limites definidos, a execução é encerrada sem resultado válido.

### 11.2.4 Geração de soluções vizinhas

A geração de vizinhos é realizada por meio da modificação de uma parte do caminho atual. Inicialmente, seleciona-se aleatoriamente uma posição de corte no caminho. Os vértices anteriores e o próprio vértice de corte são preservados. A partir desse ponto, uma nova sequência de vértices é construída utilizando a busca em profundidade aleatorizada. Dessa forma, o algoritmo mantém um prefixo do caminho original e substitui seu sufixo por uma nova alternativa.

A estratégia permite explorar diferentes rotas sem necessariamente reconstruir todo o caminho. Quando não é possível construir um sufixo válido dentro dos limites estabelecidos, a tentativa de geração é descartada.

### 11.2.5 Critério de aceitação

Seja $P_a$ a solução atual e $P_v$ a solução vizinha.

A diferença entre os custos é definida por:

$$\Delta = C(P_v) - C(P_a). \tag{85}$$

Quando $\Delta \leq 0$, a solução vizinha é aceita, pois apresenta custo menor ou igual ao da solução atual. Quando $\Delta > 0$, a solução pode ser aceita com probabilidade:

$$p = \exp\left(-\frac{\Delta}{T}\right), \tag{86}$$

em que $T$ representa a temperatura atual. Em temperaturas elevadas, existe maior probabilidade de aceitar soluções piores, favorecendo a exploração do espaço de busca. À medida que a temperatura diminui, essa probabilidade também diminui, tornando a busca mais seletiva. A aceitação de soluções piores permite que o algoritmo explore regiões diferentes do espaço de soluções, evitando depender exclusivamente de melhorias imediatas.

### 11.2.6 Resfriamento

Foi utilizado um esquema de resfriamento geométrico, definido por:

$$T_{k+1} = \alpha T_k, \tag{87}$$

em que $\alpha$ é a taxa de resfriamento e satisfaz $0 < \alpha < 1$. O processo é repetido até que a temperatura atinja o limite mínimo estabelecido ou o tempo máximo de execução seja alcançado. Durante a execução, o algoritmo mantém separadamente a melhor solução encontrada, pois a solução atual pode ser substituída temporariamente por uma solução pior.

## 11.3 Implementação

A implementação foi desenvolvida na linguagem de programação C. O código foi organizado em arquivos responsáveis pela representação do grafo, manipulação dos caminhos, execução da metaheurística e processamento da entrada. Os principais arquivos são:

- `grafo.h` e `grafo.c`: representação e manipulação do grafo.
- `caminho.h` e `caminho.c`: representação dos caminhos, construção inicial e geração de vizinhos.

- `simulated_annealing.h` e `simulated_annealing.c`: configuração e execução da metaheurística.
- `main.c`: leitura dos argumentos, execução do algoritmo e apresentação dos resultados.

A entrada utiliza o formato DIMACS, empregado também nas tarefas anteriores.

### 11.3.1 Estrutura dos caminhos

A estrutura `Caminho` armazena os vértices, os pesos das arestas, a quantidade de vértices e o custo total.

```
typedef struct {
    int *vertices;
    int64_t *pesos;
    int tamanho;
    int64_t custo;
} Caminho;
```

Código 25: Estrutura utilizada para representar um caminho.

O vetor `vertices` armazena a sequência de vértices percorridos.

O vetor `pesos` registra o peso da aresta utilizada para alcançar cada posição do caminho.

O campo `tamanho` indica a quantidade de vértices, enquanto `custo` representa a soma dos pesos das arestas.

### 11.3.2 Parâmetros da metaheurística

Os parâmetros de execução são armazenados na estrutura `ConfigSA`.

```
typedef struct {
    double temperaturaInicial;
    double temperaturaMinima;
    double taxaResfriamento;
    int iteracoesPorTemperatura;
    double tempoMaximo;
    unsigned int semente;
} ConfigSA;
```

Código 26: Estrutura de configuracao do Simulated Annealing.

A configuração utilizada nos experimentos é apresentada na Tabela 22.

Tabela 22: Parâmetros utilizados no Simulated Annealing.

| Parâmetro | Valor |
|---|---|
| Temperatura inicial | 100 |
| Temperatura mínima | 0,01 |
| Taxa de resfriamento | 0,995 |
| Iterações por temperatura | 20 |
| Tempo máximo por execução | 5 s |
| Execuções por instância | 30 |

As sementes do gerador pseudoaleatório foram modificadas entre as execuções, permitindo observar diferentes trajetórias de busca para uma mesma instância.

### 11.3.3 Execução do algoritmo

A função `executarSA` recebe o grafo, a origem, o destino e a configuração da metaheurística. Inicialmente, são alocadas as estruturas necessárias e construída uma solução válida. Em seguida, são realizadas as etapas de geração de vizinhos, avaliação, aceitação e atualização da melhor solução.

O critério de aceitação probabilística é apresentado no Código 27.

```c
int aceitar = 0;

if (vizinho.custo <= atual.custo) {
    aceitar = 1;
} else {
    double diferenca =
        (double)vizinho.custo -
        (double)atual.custo;

    double probabilidade =
        exp(-diferenca / temperatura);

    if (aleatorio01(&semente) < probabilidade)
        aceitar = 1;
}
```

Código 27: Criterio de aceitacao do Simulated Annealing.

O cálculo da diferença utiliza o tipo `double`, evitando estouro de inteiro na subtração entre custos armazenados em 64 bits. A melhor solução encontrada é mantida independentemente da solução atual, garantindo que uma solução de menor custo não seja perdida durante a busca.

## 11.4 Análise de complexidade

A análise considera a quantidade de vértices $V$, a quantidade de arestas $E$, o número de iterações $I$ e o limite de chamadas da busca aleatorizada $B$.

### 11.4.1 Complexidade de tempo

A construção de um caminho utiliza uma busca em profundidade aleatorizada. Em uma busca em profundidade convencional, o custo é $O(V+E)$. Entretanto, a implementação utilizada pode examinar diferentes alternativas durante o retrocesso e possui um limite explícito de chamadas recursivas. Em cada chamada, a geração e a permutação das opções de saída exigem tempo proporcional ao grau do vértice. Considerando uma estimativa conservadora de $O(V+E)$ por chamada, o custo de uma tentativa limitada por $B$ chamadas é dado por:

$$T_{\text{busca}} = O(B(V+E)). \tag{88}$$

Na implementação, o limite de chamadas é definido proporcionalmente à quantidade de vértices:

$$B = \min(100V, \texttt{INT_MAX}). \tag{89}$$

Desconsiderando os limites de representação inteira, tem-se $B = O(V)$. Cada iteração da metaheurística realiza uma tentativa de geração de vizinho e operações de avaliação e cópia de caminhos. A cópia de um caminho possui custo $O(V)$, pois o caminho simples contém no máximo $V$ vértices. Assim, um limite superior conservador para o tempo total é:

$$T_{\text{SA}} = O\left((I+1)B(V+E)\right). \tag{90}$$

O termo adicional corresponde à construção da solução inicial. A quantidade de iterações depende dos parâmetros de temperatura, resfriamento e iterações por nível, além da interrupção por tempo. Portanto, a complexidade da metaheurística depende tanto das dimensões do grafo quanto da configuração experimental. Essa expressão representa um limite conservador da implementação, e não uma garantia de que a solução ótima será encontrada.

### 11.4.2 Complexidade de espaço

A representação do grafo por listas de adjacência exige espaço:

$$S_{\text{grafo}} = O(V+E). \tag{91}$$

O algoritmo mantém três estruturas de caminho: atual, vizinha e melhor. Cada uma armazena até $V$ vértices e seus respectivos pesos, exigindo espaço $O(V)$. A busca em profundidade utiliza um vetor de visitados e uma pilha recursiva de profundidade máxima $O(V)$. As listas temporárias de opções armazenadas durante a recursão também ocupam memória. Ao longo de um caminho simples, a soma dos graus de saída dos vértices presentes na pilha é limitada por $E$. Portanto, o espaço total da implementação é:

$$\boxed{S_{\text{SA}} = O(V + E)}. \tag{92}$$

## 11.5 Experimentos e resultados

Os experimentos tiveram como objetivo avaliar a qualidade das soluções encontradas pelo Simulated Annealing e comparar seu desempenho com o algoritmo de Dijkstra.

### 11.5.1 Metodologia experimental

Foram utilizadas 15 instâncias sintéticas de grafos direcionados e ponderados. Os grafos possuem 5, 10, 15, 20 e 25 vértices, com densidades de 20%, 50% e 80%. Para cada instância, o Simulated Annealing foi executado 30 vezes, utilizando sementes pseudoaleatórias distintas. Ao todo, foram realizadas 450 execuções da metaheurística. Os custos ótimos foram obtidos utilizando Dijkstra, implementado na Tarefa 4. Para a análise de desempenho, Dijkstra também foi executado 30 vezes por instância.

Os resultados foram processados por scripts desenvolvidos em Python, que calcularam estatísticas e produziram arquivos CSV. As métricas analisadas incluem:

- melhor e pior custo encontrado;
- custo médio;
- desvio padrão dos custos;
- taxa de otimalidade;
- erro relativo;
- tempo médio de execução;
- desvio padrão dos tempos.

O erro relativo é definido por:

$$ER = \frac{C_{\text{SA}} - C_{\text{otimo}}}{C_{\text{otimo}}} \times 100, \tag{93}$$

para instâncias com custo ótimo positivo. A taxa de otimalidade corresponde à proporção de execuções nas quais o custo encontrado é igual ao custo ótimo de referência.

### 11.5.2 Qualidade das soluções

O Simulated Annealing encontrou caminhos válidos nas 450 execuções. Desse total, 446 produziram soluções ótimas e quatro produziram soluções subótimas. A taxa global de otimalidade foi:

$$\frac{446}{450} \times 100 \approx 99,11\%. \tag{94}$$

A Tabela 23 apresenta a classificação dos resultados.

Em 14 das 15 instâncias, todas as 30 execuções encontraram a solução ótima. A única instância com soluções subótimas foi o grafo de 25 vértices e densidade de 50%. Nessa instância, Dijkstra encontrou custo ótimo igual a 6, enquanto o Simulated Annealing apresentou custo médio igual a 6,4667. A taxa de otimalidade foi de 86,67%, correspondente a 26 execuções ótimas entre as 30 realizadas.

O erro relativo do custo médio foi:

$$ER = \frac{6,4667 - 6}{6} \times 100 \approx 7,78\%. \tag{95}$$

Tabela 23: Classificação das execuções do Simulated Annealing.

| Classificação | Execuções | Percentual |
|---|---|---|
| Solução ótima | 446 | 99,11% |
| Solução subótima | 4 | 0,89% |
| Sem caminho válido | 0 | 0,00% |
| Total | 450 | 100,00% |

Esse comportamento evidencia a influência das escolhas aleatórias sobre a qualidade das soluções. Embora o algoritmo tenha encontrado o custo ótimo em diferentes execuções, não conseguiu reproduzi-lo em todas as tentativas dessa instância.

### 11.5.3 Comparação dos tempos de execução

Os tempos médios obtidos são apresentados na Tabela 24.

Tabela 24: Tempos médios de execução do SA e de Dijkstra, em segundos.

| Densidade | $V$ | SA (s) | Dijkstra (s) |
|---|---|---|---|
| 20% | 5 | 0,066216 | 0,000003 |
| 20% | 10 | 0,072260 | 0,000003 |
| 20% | 15 | 0,271607 | 0,000002 |
| 20% | 20 | 0,142099 | 0,000006 |
| 20% | 25 | 1,032138 | 0,000003 |
| 50% | 5 | 0,065143 | 0,000002 |
| 50% | 10 | 0,107460 | 0,000004 |
| 50% | 15 | 0,141223 | 0,000006 |
| 50% | 20 | 0,442767 | 0,000005 |
| 50% | 25 | 0,283245 | 0,000005 |
| 80% | 5 | 0,058311 | 0,000002 |
| 80% | 10 | 0,119832 | 0,000002 |
| 80% | 15 | 0,151191 | 0,000004 |
| 80% | 20 | 0,227985 | 0,000005 |
| 80% | 25 | 0,288318 | 0,000011 |

Os resultados mostram que Dijkstra apresentou tempos médios inferiores aos do Simulated Annealing em todas as instâncias analisadas. A diferença está relacionada às características dos algoritmos. Dijkstra utiliza uma estratégia sistemática para determinar as menores distâncias, enquanto o Simulated Annealing realiza numerosas tentativas de construção e modificação de caminhos.

Além disso, os tempos de Dijkstra são muito pequenos, próximos da resolução da medição utilizada. Consequentemente, razões exatas entre os tempos devem ser interpretadas com cautela. Os resultados também mostram que o tempo do Simulated Annealing não cresce de maneira estritamente monotônica com a quantidade de vértices. Isso ocorre porque o esforço de construção dos caminhos depende da conectividade e da estrutura específica de cada grafo.

### 11.5.4 Discussão dos resultados

Os experimentos demonstraram que o Simulated Annealing apresentou elevada qualidade nas instâncias avaliadas, encontrando a solução ótima em 99,11% das execuções. Entretanto, essa qualidade foi obtida com um custo computacional consideravelmente superior ao observado para Dijkstra. No problema clássico de caminho mínimo com pesos não negativos, Dijkstra oferece garantia de otimalidade e possui complexidade polinomial. A implementação da Tarefa 4, baseada em fila de prioridade, apresenta desempenho adequado para esse tipo de problema. Por outro lado, o Simulated Annealing não oferece garantia de otimalidade, e seus resultados dependem da configuração dos parâmetros, da geração de vizinhos e das escolhas pseudoaleatórias. Os experimentos também possuem limitações: foram utilizados grafos relativamente pequenos, com no máximo 25 vértices, e uma única configuração de parâmetros da metaheurística.

Portanto, os resultados não devem ser generalizados para grafos maiores ou para outras configurações sem experimentos adicionais.

## 11.6 Considerações da tarefa

A implementação do Simulated Annealing permitiu estudar uma abordagem probabilística de otimização aplicada ao problema do caminho mínimo. Foram desenvolvidos procedimentos para construção de soluções iniciais, geração de caminhos vizinhos, avaliação de custos, aceitação probabilística e resfriamento.

A representação adotada garantiu que as soluções aceitas fossem caminhos válidos entre os vértices de origem e destino. Os experimentos demonstraram elevada taxa de otimalidade, mas também evidenciaram a possibilidade de obtenção de soluções subótimas. A comparação com Dijkstra mostrou que a metaheurística apresentou tempos de execução superiores, sem oferecer garantia de encontrar o ótimo.

Conclui-se que, para o problema clássico de caminho mínimo com pesos não negativos, Dijkstra é a alternativa mais adequada entre as abordagens avaliadas. O Simulated Annealing, entretanto, constitui uma ferramenta relevante para compreender técnicas de busca probabilística, exploração do espaço de soluções e equilíbrio entre qualidade e esforço computacional.

# 12 Tarefa 9 – Caminhos Mínimos utilizando Programação Dinâmica

## 12.1 Descrição do problema

A nona tarefa consiste em determinar os caminhos mínimos entre todos os pares de vértices de um grafo direcionado e ponderado, utilizando programação dinâmica. Dado um grafo $G = (V, E)$, em que cada aresta $(i, j)$ possui peso $w(i, j)$, deseja-se calcular a distância mínima $\delta(i, j)$ para todo par $i, j \in V$. Quando não existe caminho entre os vértices, a distância é representada por $\infty$.

A solução emprega o algoritmo de Floyd–Warshall, que calcula todas as distâncias em uma única execução. Além das distâncias, a implementação permite reconstruir um caminho mínimo entre dois vértices escolhidos. A entrada utiliza o formato DIMACS, com linhas `p sp` para informar as dimensões do grafo e linhas `a` para descrever suas arestas. A implementação adotada aceita pesos não negativos, conforme as restrições da função de inserção de arestas.

## 12.2 Estratégia de solução

O algoritmo de Floyd–Warshall utiliza programação dinâmica: resolve subproblemas relacionados, armazena os resultados intermediários e os reutiliza para construir soluções progressivamente mais abrangentes. Os subproblemas são definidos pelos pares de vértices e pelo conjunto de vértices intermediários autorizados em um caminho.

Seja $D^{(k)}[i][j]$ o menor custo de um caminho de $i$ até $j$ cujos vértices intermediários pertencem ao conjunto $\{1, \ldots, k\}$. Inicialmente, quando nenhum vértice intermediário é permitido, define-se:

$$D^{(0)}[i][j] = \begin{cases} 0, & \text{se } i = j, \\ w(i, j), & \text{se existe aresta de } i \text{ para } j, \\ \infty, & \text{caso contrário.} \end{cases} \tag{96}$$

Quando existem arestas paralelas, considera-se o menor peso entre elas. Em cada etapa $k$, um caminho mínimo entre $i$ e $j$ pode não utilizar o vértice $k$ como intermediário ou pode passar por ele. Portanto, a recorrência é:

$$\boxed{D^{(k)}[i][j] = \min\left\{D^{(k-1)}[i][j],\ D^{(k-1)}[i][k] + D^{(k-1)}[k][j]\right\}.} \tag{97}$$

O primeiro termo mantém o melhor caminho conhecido sem utilizar $k$ como intermediário; o segundo verifica se a passagem por $k$ produz uma distância menor. Após processar todos os vértices intermediários, $D^{(|V|)}[i][j] = \delta(i, j)$.

A implementação utiliza uma única matriz de distâncias, atualizada no próprio local. Essa otimização é válida porque a etapa correspondente a $k$ pode ser realizada sem manter uma cópia completa da matriz anterior. Como os pesos admitidos são não negativos, não existem ciclos de peso negativo nas instâncias processadas.

### 12.2.1 Reconstrução dos caminhos

Além da matriz de distâncias, utiliza-se uma matriz `proximo`. Para cada par alcançável $(i, j)$, `proximo[i][j]` indica o primeiro vértice a visitar ao sair de $i$ em direção a $j$. Inicialmente, para uma aresta direta de $i$ para $j$, define-se `proximo[i][j] = j`. Quando a passagem por $k$ melhora a distância, o primeiro passo do novo caminho é herdado de `proximo[i][k]`. A reconstrução percorre sucessivamente esses próximos vértices até atingir o destino.

## 12.3 Implementação

A solução foi desenvolvida em C, reutilizando a estrutura de grafo baseada em listas de adjacência. Os arquivos `grafo.h` e `grafo.c` definem e implementam a representação do grafo e suas operações de leitura, inserção de arestas e liberação de memória. O arquivo `floyd_warshall.h` declara a estrutura de resultados e as funções do algoritmo; `floyd_warshall.c` contém sua implementação; e `main.c` realiza a leitura, a execução e a apresentação dos resultados.

### 12.3.1 Estrutura de resultados

A estrutura `ResultadoFloydWarshall` armazena a quantidade de vértices, a matriz de distâncias e a matriz auxiliar de reconstrução de caminhos:

```
typedef struct {
    int numVertices;
    int64_t **distancia;
    int **proximo;
} ResultadoFloydWarshall;
```

Código 28: Estrutura de resultados do Floyd–Warshall.

A utilização de `int64_t` permite armazenar custos acumulados em inteiros de 64 bits. O valor `INT64_MAX` representa uma distância infinita. As matrizes são alocadas dinamicamente com índices de 1 até $|V|$, compatíveis com os identificadores dos vértices utilizados na entrada DIMACS.

### 12.3.2 Inicialização das distâncias

Inicialmente, a diagonal principal recebe zero e as demais posições recebem infinito. Em seguida, as arestas são percorridas nas listas de adjacência e seus pesos são incorporados à matriz, preservando o menor valor caso existam arestas paralelas. Para pares sem aresta direta, o campo `proximo` permanece igual a $-1$.

### 12.3.3 Atualização por programação dinâmica

A etapa principal percorre todos os possíveis vértices intermediários $k$ e, para cada um deles, examina todos os pares $(i, j)$. O Código 29 apresenta a atualização central, com verificações para evitar somas com infinito e estouro do tipo inteiro.

```
for (int k = 1; k <= n; k++) {
    for (int i = 1; i <= n; i++) {
        if (distancia[i][k] == INT64_MAX)
            continue;

        for (int j = 1; j <= n; j++) {
            if (distancia[k][j] == INT64_MAX)
                continue;

            if (distancia[i][k] >
                INT64_MAX - distancia[k][j])
                continue;

            int64_t nova = distancia[i][k] +
                           distancia[k][j];

            if (nova < distancia[i][j]) {
                distancia[i][j] = nova;
                proximo[i][j] = proximo[i][k];
            }
        }
    }
}
```

Código 29: Atualização das distâncias no Floyd–Warshall.

A verificação de estouro é válida para os pesos não negativos aceitos pelo programa. A cada melhoria, a matriz `proximo` é atualizada para preservar a possibilidade de reconstruir o caminho mínimo.

### 12.3.4 Consulta e apresentação dos resultados

A função `consultarDistancia` retorna a distância mínima entre dois vértices válidos ou `INT64_MAX` quando não existe caminho. A função `imprimirCaminhoFloyd` reconstrói a sequência de vértices

usando a matriz `proximo`. A função `imprimirMatrizDistancias` apresenta todas as distâncias, utilizando `INF` para pares inalcançáveis. Por fim, `destruirResultadoFloyd` libera as matrizes e a estrutura de resultados.

O programa pode ser compilado com o comando `make`, dentro do diretório `tarefa09`. A execução sem argumentos imprime a matriz completa; quando são fornecidos origem e destino, imprime a distância e o caminho correspondente:

```
make
./tarefa09.exe < tests/exemplo.gr
./tarefa09.exe 1 4 < tests/exemplo.gr
./tarefa09.exe 4 1 < tests/exemplo.gr
```

Código 30: Compilação e consultas da Tarefa 9.

## 12.4 Análise de complexidade

### 12.4.1 Complexidade de tempo

A inicialização da matriz de distâncias exige $\Theta(V^2)$ operações e o processamento das arestas exige $O(E)$. O núcleo do algoritmo executa três laços aninhados, cada um com $V$ iterações. Assim, o número de pares examinados para cada intermediário é $V^2$, totalizando $V^3$ verificações:

$$T(V, E) = \Theta(V^2) + O(E) + \Theta(V^3). \tag{98}$$

Como um grafo simples possui no máximo $O(V^2)$ arestas, o termo cúbico domina, resultando em:

$$\boxed{T(V) = \Theta(V^3).} \tag{99}$$

Depois do pré-processamento, a consulta a uma distância armazenada custa $\Theta(1)$. A reconstrução e impressão de um caminho com $L$ vértices custa $O(L)$, sendo $L \leq V$ nos caminhos simples considerados.

### 12.4.2 Complexidade de espaço

As matrizes `distancia` e `proximo` possuem $V \times V$ posições. Portanto, o armazenamento dos resultados exige:

$$\boxed{S(V) = \Theta(V^2).} \tag{100}$$

A representação original do grafo por listas de adjacência utiliza $O(V + E)$ de memória, de modo que o consumo total, incluindo a entrada, é $O(V^2 + E)$.

### 12.4.3 Comparação teórica com Dijkstra

A implementação da Tarefa 4 utiliza Dijkstra com fila de prioridade. Considerando uma fila baseada em heap binário, uma execução para uma origem possui limite superior $O((V + E) \log V)$. Para obter as distâncias de todos os pares, Dijkstra precisa ser executado uma vez por origem, resultando em:

$$T_{\text{Dijkstra repetido}}(V, E) = O\big(V(V + E) \log V\big). \tag{101}$$

Para grafos com densidade constante, $E = \Theta(V^2)$, e o limite superior torna-se $O(V^3 \log V)$. Em contrapartida, grafos esparsos podem favorecer Dijkstra repetido, especialmente quando há poucas arestas em relação ao número de vértices.

## 12.5 Experimentos e resultados

### 12.5.1 Metodologia dos testes

A validação foi realizada por meio de um script Python que executa os programas compilados das Tarefas 9 e 4 de maneira independente. Para cada grafo, o script obtém a matriz completa produzida por Floyd–Warshall e executa Dijkstra uma vez para cada vértice de origem. As distâncias são então comparadas para todos os pares, incluindo os casos inalcançáveis. A comparação utiliza os valores

numéricos das distâncias, sem exigir que os dois algoritmos reconstruam exatamente a mesma sequência de vértices quando há múltiplos caminhos mínimos.

Foram utilizados dois conjuntos de instâncias sintéticas direcionadas: 15 grafos com 5, 10, 15, 20 e 25 vértices, e 12 grafos com 50, 100, 200 e 500 vértices. Em ambos os conjuntos foram consideradas densidades de 20%, 50% e 80%. Os grafos maiores foram gerados com pesos inteiros positivos entre 1 e 100 e sementes pseudoaleatórias fixas.

Para cada instância pequena foram realizadas 30 repetições das medições; para cada instância maior, três repetições. Os tempos foram medidos internamente pelos programas em C, com `clock()`, sem incluir a leitura do arquivo de entrada ou a impressão dos caminhos. No caso de Dijkstra, somaram-se os tempos de suas $V$ execuções, uma para cada origem. Os executáveis imprimem os tempos com resolução de microssegundos, o que limita a precisão das medições muito pequenas.

### 12.5.2 Validação da correção

Nos 15 grafos pequenos foram comparadas 4.125 distâncias. Nos 12 grafos maiores foram comparadas 907.500 distâncias. Ao todo, 911.625 pares foram verificados, sem nenhuma divergência entre Floyd–Warshall e Dijkstra. Esse resultado constitui evidência de correção para as instâncias testadas, embora não substitua uma demonstração formal para todas as entradas possíveis.

No teste funcional com quatro vértices, o menor caminho do vértice 1 ao vértice 4 apresentou custo 6 e sequência $1 \to 2 \to 3 \to 4$. A consulta do vértice 4 ao vértice 1 retornou distância infinita, indicando corretamente a inexistência de caminho.

### 12.5.3 Resultados de desempenho

A Tabela 25 apresenta os tempos médios observados nos grafos maiores, em segundos. A coluna de Dijkstra corresponde à soma dos tempos necessários para processar todas as origens do grafo, tornando o objetivo computacional comparável ao do Floyd–Warshall.

Tabela 25: Tempos médios para o cálculo de caminhos mínimos entre todos os pares (segundos).

| Densidade | Vértices | Floyd–Warshall | Dijkstra repetido |
|---|---|---|---|
| 20% | 50 | 0,000180 | 0,000442 |
| 20% | 100 | 0,001209 | 0,002963 |
| 20% | 200 | 0,007769 | 0,023951 |
| 20% | 500 | 0,131219 | 1,323121 |
| 50% | 50 | 0,000261 | 0,000958 |
| 50% | 100 | 0,001339 | 0,007193 |
| 50% | 200 | 0,008490 | 0,113260 |
| 50% | 500 | 0,125914 | 4,300599 |
| 80% | 50 | 0,000197 | 0,001043 |
| 80% | 100 | 0,001512 | 0,013555 |
| 80% | 200 | 0,009781 | 0,260439 |
| 80% | 500 | 0,127403 | 7,538538 |

Para os grafos com 500 vértices, Floyd–Warshall apresentou tempos próximos de 0,13 segundo nas três densidades. Já o Dijkstra repetido apresentou tempos de aproximadamente 1,32, 4,30 e 7,54 segundos nas densidades de 20%, 50% e 80%, respectivamente. Assim, nessas instâncias, o tempo de Dijkstra repetido foi aproximadamente 10,08, 34,16 e 59,17 vezes o tempo de Floyd–Warshall.

### 12.5.4 Discussão dos resultados

Os resultados mostram que Floyd–Warshall foi mais rápido em todas as instâncias avaliadas para o cálculo das distâncias entre todos os pares. O aumento da densidade elevou substancialmente o custo das execuções sucessivas de Dijkstra, uma vez que mais arestas precisam ser examinadas. O tempo de Floyd–Warshall variou menos com a densidade, pois seus três laços principais dependem diretamente da quantidade de vértices.

Esse comportamento é compatível com as análises assintóticas apresentadas, mas não implica superioridade universal de Floyd–Warshall. Em grafos muito esparsos, ou quando se deseja calcular

caminhos a partir de apenas uma origem, Dijkstra pode ser mais apropriado. Além disso, Floyd–Warshall exige memória quadrática para armazenar as distâncias de todos os pares.

Os experimentos apresentam limitações: foi utilizado um grafo por combinação de tamanho e densidade, as instâncias possuem pesos não negativos, os grafos maiores foram medidos apenas três vezes e os tempos dos programas foram impressos com seis casas decimais. Dessa forma, as diferenças observadas devem ser interpretadas como resultados das implementações e instâncias utilizadas, não como uma garantia de desempenho em todos os cenários.

## 12.6 Considerações da tarefa

A Tarefa 9 permitiu aplicar programação dinâmica ao problema de caminhos mínimos entre todos os pares de vértices. A formulação por subproblemas, baseada na inclusão progressiva de vértices intermediários, possibilitou implementar Floyd–Warshall com complexidade de tempo $\Theta(V^3)$ e espaço $\Theta(V^2)$. A matriz auxiliar `proximo` também permitiu reconstruir os caminhos mínimos, além de consultar suas distâncias.

A comparação com a implementação de Dijkstra da Tarefa 4 confirmou a igualdade das distâncias em todos os pares testados. Nos experimentos realizados, Floyd–Warshall apresentou menores tempos para calcular todas as distâncias, sobretudo nos grafos densos. Entretanto, a escolha entre os algoritmos depende do objetivo da consulta: Dijkstra é adequado para uma origem específica, enquanto Floyd–Warshall é conveniente quando são necessárias muitas consultas entre diferentes pares de vértices após um único pré-processamento.

# 13 Análise Comparativa Final

## 13.1 Objetivos e critérios de comparação

As Tarefas 2 a 9 abordaram o problema dos caminhos mínimos em grafos direcionados e ponderados, utilizando diferentes estruturas de dados e estratégias algorítmicas. Esta seção reúne os resultados já apresentados e discute a relação entre custo computacional, qualidade das soluções e consumo de memória.

As comparações devem considerar o problema efetivamente resolvido. Dijkstra, busca completa, backtracking, estratégia gulosa local e Simulated Annealing foram avaliados principalmente em consultas envolvendo uma origem ou um par origem–destino. Floyd–Warshall, por sua vez, calcula as distâncias entre todos os pares de vértices. Por isso, a comparação direta de tempos entre Floyd–Warshall e Dijkstra somente é adequada quando Dijkstra é executado uma vez para cada origem. Além disso, os experimentos das tarefas utilizaram conjuntos de instâncias, números de repetições e métricas parcialmente distintos. Os valores reunidos a seguir não constituem uma classificação universal dos algoritmos.

## 13.2 Comparação teórica das estratégias

Na Tabela 26, $V$ representa a quantidade de vértices, $E$ a quantidade de arestas, $P$ a quantidade de caminhos simples examinados, $I$ o número de iterações da metaheurística e $B$ o limite de chamadas utilizado na construção de caminhos da Tarefa 8. As complexidades referem-se às implementações discutidas nas tarefas e, quando aplicável, desconsideram o armazenamento comum do grafo.

Tabela 26: Comparação teórica dos algoritmos de caminhos mínimos.

| **Algoritmo** | **Tempo assintótico** | **Espaço auxiliar** | **Ótimo?** |
|---|---|---|---|
| Dijkstra – vetores | $O(V^2 + E)$ | $O(V)$ | Sim |
| Dijkstra – listas | $O(V^2 + E)$ | $O(V)$ | Sim |
| Dijkstra – heap | $O((V + E) \log V)$ | $O(V)$ | Sim |
| Busca completa | Exponencial/fatorial no pior caso | $O(V)$ | Se concluir |
| Backtracking com podas | Exponencial/fatorial no pior caso | $O(V)$, sem pré-processamento | Se concluir |
| Guloso local | $O(V + E)$, conforme a implementação | $O(V)$ | Não |
| Simulated Annealing | $O((I+1)B(V+E))$ (limite conservador) | $O(V + E)$, total | Não |
| Floyd–Warshall | $\Theta(V^3)$ | $\Theta(V^2)$ | Sim |

A garantia de otimalidade de Dijkstra pressupõe pesos não negativos. A busca completa e o backtracking com podas seguras encontram o ótimo se a exploração necessária for concluída; interrupções por limite de tempo não permitem afirmar a mesma garantia. Floyd–Warshall também produz distâncias ótimas quando não existem ciclos negativos que tornem indefinido o menor caminho. Nas instâncias deste trabalho foram utilizados pesos não negativos.

## 13.3 Impacto da estrutura de dados em Dijkstra

As Tarefas 2, 3 e 4 permitiram isolar o efeito da estrutura utilizada para selecionar o próximo vértice. A Tabela 27 reproduz os tempos médios documentados para as instâncias de Roma e Nova York.

Tabela 27: Tempos médios das três implementações de Dijkstra (s).

| **Implementação** | **Roma** | **Nova York** |
|---|---|---|
| Vetores | 0,0083616 | 44,0240868 |
| Listas encadeadas | 0,0086466 | 42,0207908 |
| Heap binário | 0,0004386 | 0,0524988 |

O heap binário foi a implementação mais rápida nas duas instâncias. Em Nova York, apresentou aceleração de aproximadamente 838,57 vezes em relação à versão com vetores e 800,41 vezes em relação à versão com listas encadeadas. O resultado é compatível com a substituição da seleção linear do vértice de menor distância por operações de fila de prioridade com custo logarítmico. A representação por listas de adjacência, isoladamente, não eliminou o custo da seleção linear.

## 13.4 Busca completa e impacto das podas

A busca completa enumera caminhos simples entre origem e destino, o que pode provocar crescimento combinatório do número de chamadas recursivas. Nos testes comparativos da Tarefa 5, encontrou os mesmos custos que Dijkstra nos grafos de 5, 10 e 15 vértices. Já nas instâncias de 20 e 25 vértices, a execução foi interrompida e os custos encontrados foram superiores aos ótimos. Portanto, a conclusão da busca é uma condição essencial para a garantia de otimalidade.

Na Tarefa 6, foram avaliadas duas estratégias de poda: uma baseada no custo acumulado e outra que também utiliza um limite inferior obtido com Dijkstra. A Tabela 28 apresenta exemplos da redução de chamadas recursivas registrada em relação à busca completa.

Tabela 28: Redução das chamadas recursivas com podas.

| $V$ | **Densidade** | **Poda 1** | **Podas 1+2** |
|---|---:|---:|---:|
| 10 | 20% | 46,6667% | 66,6667% |
| 15 | 20% | 95,2381% | 97,6190% |
| 20 | 20% | 99,9714% | 99,9893% |
| 15 | 50% | 99,9993% | 99,9999% |
| 20 | 50% | 99,9994% | 99,9999% |

A redução do número de chamadas não implica necessariamente redução proporcional do tempo. O cálculo do limite inferior introduz um custo adicional de pré-processamento, que pode superar o benefício das podas em instâncias pequenas. Além disso, quando a busca completa é interrompida por tempo, a contagem de chamadas representa somente a parte explorada.

## 13.5 Qualidade das soluções aproximadas

A Tarefa 7 demonstrou que selecionar localmente a aresta de menor peso não equivale ao critério de Dijkstra, que considera a menor distância acumulada desde a origem. Nas 15 instâncias avaliadas, a estratégia gulosa local encontrou quatro soluções ótimas, nove soluções subótimas e deixou de alcançar o destino em duas instâncias, apesar de existir caminho.

O contraexemplo da Tarefa 7 evidencia a limitação: o caminho $1 \to 2 \to 4$, escolhido pela regra local, tem custo 101, enquanto $1 \to 3 \to 4$ tem custo 8. O erro relativo da solução gulosa é de 1162,5%.

O Simulated Annealing da Tarefa 8 obteve resultados melhores em termos de qualidade: das 450 execuções, 446 encontraram o custo ótimo e quatro retornaram soluções subótimas, resultando em taxa de otimalidade observada de 99,11%. Entretanto, seus tempos foram superiores aos de Dijkstra nas instâncias comparadas. A metaheurística não oferece garantia de otimalidade, e seu comportamento depende das sementes pseudoaleatórias, do resfriamento, do número de iterações e da vizinhança utilizada. Sua aplicação torna-se mais interessante em variantes de otimização com restrições adicionais ou espaços de busca para os quais não exista um algoritmo exato tão eficiente quanto Dijkstra.

## 13.6 Floyd–Warshall e consultas entre todos os pares

A Tarefa 9 comparou Floyd–Warshall com a execução de Dijkstra com heap uma vez para cada origem, de modo que ambos resolvessem o problema de distâncias entre todos os pares. Foram validadas 911.625 distâncias, sem divergências, em 27 instâncias sintéticas.

Floyd–Warshall foi mais rápido em todas as instâncias desse experimento, especialmente nos grafos densos. Entretanto, seu custo $\Theta(V^3)$ e o armazenamento de duas matrizes $V \times V$ podem torná-lo inadequado para grafos muito grandes. Para consultas a partir de uma única origem, Dijkstra com heap evita o processamento desnecessário de todos os pares e tende a ser mais apropriado.

Tabela 29: Comparação para todos os pares em grafos de 500 vértices.

| Densidade | Floyd (s) | Dijkstra repetido (s) | Razão |
|---|---|---|---|
| 20% | 0,131219 | 1,323121 | 10,08 |
| 50% | 0,125914 | 4,300599 | 34,16 |
| 80% | 0,127403 | 7,538538 | 59,17 |

## 13.7 Consumo de memória e número de operações

Entre as estratégias avaliadas, Floyd–Warshall se destaca pelo espaço auxiliar quadrático, $\Theta(V^2)$, utilizado nas matrizes de distâncias e de reconstrução de caminhos. Nas implementações de Dijkstra, o armazenamento do grafo por listas de adjacência exige $O(V + E)$, enquanto os vetores auxiliares e a fila de prioridade utilizam espaço proporcional a $V$. As buscas recursivas também utilizam pilha de chamadas proporcional à profundidade do caminho. Assim, Floyd–Warshall apresenta o maior crescimento assintótico de memória auxiliar entre as implementações comparadas, embora não tenham sido realizadas medições padronizadas do pico de memória de todos os executáveis.

A expressão "menor número de operações" depende da métrica adotada. Na Tarefa 4, foram contabilizadas operações do heap; na busca completa e no backtracking, chamadas recursivas; e, na Tarefa 9, o trabalho principal decorre dos três laços aninhados. Essas contagens não são diretamente equivalentes. Para uma comparação quantitativa rigorosa, seria necessário definir e instrumentar operações comparáveis, como relaxamentos de arestas, comparações de custos e expansões de estados, distinguindo os diferentes objetivos computacionais.

## 13.8 Análise de desempenho e possíveis gargalos

Os resultados permitem identificar gargalos prováveis, mas não substituem uma análise de profiling instrumentalizada. Nas versões de Dijkstra com vetores e listas, o principal candidato a gargalo é a seleção sequencial do próximo vértice. No Dijkstra com heap, os custos relevantes incluem operações de atualização e remoção da fila de prioridade, além do percurso das listas de adjacência. Na busca completa, o principal fator de crescimento é a quantidade de caminhos simples; no backtracking, o custo adicional de cálculo dos limites inferiores deve ser comparado à economia de chamadas recursivas. No Simulated Annealing, a construção e avaliação repetida de caminhos candidatos merece investigação. No Floyd–Warshall, os três laços e os acessos às matrizes concentram o trabalho computacional.

Como melhorias e experimentos complementares, recomenda-se compilar versões específicas para profiling, medir o tempo por função e o pico de memória, além de contabilizar operações homogêneas. No Linux ou WSL, ferramentas como `gprof`, `perf` e `/usr/bin/time -v` podem auxiliar nessa avaliação, quando disponíveis. É importante distinguir resultados medidos de hipóteses de gargalo e utilizar as mesmas opções de compilação e entradas para cada comparação.

## 13.9 Síntese da análise comparativa

Os resultados indicam que a estrutura de dados é decisiva para o desempenho de Dijkstra: a implementação com heap foi significativamente superior às versões com seleção linear nas instâncias de Roma e Nova York. A busca completa e o backtracking preservam a capacidade de encontrar o ótimo quando a exploração é concluída, mas o crescimento combinatório limita sua escalabilidade; as podas reduzem o esforço, embora possam acrescentar custos de pré-processamento. A estratégia gulosa local apresenta baixa confiabilidade quanto à qualidade da solução, enquanto o Simulated Annealing alcançou elevada taxa de otimalidade experimental sem garantir o resultado ótimo e com custo superior ao de Dijkstra. Para consultas de uma única origem em grafos com pesos não negativos, Dijkstra com heap mostrou-se a alternativa mais adequada entre as implementações avaliadas. Para o cálculo de distâncias entre todos os pares nos grafos densos estudados, Floyd–Warshall apresentou melhor desempenho que Dijkstra repetido, ao custo de maior uso assintótico de memória auxiliar.

# 14 Considerações Finais

O desenvolvimento deste trabalho permitiu aplicar e analisar diferentes técnicas de projeto de algoritmos, relacionando os conceitos teóricos estudados na disciplina PCC104 – Projeto e Análise de Algoritmos com sua implementação prática na linguagem C. Ao longo das nove tarefas, foram exploradas estratégias como divisão e conquista, algoritmos gulosos, busca completa, backtracking, metaheurísticas e programação dinâmica, com ênfase na análise de complexidade, na correção das soluções e no desempenho computacional.

Na primeira tarefa, a implementação da multiplicação recursiva de matrizes possibilitou compreender a técnica de divisão e conquista e o impacto da decomposição de problemas em subproblemas menores. O tratamento de matrizes de ordens arbitrárias por meio do preenchimento com zeros também demonstrou a importância de adaptar uma solução teórica para atender às condições reais de entrada.

Nas tarefas relacionadas ao problema do caminho mínimo, foi possível observar como diferentes estratégias podem resolver um mesmo problema com custos computacionais significativamente distintos. As implementações do algoritmo de Dijkstra evidenciaram a influência das estruturas de dados sobre o desempenho. Em particular, a utilização de uma fila de prioridade baseada em heap binário apresentou vantagens expressivas em determinadas instâncias, especialmente quando comparada às implementações que realizam buscas lineares para selecionar o próximo vértice.

A busca completa e o backtracking permitiram investigar o custo da exploração de diferentes caminhos possíveis. Embora essas abordagens possam encontrar a solução ótima quando a busca é concluída, seu crescimento combinatório limita a aplicação em instâncias maiores. Nesse contexto, as técnicas de poda empregadas no backtracking demonstraram ser fundamentais para reduzir a quantidade de chamadas recursivas e evitar a exploração de alternativas que não poderiam melhorar a solução encontrada.

Por outro lado, a estratégia gulosa local e a metaheurística Simulated Annealing evidenciaram a relação entre eficiência computacional e qualidade das soluções. A abordagem gulosa apresentou limitações decorrentes da tomada de decisões exclusivamente locais, enquanto o Simulated Annealing permitiu explorar alternativas de maneira probabilística. Os resultados experimentais mostraram que a metaheurística conseguiu encontrar soluções ótimas em grande parte das execuções avaliadas, embora não exista garantia de otimalidade para uma execução com recursos limitados.

A implementação do algoritmo de Floyd–Warshall, na última tarefa, demonstrou a aplicação da programação dinâmica ao cálculo dos caminhos mínimos entre todos os pares de vértices. A comparação experimental com execuções repetidas do algoritmo de Dijkstra confirmou a equivalência dos custos calculados nas instâncias testadas e mostrou vantagens de desempenho para Floyd–Warshall nos grafos sintéticos avaliados. Entretanto, seu custo de memória quadrático representa uma limitação importante para grafos de grande dimensão.

A análise comparativa final reforçou que não existe uma única estratégia mais adequada para todas as situações. A escolha de um algoritmo depende de fatores como a quantidade de vértices e arestas, a densidade do grafo, a necessidade de obter uma solução ótima, o consumo de memória e o tipo de consulta desejada. Além disso, os experimentos evidenciaram a importância de considerar não apenas a complexidade assintótica, mas também as características da implementação e das entradas utilizadas.

Como limitações do trabalho, destacam-se a utilização de conjuntos específicos de instâncias, as diferenças entre os procedimentos experimentais de algumas tarefas e a ausência de medições detalhadas de consumo de memória e de profiling para todas as implementações. Como possibilidades de continuidade, sugere-se ampliar os testes com instâncias de diferentes dimensões e densidades, padronizar as medições de desempenho e investigar os principais gargalos por meio de ferramentas de análise de execução. Também seria interessante avaliar outras estruturas de dados e diferentes configurações para os algoritmos aproximativos.

Por fim, o trabalho contribuiu para consolidar conhecimentos sobre estruturas de dados, recursividade, análise de complexidade, otimização e experimentação computacional. A implementação e a comparação das diferentes estratégias demonstraram a importância de compreender tanto os fundamentos matemáticos dos algoritmos quanto os aspectos práticos que influenciam sua eficiência, constituindo uma experiência relevante para a formação em Ciência da Computação.

# Referências


CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. **Introduction to Algorithms**. 4. ed. Cambridge, MA: MIT Press, 2022.

DIJKSTRA, Edsger W. A note on two problems in connexion with graphs. **Numerische Mathematik**, v. 1, p. 269–271, 1959. Disponivel em: ⟨https://doi.org/10.1007/BF01386390⟩. Acesso em: 13 nov. 2026.

FLOYD, Robert W. Algorithm 97: Shortest Path. **Communications of the ACM**, v. 5, n. 6, p. 345, 1962. Disponivel em: ⟨https://doi.org/10.1145/367766.368168⟩. Acesso em: 13 nov. 2026.

WARSHALL, Stephen. A theorem on Boolean matrices. **Journal of the ACM**, v. 9, n. 1, p. 11–12, 1962. Disponivel em: ⟨https://doi.org/10.1145/321105.321107⟩. Acesso em: 13 nov. 2026.

KIRKPATRICK, Scott; GELATT, C. Daniel; VECCHI, Mario P. Optimization by Simulated Annealing. **Science**, v. 220, n. 4598, p. 671–680, 1983. Disponivel em: ⟨https://doi.org/10.1126/science.220.4598.671⟩. Acesso em: 13 nov. 2026.

MASSACHUSETTS INSTITUTE OF TECHNOLOGY (MIT). **Introduction to Algorithms (6.006): Lecture Videos**. MIT OpenCourseWare, 2020. Disponivel em: ⟨https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/resources/lecture-videos/⟩. Acesso em: 13 nov. 2026.

MASSACHUSETTS INSTITUTE OF TECHNOLOGY (MIT). **Introduction to Algorithms (6.006): Lecture Notes**. MIT OpenCourseWare, 2008. Disponivel em: ⟨https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2008/pages/lecture-notes/⟩. Acesso em: 13 nov. 2026.

CP-ALGORITHMS. **Floyd-Warshall: finding all shortest paths**. [S. l.], [s. d.]. Disponivel em: ⟨https://cp-algorithms.com/graph/all-pair-shortest-path-floyd-warshall.html⟩. Acesso em: 13 nov. 2026.

SEDGEWICK, Robert; WAYNE, Kevin. **Shortest Paths**. Algorithms, 4th Edition. Princeton University, [s. d.]. Disponivel em: ⟨https://algs4.cs.princeton.edu/44sp/⟩. Acesso em: 13 nov. 2026.

YOUTUBE. **Videoaula sobre algoritmos**. [S. l.], [s. d.]. Disponivel em: ⟨https://www.youtube.com/watch?v=aa57NTTahrQ⟩. Acesso em: 13 nov. 2026.

FREECODECAMP.ORG. **Graph Theory Algorithms**. YouTube, 2019. Disponivel em: ⟨https://www.youtube.com/watch?v=09_LlHjoEiY⟩. Acesso em: 13 nov. 2026.

YOUTUBE. **Videoaula sobre metaheuristicas**. [S. l.], [s. d.]. Disponivel em: ⟨https://www.youtube.com/watch?v=2_pSh-D5Rvc⟩. Acesso em: 13 nov. 2026.

YOUTUBE. **Videoaula sobre metaheuristicas**. [S. l.], [s. d.]. Disponivel em: ⟨https://www.youtube.com/watch?v=AEeYp5VtI08⟩. Acesso em: 13 nov. 2026.

YOUTUBE. **Videoaula sobre metaheuristicas**. [S. l.], [s. d.]. Disponivel em: ⟨https://www.youtube.com/watch?v=5ElAUPABh6U⟩. Acesso em: 13 nov. 2026.